Optimization

Muchang Bahng

Spring 2025

Contents

Definition 0.1 (Function Class) . . . 4
Definition 0.2 (Oracle) . . . 4
Example 0.1 (First Order Oracle) . . . 4
Definition 0.3 (Algorithm) . . . 5
Definition 0.4 (Gradient Descent) . . . 5
Definition 0.5 (Performance Criterion) . . . 5
Example 0.2 (First Order Stationarity) . . . 6
Definition 0.6 (ϵ-Critical Point) . . . 6
Example 0.3 (Function Suboptimality) . . . 6
Example 0.4 (Distance to the Minimizer) . . . 6
Definition 0.7 (Worst-Case Iteration Complexity) . . . 6
Definition 0.8 (Minimax Oracle Error) . . . 7

1 Linear Programming **8**
1.1 The Duality Theorem . . . 8
Lemma 1.1 (Dual Formulation of Span) . . . 8
Theorem 1.2 (Separating Hyperplane Theorem) . . . 8
Theorem 1.3 (Dual Formulation of Positive Span) . . . 9
Corollary 1.4 (Farkas' Lemma, Equality Form) . . . 12
Theorem 1.5 (Farkas' Lemma, Inequality Form) . . . 12
Definition 1.1 (Primal and Dual Feasibility) . . . 13
Example 1.1 (Manual Derivation of Dual from Primal) . . . 14
Lemma 1.6 (Dual of Dual is Primal) . . . 14
Lemma 1.7 (Weak Duality) . . . 15
Theorem 1.8 (Strong Duality) . . . 15
Corollary 1.9 (Primal-Dual Optimality Criterion) . . . 16
Example 1.2 (Production Planning and Resource Pricing) . . . 17
Example 1.3 (Primal Feasible, Dual Infeasible) . . . 18
Example 1.4 (Both Infeasible) . . . 18
1.2 Complementary Slackness and Sensitivity . . . 19
Theorem 1.10 (Complementary Slackness) . . . 19
Example 1.5 (Complementary Slackness for Furniture, No Slack) . . . 20
Theorem 1.11 (Dual Variables are Maringal/Shadow Prices) . . . 20
1.3 Geometry of Linear Programs . . . 21
Definition 1.2 (Extreme Point, Basic Feasible Solution) . . . 21
Lemma 1.12 (Extreme Points are Basic Feasible Solutions) . . . 21
Theorem 1.13 (Fundamental Theorem of Linear Programming) . . . 21
1.4 The Simplex Method . . . 22
Theorem 1.14 (Optimality Test) . . . 22
Example 1.6 (Simplex on the Furniture Problem) . . . 23
1.5 Interior-Point Methods and the Lagrangian View . . . 24

2 Smooth Optimization **25**
Definition 2.1 (L-Smooth) . . . 25
Lemma 2.1 (Properties of L-Smoothness) . . . 25
Theorem 2.2 (Taylor's 1st Order Bound) . . . 25
Lemma 2.3 (Gradient Descent on L-Smooth Functions) . . . 26
Lemma 2.4 (Descent Lemma) . . . 27
Example 2.1 (Descent Lemma on a Nonconvex L-Smooth Function) . . . 28

Theorem 2.5 (Convergence to Criticality of GD) . . . 29
Example 2.2 (Reaching an ϵ-Critical Point) . . . 29
Example 2.3 (A Near-Extremal Function for the Criticality Bound) . . . 30
2.1 Convexity . . . 31
Theorem 2.6 (Convergence of Gradient Descent for Smooth Convex Functions) . . . 31
Lemma 2.7 (Unique Minimizer and Quadratic Growth of Strongly Convex Functions) . . . 32
Example 2.4 (Ridge Loss is Strongly Convex) . . . 33
Lemma 2.8 (L-Smooth and μ-Strongly Convex) . . . 33
Theorem 2.9 (Linear Convergence of Objective Value on GD Under Strong Convexity) . . . 33
Corollary 2.10 (Linear Convergence of Distance to Minimizer on GD Under Strong Convexity) . . . 34
Definition 2.2 (Condition Number) . . . 35
Example 2.5 (Optimization on Quadratics with Varying Condition Numbers) . . . 35
Theorem 2.11 (Convergence Lower Bound of GD on Strongly Convex Functions) . . . 36
2.2 Polyak-Lojasiewicz Inequality . . . 39
Definition 2.3 (Polyak-Lojasiewicz Inequality) . . . 39
Lemma 2.12 (Strong Convexity Implies PL-Inequality) . . . 39
Example 2.6 (PL and non-PL Functions) . . . 40
Example 2.7 (μ-PL vs μ-Strongly Convex) . . . 40
Lemma 2.13 (Properties of the PL-Inequality) . . . 41
Theorem 2.14 (Linear Convergence of Objective Value on GD Under PL Inequality) . . . 42
Corollary 2.15 (One-Shot Minimum for GD) . . . 42
Theorem 2.16 (Linear Convergence of GD in Overparameterized Linear Regression) . . . 43
Theorem 2.17 (Convergence Lower Bound of GD on Strongly Convex Functions) . . . 43
Theorem 2.18 (Convergence Lower Bound of GD on μ-PL Functions[YFL23]) . . . 44
Example 2.8 (Implicit Regularization) . . . 44
Lemma 2.19 (Limit of the Ridge Path) . . . 44
Theorem 2.20 (Implicit Bias) . . . 45
2.3 Momentum . . . 45
Definition 2.4 (Gradient Descent with Momentum) . . . 45
Theorem 2.21 (Momentum GD is Optimal for Quadratics [Pol64]) . . . 46
Theorem 2.22 (Suboptimality of Heavy Ball for General Smooth Functions) . . . 46
Definition 2.5 (Nesterov's Auxiliary Estimation Function) . . . 47
Lemma 2.23 (One-Step Estimate) . . . 48
Definition 2.6 (Nesterov Accelerated Gradient Descent) . . . 50
Theorem 2.24 (Convergence) . . . 50
2.4 Preconditioning . . . 51
Definition 2.7 (Mahalanobis Norm) . . . 51
Definition 2.8 (Preconditioned Gradient Descent) . . . 51
Definition 2.9 (L_P-Smooth) . . . 51
Definition 2.10 (μ_P-Strongly Convex) . . . 51
Definition 2.11 (μ_P-PL) . . . 52
Lemma 2.25 (Scale Change) . . . 52
Theorem 2.26 (Convergence to Criticality of PGD) . . . 52
Theorem 2.27 (Linear Convergence of Objective Value on PGD Under PL Inequality) . . . 52
Example 2.9 (Quadratic) . . . 53
2.5 Saddle Points . . . 53
Definition 2.12 (Strict Saddle) . . . 53
Example 2.10 . . . 53
Theorem 2.28 (Escaping Strict Saddles) . . . 53
Lemma 2.29 (A Three-Point Identity for Smooth Functions) . . . 56
Lemma 2.30 (Lipschitz Maps Preserve Null Sets) . . . 56

3 Nonsmooth Optimization **57**
3.1 Convex Conjugacy . . . 57
Definition 3.1 (Proper Functions) . . . 57
Definition 3.2 (Convex, Fenchel Conjugate) . . . 57
Theorem 3.1 (Fenchel-Young Inequality) . . . 57
Definition 3.3 (Subgradients, Subdifferentials) . . . 57
Example 3.1 (Subgradients of Absolute Value Function) . . . 57
Theorem 3.2 (Equality in Fenchel-Young) . . . 58
Definition 3.4 (Biconjugate) . . . 58
Corollary 3.3 (Affine Lower Bound Reconstruction) . . . 58
Theorem 3.4 (Fenchel-Moreau) . . . 59
Theorem 3.5 (Reciprocity of Subgradients) . . . 59
3.2 Subgradient Descent . . . 59
Definition 3.5 (Weak Subgradient Oracle) . . . 59

Definition 3.6 (Subgradient Descent) . . . 59
Example 3.2 (Subgradients of $|w_1| + 2|w_2|$) . . . 60
Theorem 3.6 (Convergence of Subgradient Descent) . . . 61
3.3 Proximal Methods . . . 61
Definition 3.7 (Proximal Operator) . . . 61

4 Second-Order Optimizers **63**
Theorem 4.1 (Newton-Raphson Method) . . . 63
4.1 Newton's Method . . . 63
Theorem 4.2 (Taylor's 2nd Order Bound) . . . 63
Definition 4.1 (Netwon's Method) . . . 64
Lemma 4.3 (Affine Invariance) . . . 65
Theorem 4.4 (Quadratic Convergence) . . . 65
4.2 BFGS . . . 66
4.3 Davidon-Fletcher-Powell (DFP) . . . 69
4.4 Broyden's Method . . . 70
4.5 Symmetric Rank-1 Update (SR1) . . . 70

5 Constrained Optimization **72**
5.1 Lagrangian Duality and KKT Conditions . . . 72
5.2 Augmented Lagrangian Methods . . . 72
5.3 Projected Gradient Descent . . . 72
5.4 Interior Point Methods . . . 72

6 Stochastic Optimization **73**
Example 6.1 (Coordinate Descent) . . . 73
Example 6.2 (Finite Sums) . . . 73
Example 6.3 (Stochastic Programming) . . . 73
Theorem 6.1 (Expected Descent) . . . 74
Theorem 6.2 (Minibatch Gradient is an Unbiased Estimator) . . . 75
Example 6.4 (Linear Regression) . . . 75
Heuristic 6.1 (Choosing Batch Size) . . . 76
6.1 Convergence . . . 77
6.2 Momentum and Adaptive Methods . . . 77
6.3 Variance Reduction . . . 77
6.4 Proximal Methods . . . 77
6.5 Block Coordinate Descent . . . 77

7 Gradient Free Methods **79**
7.1 Simulated Annealing . . . 79
7.2 Nelder-Mead . . . 79

References **80**

Optimization is such an important tool that it deserves a set of notes in itself. All problems in model training essentially stems from non-ideal optimization. Knowing the strengths and weaknesses of each optimizer allows you to diagnose which ones to use.

Generally, we (non-exclusively) categorize optimization algorithms as such:

1. *Convex*? Convex optimization is pretty easy to solve and has been studied extensively. For nonconvex optimization, none of the algorithms can guarantee that we will find the global minima, and this is one of the hardest problems in statistics.[1]
2. *Constrained*? Is the parameter space constrained to a certain manifold?
3. *Order*. Do we use derivatives at all? First-order derivatives (gradient)? Second-order (Hessian)?

These algorithms try to solve the following potential problems.

1. *Convergence*. Do we converge to some point?
2. *Optimality*. Is this point close to the true global minima?
3. *Efficiency*. Can we iterate efficiently?

In optimization, we are given a function $f : S \subset \mathbb{R}^d \to \mathbb{R}$, and we want to find

$$\inf_{w \in S} f \tag{1}$$

Ideally, S and f would have some nice properties such that the minimizer actually exists, and we can actually find $w^* = \text{argmin}_{w \in S} f$ but this may not be required all the time.

Definition 0.1 (Function Class)

Given some domain $S \subset \mathbb{R}$, a **function class** is some set of functions

$$\mathcal{F} \subset \{f : S \to \mathbb{R}\} \tag{2}$$

Given a function $f \in \mathcal{F}$, we would like to formalize extracting some information out of this function, such as its value, derivative, or whether it is continuous at a point. We can describe this in terms of an oracle.

Definition 0.2 (Oracle)

Let $\mathcal{Y}$ be an arbitrary space.[a] An **oracle**

1. for a function $f : S \to \mathbb{R}$ is a map
$$\mathcal{O}_f : S \to \mathcal{Y} \tag{3}$$
2. for a function class $\mathcal{F} = \{f : S \to \mathbb{R}\}$ is a family of oracles
$$\mathcal{O} = \{\mathcal{O}_f\}_{f \in \mathcal{F}}, \quad \mathcal{O}_f : S \to \mathcal{Y} \tag{4}$$

[a] It is a space where the information that we wish to query lives.

Note that an oracle can be defined for a particular function or a function class.

[1] In practice, when we are doing high-dimensional nonconvex optimization, the best we can do is play around with some properties. In these cases, 0th order approximations are hopeless since the dimensions are too high, and second order approximations are hopeless either since computing the Hessian is too expensive for one run. Therefore, we must resort to some first order methods.

Example 0.1 (First Order Oracle)

If $\mathcal{F}$ be the space of functions that are differentiable in an open neighborhood of S. The first-order oracle is

$$\mathcal{O}_f(x) = \big(f(x), \nabla f(x)\big) \tag{5}$$

Therefore, it allows us to query both the value of the function and the gradient at that point. In practice, we do this using *automatic differentiation*.

Definition 0.3 (Algorithm)

Let $\mathcal{F}$ be a function class over $S \subset \mathbb{R}^d$ and fix $w_0 \in S$. A **deterministic K-query oracle algorithm** for $f \in \mathcal{F}$ is an ordered tuple $\mathcal{A}_K = (\Phi_1, \dots, \Phi_{K-1}, \Psi_K)$ consisting of

1. *Query Rules.* A function that decides where the algorithm should evaluate the oracle next.

$$\Phi_k : (S \times \mathcal{Y})^k \to S, \quad k = 1, \dots, K-1 \tag{6}$$

defined recursively as

$$y_k = \mathcal{O}_f(w_k) \tag{7}$$
$$w_{k+1} = \Phi_{k+1}(w_0, y_0, \dots, w_k, y_k) \tag{8}$$

2. *Output Rule.* A function that decides which point to return after collecting K oracle responses.[a]

$$\Psi_K : (S \times \mathcal{Y})^K \to S \tag{9}$$

and the **output** of the algorithm is defined to be

$$w_{\mathcal{A}_K}(f) = \Psi_K(w_0, y_0, \dots, w_{K-1}, y_{K-1}) \tag{10}$$

[a] We can think of K as the number of iterations.

Given a differentiable function, we can use the fact that gradients point in the step of steepest ascent. This inspires a greedy approach to simply walk in this direction $-\nabla f(x)$ to minimize a function. Let's formalize this into an algorithm.

Definition 0.4 (Gradient Descent)

Let $\mathcal{F}$ be the class of differentiable functions $f : S \subset \mathbb{R}^d \to \mathbb{R}$. Fix an initial point $w_0 \in S$ and a sequence of positive step sizes (η_k). Then, **gradient descent** is a deterministic algorithm defined by the first-order oracle

$$\mathcal{O}_f : S \to \mathbb{R} \times \mathbb{R}^d, \quad \mathcal{O}_f(w) = \big(f(w), \nabla f(w)\big) \tag{11}$$

and update maps $\Phi_k : (S \times (\mathbb{R} \times \mathbb{R}^d))^{k+1} \to S$ defined

$$w_{k+1} = \Phi_k\big(w_0, (f(w_0), \nabla f(w_0)), \dots, w_k, (f(w_k), \nabla f(w_k))\big) \coloneqq w_k - \eta_k \nabla f(w_k) \tag{12}$$

After K iterations, the algorithm outputs $w_{\mathcal{A}_K}(f) = w_K$.

Definition 0.5 (Performance Criterion)

Given a function class $\mathcal{F}$ defined over S, a **performance criterion** is a function

$$\mathcal{L} : \mathcal{F} \times S \to [0, +\infty] \tag{13}$$

Sometimes, we write $\mathcal{L}_f(w) = \mathcal{L}(f, w)$. For minimizations, the performance criterion should ideally satisfy $\mathcal{L}_f(w^*) = 0$ for some optimal value w^*.

There are many metrics that we can use to measure optimality, in increasing importance.

1. *Small Gradient Norm.* How small is $\|\nabla f(w)\|$? By first-order optimality conditions, a local minimum must have vanishing gradient, so perhaps bounding the norm gradient gives us a few clues.
2. *Small Objective Gap.* Maybe we wish to bound
$$f(w) - f(w^*) \tag{14}$$
3. *Small Distance to Solution.* Maybe we wish to bound
$$\|w - w^*\| \tag{15}$$

Example 0.2 (First Order Stationarity)

If $\mathcal{F}$ is the class of differentiable functions, then we can define

$$\mathcal{L}_f(w) = \|\nabla f(w)\| \tag{16}$$

Definition 0.6 (ϵ-Critical Point)

For differentiable function $f : \mathbb{R}^d \to \mathbb{R}$, $x \in \mathbb{R}^d$ is an **ϵ-critical point** if

$$\|\nabla f(x)\| \leq \epsilon \tag{17}$$

Example 0.3 (Function Suboptimality)

For a given function f where $f^* = \inf f$ is finite, we can define

$$\mathcal{L}_f(w) = f(w) - f^* \tag{18}$$

Example 0.4 (Distance to the Minimizer)

Assuming a minimizer exists for $f : S \to \mathbb{R}$, let $\operatorname{argmin} f$ be the set of all minimizers. Then, we can define

$$\mathcal{L}_f(w) = d(w, \operatorname{argmin} f) \tag{19}$$

For some metric d defined on S.

What we hope to do is find bounds for all three performance criterions for various algorithms.

To analyze an optimiztion algorithm, we want to see its broad performance over not just one function, but a *class of functions*. For a fixed algorithm, the worst-case iteration complexity is the most popular.

Definition 0.7 (Worst-Case Iteration Complexity)

Given function class $\mathcal{F}$, suppose an algorithm $\mathcal{A}$ generates a sequences of iterates $w_0(f), w_1(f), w_2(f), \dots$ for each $f \in \mathcal{F}$. Define the first iteration at which the algorithm achieves accuracy ϵ as

$$T_\epsilon(A, f) \colon \inf\{k \geq 0 \colon \mathcal{L}_f(w_k(f)) \leq \epsilon\} \tag{20}$$

Then, the

1. **worst-case iteration complexity** is the maximum number of iterations required for algorithm $\mathcal{A}$ to achieve accuracy ϵ over the entire function class $\mathcal{F}$

$$K_\epsilon(A, \mathcal{F}, \mathcal{L}) \coloneqq \sup_{f \in \mathcal{F}} T_\epsilon(A, f) \tag{21}$$

2. **uniform worst-case iteration complexity** is defined

$$\widetilde{K}_\epsilon(A, \mathcal{F}, \mathcal{L}) \coloneqq \inf\left\{k \geq 0 \colon \sup_{f \in \mathcal{F}} \mathcal{L}_f(w_k(f)) \leq \epsilon\right\} \tag{22}$$

Definition 0.8 (Minimax Oracle Error)

Let $\mathcal{F}$ be a function class, $\mathcal{L}$ a performance criterion, and $\mathcal{A}_K(\mathcal{O})$ denote a specified class of algorithms making at most K calls to oracle $\mathcal{O}$. The **minimax oracle error** is the smallest worst case error achievable by any admissible algorithm after K oracle queries.

$$\mathcal{E}_K(\mathcal{F}, \mathcal{O}, \mathcal{L}) \coloneqq \inf_{A \in \mathcal{A}_K(())} \sup_{f \in \mathcal{F}} \mathcal{L}_f\big(w_{\mathcal{A}_K}(f)\big) \tag{23}$$

1 Linear Programming

1.1 The Duality Theorem

A linear program (LP) asks us to optimize a linear function over a set cut out by finitely many linear inequalities. The duality theorem lets us pair every LP (the *primal*) with a second LP (the *dual*) whose optimal value is the same, and it is the source of both optimality certificates and of the simplex method below. The route to it goes through a sequence of "theorems of the alternative" that characterize when a vector lies in the span of some generators, first with arbitrary coefficients (equalities), and then with nonnegative coefficients (inequalities).

Given vectors $v_1, \dots, v_m \in V$, we know that $v \in \text{span}\{v_i\}$ if v can be written as a linear combination of the v_i's. We wish to formulate an equivalent condition in terms of the dual space.

Lemma 1.1 (Dual Formulation of Span)

Let V be a vector space, with $v_1, \dots, v_m \in V$. For any $v \in V$, the following are equivalent.

1. *Primal.* $v \in \text{span}\{v_i\}$
2. *Dual.* For every $\ell \in V^\dagger$ satisfying $\ell(v_i) = 0$ for all i, it is the case that $\ell(v) = 0$.

Proof. We prove bidirectionally.

1. ($\to$). This is easy. Let $v \in \text{span}\{v_i\}$. Then, if $\ell(v_i) = 0$ for all i, then

$$\ell(v) = \ell\left(\sum_{i=1}^m a_i v_i\right) = \sum_{i=1}^m a_i \ell(v_i) = \sum_{i=1}^m 0 = 0 \tag{24}$$

2. ($\leftarrow$). We prove the contrapositive. Let $W = \text{span}\{v_i\}$ and $v \notin W$. Since $W \cap \text{span}\{v\} = \{0\}$, we can take their direct sum $W \oplus \text{span}\{v\}$ and define the linear functional

$$\ell : W \oplus \text{span}\{v\} \to \mathbb{F}, \quad \ell(w + av) = a \tag{25}$$

Then, $\ell \in W^\perp$ and $\ell(v) = 1$. We can extend the domain linearly—which is always possible by extending a basis of $W \oplus \text{span}\{v\}$ to a basis of V—to create a functional $\ell \in V^\dagger$.

Now, we state a theorem that allows us to check if a vector v can be written as a *nonnegative* linear combination of the v_i's. We need the separating hyperplane theorem first.

Theorem 1.2 (Separating Hyperplane Theorem)

Let $C \subset \mathbb{R}^n$ be a nonempty closed convex set, and let $v \in \mathbb{R}^n \setminus C$. Then there exists a nonzero linear functional $\ell \in (\mathbb{R}^n)^\dagger$ and $\alpha \in \mathbb{R}$ such that

$$\ell(v) < \alpha \leq \ell(c) \qquad \forall c \in C. \tag{26}$$

Equivalently, there exists a hyperplane

$$H = \{x \in \mathbb{R}^n : \ell(x) = \alpha\} \tag{27}$$

that strictly separates v from C.

Proof. Since C is nonempty and closed, there exists a point $p \in C$ minimizing the Euclidean distance to v, i.e.

$$\|v - p\| = \inf_{c \in C} \|v - c\|. \tag{28}$$

Furthermore, since $v \notin C$, we have $p \neq v$. For any $c \in C$ and $t \in [0,1]$, convexity implies

$$p + t(c-p) = (1-t)p + tc \in C. \tag{29}$$

By the minimality of p, we have

$$\|v-p\|^2 \leq \|v-p-t(c-p)\|^2. \tag{30}$$

Expanding the right-hand side gives

$$\|v-p\|^2 \leq \|v-p\|^2 - 2t\langle v-p, c-p\rangle + t^2\|c-p\|^2. \tag{31}$$

Subtracting $\|v-p\|^2$ and dividing by $t > 0$,

$$2\langle v-p, c-p\rangle \leq t\|c-p\|^2. \tag{32}$$

Taking $t \to 0^+$, we obtain

$$\langle v-p, c-p\rangle \leq 0 \qquad \forall c \in C. \tag{33}$$

Now define the linear functional

$$\ell : \mathbb{R}^n \to \mathbb{R}, \qquad \ell(x) = \langle p-v, x\rangle. \tag{34}$$

Since $p \neq v$, this functional is nonzero. Furthermore, the preceding inequality implies

$$\ell(c) \geq \ell(p) \qquad \forall c \in C. \tag{35}$$

On the other hand,

$$\begin{aligned} \ell(p) - \ell(v) &= \langle p-v, p-v\rangle && (36)\\ &= \|p-v\|^2 && (37)\\ &> 0. && (38) \end{aligned}$$

Therefore, setting $\alpha = \ell(p)$, we have

$$\ell(v) < \alpha \leq \ell(c) \qquad \forall c \in C, \tag{39}$$

as desired.

Theorem 1.3 (Dual Formulation of Positive Span)

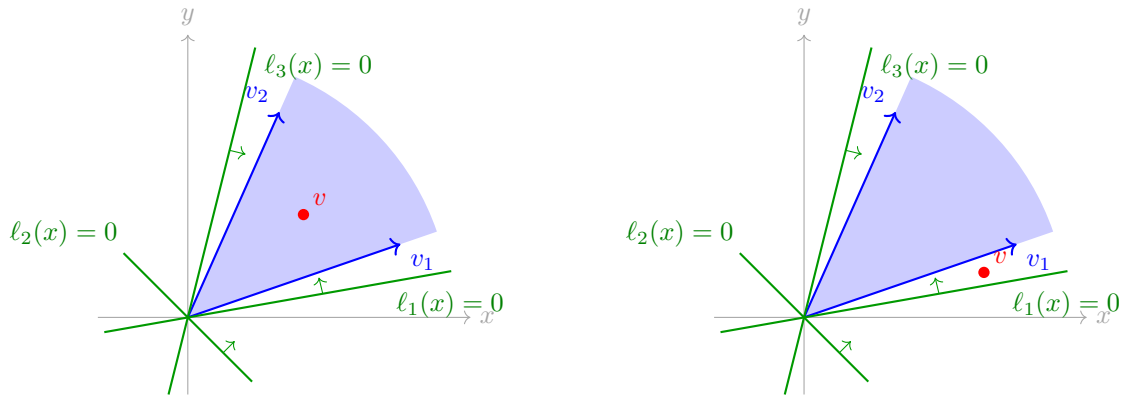
Let $V = \mathbb{R}^n$, with $v_1, \dots, v_m \in V$. For any $v \in V$, the following are equivalent.

1. *Primal.* There exists $a_i \geq 0$, $i = 1, \dots, m$ such that

$$v = \sum_{i=1}^m a_i v_i, \quad a_i \geq 0 \tag{40}$$

2. *Dual.* For every $\ell \in V^\dagger$ satisfying $\ell(v_i) \geq 0$ for all i, it is the case that

$$\ell(v) \geq 0 \tag{41}$$



(a) $v \in \text{cone}(v_1, v_2)$. (b) $v \notin \text{cone}(v_1, v_2)$.

Figure 1: Geometric illustration of the dual characterization of positive span. The green lines represent kernels of linear functionals, with arrows indicating their nonnegative half-planes.

Proof. We prove bidirectionally.

1. ($\to$). Let us have such an $\ell \in V^\dagger$ satisfying $\ell(v_i) \geq 0$. Then,

$$\ell(v) = \ell\left(\sum_{i=1}^m a_i v_i\right) = \sum_{i=1}^m a_i \ell(v_i) \geq 0 \tag{42}$$

since $a_i \geq 0$ and $\ell(v_i) \geq 0$.

2. ($\leftarrow$). We would like to prove the contrapositive. Let

$$C = \left\{ \sum_{i=1}^m a_i v_i \colon a_i \geq 0 \right\} \tag{43}$$

by the cone spanned by v_i. If $v \notin C$, then we have to show there exists a $\ell \in V^\dagger$ satisfying $\ell(v_i) \geq 0$ but $\ell(v) < 0$. From the visual, we basically want to construct a small hyperplane squeezed in between the cone and v such that $\ell(v) < 0$. We note three things.

   (a) C is convex since given two vectors $u = \sum_i a_i v_i$ and $w = \sum_i b_i v_i$, we can see that

$$tu + (1-t)w = \sum_i (ta_i + (1-t)b_i)v_i \in C \tag{44}$$

   (b) C is a cone, i.e. $tu \in C$ for $t \geq 0, u \in C$.

   (c) C is closed. This is the hard part. Note that a point $u \in C$ can have multiple representations $u = \sum_{i=1}^m a_i v_i$, $a_i \geq 0$. We select one minimizing $A = \sum_{i=1}^m a_i$. Such a minimizer exists since given any feasible representation with coefficient sum M, we can restrict the search to

$$S = \left\{ a \in \mathbb{R}^m_{\geq 0} \colon \sum_i a_i v_i = u, \sum_i a_i \leq M \right\} \tag{45}$$

   This is clearly bounded by $[0, M]^m$, and it is closed since we can define the continuous linear map

$$T : \mathbb{R}^m \to \mathbb{R}^n, T(a) = \sum_{i=1}^m a_i v_i \tag{46}$$

   Therefore,

$$S = T^{-1}(\{u\}) \cap \mathbb{R}^m_{\geq 0} \cap \left\{ a \in \mathbb{R}^m \colon \sum_{i=1}^m a_i \leq M \right\} \tag{47}$$

which is a finite intersection of closed sets and hence closed. By Heine-Borel, it is compact. Therefore, by the extreme value theorem, we can find such a minimizer. Now we take a convergence sequence $(u_n) \in C$ such that $u_n \to u$. We are done if we can prove $u \in C$. We do this by first choosing a minimal representation for each u_n:

$$u_n = \sum_{i=1}^{m} a_{n,i} v_i, \quad A_n = \sum_{i=1}^{m} a_{n,i} \tag{48}$$

We claim that (A_n) is bounded. Suppose it isn't. Then, by passing to a subsequence, we can assume that $A_n \to \infty$. Because $u_n \to u$, (u_n) is bounded, so

$$\frac{u_n}{A_n} \to 0 \tag{49}$$

But

$$\frac{u_n}{A_n} = \sum_{i=1}^{m} \underbrace{\frac{a_{n,i}}{A_n}}_{q_{n,i}} v_i, \quad q_{n,i} \geq 0, \sum_{i=1}^{m} q_{n,i} = 1 \tag{50}$$

Therefore, the vectors $(q_{n,1}, \dots q_{n,m})$ lie in the standard simplex

$$\Delta^{m-1} = \left\{ q \in \mathbb{R}^m \colon q_i \geq 0, \sum_i q_i = 1 \right\} \tag{51}$$

Since this simplex is compact, there exists a subsequence along which $q_{n,i} \to q_i$ for every i; we relabel it by n. Passing to the limit gives

$$\sum_{i=1}^{m} q_i v_i = 0, \quad q_i \geq 0, \sum_{i=1}^{m} q_i = 1 \tag{52}$$

Since $\sum_{i=1}^{m} q_i = 1$, there exists some j such that $q_j > 0$. Furthermore, for every i such that $q_i > 0$, we have $q_{n,i} \to q_i > 0$, and hence $a_{n,i} = A_n q_{n,i} > 0$ for sufficiently large n.
Now define

$$t_n = \min_{i \colon q_i > 0} \frac{a_{n,i}}{q_i} > 0. \tag{53}$$

Consider the new coefficients

$$a'_{n,i} = a_{n,i} - t_n q_i. \tag{54}$$

By the definition of t_n, we have $a'_{n,i} \geq 0$ for all i. Furthermore,

$$\sum_{i=1}^{m} a'_{n,i} v_i = \sum_{i=1}^{m} (a_{n,i} - t_n q_i) v_i \tag{55}$$

$$= u_n - t_n \sum_{i=1}^{m} q_i v_i \tag{56}$$

$$= u_n. \tag{57}$$

Therefore, $(a'_{n,i})$ is another nonnegative representation of u_n. However,

$$\sum_{i=1}^{m} a'_{n,i} = \sum_{i=1}^{m} a_{n,i} - t_n \sum_{i=1}^{m} q_i \tag{58}$$

$$= A_n - t_n \tag{59}$$

$$< A_n, \tag{60}$$

contradicting the minimality of A_n. Therefore, (A_n) must be bounded.
Since $0 \leq a_{n,i} \leq A_n$ for all i, the sequence of coefficient vectors $(a_{n,1}, \dots, a_{n,m})$ is bounded in $\mathbb{R}^m$. By Bolzano-Weierstrass, there exists a convergent subsequence such that

$$a_{n,i} \to a_i \geq 0 \qquad \forall i. \tag{61}$$

Taking the limit in $u_n = \sum_{i=1}^m a_{n,i} v_i$ gives

$$u = \sum_{i=1}^m a_i v_i \in C. \tag{62}$$

Therefore, C is sequentially closed, and hence closed in $\mathbb{R}^n$.
Since we have shown that C is a closed convex cone, by the separating hyperplane theorem, there exists a nonzero linear functional $\ell \in V^\dagger$ and $\alpha \in \mathbb{R}$ such that

$$\ell(v) < \alpha \leq \ell(c) \qquad \forall c \in C. \tag{63}$$

Since $0 \in C$, we have

$$\ell(v) < \alpha \leq \ell(0) = 0, \tag{64}$$

and hence $\ell(v) < 0$. We claim that $\ell(c) \geq 0$ for all $c \in C$. Suppose otherwise, so that there exists $c \in C$ with $\ell(c) < 0$. Since C is a cone, $tc \in C$ for every $t \geq 0$. Thus,

$$t\ell(c) = \ell(tc) \geq \alpha \qquad \forall t \geq 0. \tag{65}$$

Taking $t \to \infty$ gives a contradiction, since $t\ell(c) \to -\infty$. Therefore, $\ell(c) \geq 0$ for all $c \in C$. In particular, since $v_i \in C$,

$$\ell(v_i) \geq 0 \quad \forall i, \qquad \ell(v) < 0. \tag{66}$$

In the rest of this section, vector inequalities are entry-wise. Elements of $\mathbb{R}^n$ are column vectors, and elements of the dual space $(\mathbb{R}^n)^\dagger$ are row vectors, so that a functional λ acts on r by the matrix product λr.

Corollary 1.4 (Farkas' Lemma, Equality Form)

Let $W \in \mathbb{R}^{n \times m}$ be the matrix whose columns are the generators[a] of a cone. For any $r \in \mathbb{R}^n$, the following are equivalent.

1. *Primal.* There exists an $a \in \mathbb{R}^m$ such that

$$r = Wa, \quad a \geq 0 \tag{67}$$

   That is, r lies in the *cone* generated by W.
2. *Dual.* For every row vector $\lambda \in \mathbb{R}^{1 \times n}$ satisfying $\lambda W \geq 0$, it is the case that

$$\lambda r \geq 0 \tag{68}$$

[a] Not necessarily linearly independent.

Proof. Obvious from theorem above.

Now here is a variant that relaxes the primal condition (the equality $r = Wa$ becomes the inequality $r \geq Wa$). On the dual side, we must now check $\lambda r \geq 0$ only for the λ that also satisfy $\lambda \geq 0$. This is a smaller set of λ's to check, so the dual condition is weaker, which matches the weaker primal condition.

Theorem 1.5 (Farkas' Lemma, Inequality Form)

Let $W \in \mathbb{R}^{n\times m}$ be the matrix whose columns are the generators[a] of a cone. For any $r \in \mathbb{R}^n$, the following are equivalent.

1. *Primal.* There exists an $a \in \mathbb{R}^m$ such that

$$r \geq Wa, \quad a \geq 0 \tag{69}$$

2. *Dual.* For every row vector $\lambda \in \mathbb{R}^{1\times n}$ satisfying $\lambda W \geq 0$ and $\lambda \geq 0$, it is the case that $\lambda r \geq 0$.

[a] Not necessarily linearly independent.

Proof. We prove bidirectionally.

1. ($\to$). Suppose there exists $a \geq 0$ such that $r \geq Wa$. For every $\lambda \geq 0$ satisfying $\lambda W \geq 0$, we have

$$\lambda r \geq \lambda W a \geq 0. \tag{70}$$

2. ($\leftarrow$). Suppose that for every $\lambda \geq 0$ satisfying $\lambda W \geq 0$, we have $\lambda r \geq 0$. Observe that the primal condition can be rewritten by introducing a slack variable $s \in \mathbb{R}^n_{\geq 0}$:

$$r \geq Wa, \quad a \geq 0 \iff r = Wa + s, \quad a, s \geq 0 \tag{71}$$

$$\iff r = \begin{pmatrix} W & I_n \end{pmatrix} \begin{pmatrix} a \\ s \end{pmatrix}, \quad \begin{pmatrix} a \\ s \end{pmatrix} \geq 0. \tag{72}$$

Define the augmented matrix

$$\widetilde{W} = \begin{pmatrix} W & I_n \end{pmatrix} \in \mathbb{R}^{n\times(m+n)}. \tag{73}$$

By the dual formulation of positive span, there exists a nonnegative coefficient vector b satisfying $r = \widetilde{W}b$ if and only if

$$\lambda\widetilde{W} \geq 0 \implies \lambda r \geq 0 \tag{74}$$

for every $\lambda \in \mathbb{R}^{1\times n}$. But

$$\lambda\widetilde{W} \geq 0 \iff \begin{pmatrix} \lambda W & \lambda I_n \end{pmatrix} \geq 0 \tag{75}$$

$$\iff \lambda W \geq 0, \quad \lambda \geq 0. \tag{76}$$

By assumption, every such λ satisfies $\lambda r \geq 0$. Therefore, there exists $b \geq 0$ such that

$$r = \widetilde{W}b. \tag{77}$$

Writing $b = (a^\top, s^\top)^\top$, we obtain

$$r = Wa + s, \qquad a, s \geq 0. \tag{78}$$

Hence $r \geq Wa$ with $a \geq 0$.

Definition 1.1 (Primal and Dual Feasibility)

Let $W \in \mathbb{R}^{n\times m}$, $r \in \mathbb{R}^n$, and $p \in \mathbb{R}^{1\times m}$. Consider the following two optimization problems.

1. *Primal.* The **primal problem** is

$$S = \sup_{a\in\mathbb{R}^m} \{pa \colon Wa \leq r,\ a \geq 0\}. \tag{79}$$

A vector $a \in \mathbb{R}^m$ satisfying $Wa \leq r, \qquad a \geq 0$ is called **primal feasible**.

2. *Dual.* The **dual problem** is

$$s = \inf_{\lambda \in \mathbb{R}^{1\times n}} \{\lambda r \colon \lambda W \geq p,\ \lambda \geq 0\}. \tag{80}$$

A row vector $\lambda \in \mathbb{R}^{1\times n}$ satisfying $\lambda W \geq p, \qquad \lambda \geq 0$ is called **dual feasible**.
For each problem, if the feasible set is nonempty, then the problem is called **feasible**.

In case the motivation is still not clear, we can do a manual derivation of the dual from the primal.

Example 1.1 (Manual Derivation of Dual from Primal)

Given a primal problem such as

$$\max 2x + y \text{ subject to } \begin{cases} x + y \leq 3 \\ x - y \leq 1 \\ x, y \geq 0 \end{cases} \tag{81}$$

we can just find *one* solution satisfying the constraints and use it as a lower bound on the optimum. Say we find one, e.g. $x = y = 0$, which does satisfy all constraints. Now if we want to find an *upper bound* on the value of the solution, we can use duality. We create new variables $a, b \geq 0$, and observe that

$$x + y \leq 3 \implies a(x + y) \leq 3a \tag{82}$$
$$x - y \leq 1 \implies b(x - y) \leq b \tag{83}$$

Thus by adding the two we have $(a + b)x + (a - b)y \leq 3a + b$.
Now if we suppose $a + b \geq 2$ and $a - b \geq 1$ (which are the coeffients of $2x + y$), then since $x, y \geq 0$, we have

$$2x + y \leq (a + b)x + (a - b)y \leq 3a + b \tag{84}$$

so any a, b satisfying the above inequalities give an upper bound of $3a + b$ on the value of the optimal solution. Say we have $a = 2, b = 1$, which does satisfy. Then $3a + b = 7$ is an upper bound of $2x + y$. Therefore, to get the tightest upper bound, we can minimize the dual problem

$$\min 3a + b \text{ subject to } \begin{cases} a + b \geq 2 \\ a - b \geq 1 \\ a, b \geq 0 \end{cases} \tag{85}$$

It should be clear from our line of results that point that we wish to make is that the primal and dual are two reformulations of the same problem, and by solving one, we can solve the other.

Lemma 1.6 (Dual of Dual is Primal)

Given the primal and dual problems above,

$$S = \sup_{a \in \mathbb{R}^m} \{pa \colon Wa \leq r,\ a \geq 0\}, \quad s = \inf_{\lambda \in \mathbb{R}^{1\times n}} \{\lambda r \colon \lambda W \geq p,\ \lambda \geq 0\}. \tag{86}$$

we can reformulate the dual problem as a primal problem, and the dual of the dual is the primal.

Proof. Let $\nu = \lambda^T$. Then, we can reformulate the dual as a primal problem

$$s = \inf_{\nu \in \mathbb{R}^n} \{r^T \nu \colon W^T \nu \geq p^T, \nu \geq 0\} \tag{87}$$

$$= \sup_{\nu \in \mathbb{R}^n} \{(-r)^T \nu : (-W)^T \nu \leq (-p)^T, \nu \geq 0\} \tag{88}$$

The dual of this is

$$\inf_{\phi \in \mathbb{R}^{1 \times n}} \{\phi(-p)^T \colon \phi(-W)^T \geq (-r)^T, \phi \geq 0\} \tag{89}$$

We can reformulate it as a primal again, and substituting $\mu = \phi^T$ gives us

$$\sup_{\phi \in \mathbb{R}^{1 \times n}} \{\phi p^T \colon W\phi^T \leq r, \phi \geq 0\} = \sup_{\mu \in \mathbb{R}^n} \{p\mu \colon W\mu \leq r, \mu \geq 0\} \tag{90}$$

which is exactly the first primal problem.

Lemma 1.7 (Weak Duality)

Let $W \in \mathbb{R}^{n \times m}$, $r \in \mathbb{R}^n$, and $p \in \mathbb{R}^{1 \times m}$. For any primal feasible vector a and dual feasible vector λ, we have

$$pa \leq \lambda r. \tag{91}$$

Consequently, if both problems are feasible,

$$S \leq s. \tag{92}$$

Proof. Since $\lambda W \geq p$ and $a \geq 0$,

$$pa \leq \lambda W a. \tag{93}$$

Furthermore, since $Wa \leq r$ and $\lambda \geq 0$,

$$\lambda W a \leq \lambda r. \tag{94}$$

Combining these inequalities gives

$$pa \leq \lambda W a \leq \lambda r. \tag{95}$$

Taking the supremum over all primal feasible a and the infimum over all dual feasible λ gives $S \leq s$.

Now, we can establish the equivalence of the primal and dual problems.

Theorem 1.8 (Strong Duality)

Let $W \in \mathbb{R}^{n \times m}$, $r \in \mathbb{R}^n$, and $p \in \mathbb{R}^{1 \times m}$, with the conventions $\sup \emptyset = -\infty$ and $\inf \emptyset = +\infty$. Then exactly one of the following holds.

1. Both optimal values are finite and attained, i.e. there exist a primal feasible vector a^* and a dual feasible vector λ^* such that
$$pa^* = S = s = \lambda^* r. \tag{96}$$
2. The primal is feasible and unbounded ($S = +\infty$), and the dual is infeasible.
3. The dual is feasible and unbounded ($s = -\infty$), and the primal is infeasible.
4. Both problems are infeasible.

In particular, if either problem has a finite optimal value, so does the other.

Proof. The four cases correspond to the four combinations of feasibility, so we treat each.

1. *Both feasible.* Fix a primal feasible a_0 and a dual feasible λ_0. We claim that there exist $a \geq 0$ and $\nu \geq 0$ (a column vector, $\nu = \lambda^\top$) such that

$$Wa \leq r, \qquad -W^\top \nu \leq -p^\top, \qquad -pa + r^\top \nu \leq 0. \tag{97}$$

That is, a is primal feasible, $\lambda = \nu^\top$ is dual feasible, and $pa \geq \lambda r$. Given this claim, weak duality gives $pa \leq S \leq s \leq \lambda r$, and combined with $pa \geq \lambda r$ all of these are equalities. Hence $S = s$ is finite and attained at $a^* = a$ and $\lambda^* = \lambda$.
To prove the claim, write the system as $Mz \leq b$ with $z = (a, \nu) \geq 0$, where

$$M = \begin{pmatrix} W & 0 \\ 0 & -W^\top \\ -p & r^\top \end{pmatrix}, \qquad b = \begin{pmatrix} r \\ -p^\top \\ 0 \end{pmatrix}. \tag{98}$$

By Farkas' lemma (inequality form), such a z exists if and only if every row vector $y = (y_1, y_2, t) \geq 0$, with $y_1 \in \mathbb{R}^{1\times n}$, $y_2 \in \mathbb{R}^{1\times m}$, $t \in \mathbb{R}$, satisfying $yM \geq 0$ also satisfies $yb \geq 0$. Computing,

$$yM \geq 0 \iff y_1 W \geq tp \text{ and } Wy_2^\top \leq tr, \qquad yb = y_1 r - py_2^\top. \tag{99}$$

Let $y \geq 0$ satisfy the left-hand conditions. There are two cases.

- $t > 0$. Dividing by t, the vector $\lambda = y_1/t$ is dual feasible and $a = y_2^\top/t$ is primal feasible. By weak duality, $pa \leq \lambda r$, and multiplying by t gives $y_1 r - py_2^\top \geq 0$.
- $t = 0$. Then $y_1 W \geq 0$ and $Wy_2^\top \leq 0$. Using $y_1 \geq 0$, $Wa_0 \leq r$, $a_0 \geq 0$,

$$y_1 r \geq y_1 W a_0 \geq 0. \tag{100}$$

Using $y_2 \geq 0$, $\lambda_0 W \geq p$, $\lambda_0 \geq 0$,

$$py_2^\top \leq \lambda_0 W y_2^\top \leq 0. \tag{101}$$

Together, $y_1 r - py_2^\top \geq 0$.
This proves the claim.

2. *Primal feasible, dual infeasible.* That the dual is infeasible means there is no $\nu \geq 0$ with $-W^\top \nu \leq -p^\top$. By Farkas' lemma (inequality form) applied to the matrix $-W^\top$ and the vector $-p^\top$, there exists a row vector $y \geq 0$ with $-yW^\top \geq 0$ and $-yp^\top < 0$. Setting $d = y^\top \in \mathbb{R}^m$, this says $d \geq 0$, $Wd \leq 0$, and $pd > 0$. For a primal feasible a_0 and any $\tau \geq 0$, the vector $a_0 + \tau d$ is primal feasible, since $W(a_0 + \tau d) \leq Wa_0 \leq r$. Its objective value is $pa_0 + \tau pd \to \infty$, so $S = +\infty$.
3. *Dual feasible, primal infeasible.* That the primal is infeasible means there is no $a \geq 0$ with $Wa \leq r$. By Farkas' lemma (inequality form) applied directly, there exists $\lambda \geq 0$ with $\lambda W \geq 0$ and $\lambda r < 0$. For a dual feasible λ_0 and any $\tau \geq 0$, the vector $\lambda_0 + \tau\lambda$ is dual feasible, and its objective value is $\lambda_0 r + \tau \lambda r \to -\infty$, so $s = -\infty$.
4. *Both infeasible.* Nothing to prove.

The four cases are mutually exclusive since they have distinct feasibility patterns.

Corollary 1.9 (Primal-Dual Optimality Criterion)

A primal feasible vector a^* and a dual feasible vector λ^* are both optimal if and only if

$$pa^* = \lambda^* r. \tag{102}$$

Proof. We prove bidirectionally.

1. ($\rightarrow$). By strong duality,
$$pa^* = S = s = \lambda^* r. \tag{103}$$
2. ($\leftarrow$). By weak duality, for any primal feasible a and dual feasible λ,
$$pa \leq \lambda^* r = pa^* \leq \lambda r. \tag{104}$$
Therefore, a^* maximizes the primal objective and λ^* minimizes the dual objective.

Weak duality is elementary and follows directly from the inequalities. Strong duality is the substantial theorem, and it is exactly where Farkas' lemma is needed. Note that the optimality criterion above is a *certificate*: to prove that a feasible a^* is optimal, it suffices to exhibit a dual feasible λ^* with the same objective value.

Example 1.2 (Production Planning and Resource Pricing)

Suppose a furniture factory produces tables and chairs. Each table requires 2 units of wood and 1 hour of labor, generating a profit of \$50. Each chair requires 1 unit of wood and 2 hours of labor, generating a profit of \$40. The factory has 100 units of wood and 80 hours of labor available.
Let $a_1, a_2 \geq 0$ denote the quantities of tables and chairs produced, respectively. We define
$$W = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}, \qquad r = \begin{pmatrix} 100 \\ 80 \end{pmatrix}, \qquad p = \begin{pmatrix} 50 & 40 \end{pmatrix}. \tag{105}$$

1. *Primal.* The factory wishes to maximize its profit subject to its resource constraints:
$$S = \sup_{a \geq 0} \{pa \colon Wa \leq r\}. \tag{106}$$
Equivalently,
$$\begin{aligned} \text{maximize} \quad & 50a_1 + 40a_2 && (107)\\ \text{subject to} \quad & 2a_1 + a_2 \leq 100, && (108)\\ & a_1 + 2a_2 \leq 80, && (109)\\ & a_1, a_2 \geq 0. && (110) \end{aligned}$$
2. *Dual.* Let $\lambda_1, \lambda_2 \geq 0$ represent the implicit prices of wood and labor.[a] The total resource value required to produce each product must be at least its profit. Therefore,
$$s = \inf_{\lambda \geq 0} \{\lambda r \colon \lambda W \geq p\}. \tag{111}$$
Equivalently,
$$\begin{aligned} \text{minimize} \quad & 100\lambda_1 + 80\lambda_2 && (112)\\ \text{subject to} \quad & 2\lambda_1 + \lambda_2 \geq 50, && (113)\\ & \lambda_1 + 2\lambda_2 \geq 40, && (114)\\ & \lambda_1, \lambda_2 \geq 0. && (115) \end{aligned}$$

Solving the primal problem gives
$$a^* = \begin{pmatrix} 40 \\ 20 \end{pmatrix}, \qquad S = pa^* = 2800. \tag{116}$$

Solving the dual problem gives

$$\lambda^* = \begin{pmatrix} 20 & 10 \end{pmatrix}, \qquad s = \lambda^* r = 2800. \tag{117}$$

Therefore,

$$S = s = 2800, \tag{118}$$

verifying strong duality.

[a] Suppose a buyer wants to purchase all your wood and labor instead of letting you make furniture. What is the lowest price the buyer could offer that you would still accept? Let λ_1 be the price per unit of wood and λ_2 the price per hour of labor. The buyer's total cost is $100\lambda_1 + 80\lambda_2$ and the buyer wants to minimize it. This is the dual objective. You would refuse any offer that pays less than you could earn by making furniture. A table uses 2 wood and 1 labor hour and earns \$50, so you need $2\lambda_1 + \lambda_2 \geq 50$. A chair needs $\lambda_1 + 2\lambda_2 \geq 40$. These are the dual constraints $\lambda W \geq p$. Prices can't be negative, which gives $\lambda \geq 0$.

Example 1.3 (Primal Feasible, Dual Infeasible)

An arbitrage is a trading strategy that requires no net investment and cannot lose money, but has a positive profit in at least one outcome. For example, suppose a stock ABC trades at \$100, and a European call option on ABC with strike \$90 expiring in one year trades at \$8. Since the right to buy at \$90 a stock worth \$100 is worth at least \$10, the option is underpriced. Consider the following trade today: buy one call for \$8 and short sell one share of ABC for \$100, for a net cash inflow of \$92. At expiration, with stock price S_T,

- if $S_T \geq 90$, exercise the call, buy the share at \$90 and return it to close the short, leaving a profit of $92 - 90 = \$2$;
- if $S_T < 90$, let the call expire, buy the share on the market at S_T and return it, leaving a profit of $92 - S_T > \$2$.

In every outcome we keep at least \$2 without having invested anything (ignoring interest). Doing the same trade on t contracts multiplies the profit by t.
Let

$$W = \begin{pmatrix} 1 & -1 \end{pmatrix}, \qquad r = 1, \qquad p = \begin{pmatrix} 1 & 1 \end{pmatrix}, \tag{119}$$

so that the primal is

$$\begin{aligned} \text{maximize} \quad & a_1 + a_2 && (120)\\ \text{subject to} \quad & a_1 - a_2 \leq 1, && (121)\\ & a_1, a_2 \geq 0. && (122) \end{aligned}$$

The primal is feasible, since $a = (0, 0)$ satisfies the constraints. It is also unbounded: for any $t \geq 0$, the vector $a = (t, t)$ satisfies $Wa = 0 \leq 1$ and has objective value $2t \to \infty$.
The dual, with a single variable $\lambda \geq 0$, is

$$\begin{aligned} \text{minimize} \quad & \lambda && (123)\\ \text{subject to} \quad & \lambda \geq 1, && (124)\\ & -\lambda \geq 1, && (125)\\ & \lambda \geq 0. && (126) \end{aligned}$$

The first two constraints say $\lambda \geq 1$ and $\lambda \leq -1$, which cannot both hold, so the dual is infeasible. This is case 2 of the strong duality theorem. The vector $d = (1, 1)^\top$ is the direction from the proof: $d \geq 0$, $Wd = 0 \leq 0$, and $pd = 2 > 0$. Moving along d keeps the primal feasible while increasing the objective without bound. Weak duality is consistent with this: it says every dual feasible λ gives an upper bound on the primal, and here there is no such λ, so nothing stops the primal from growing.

Example 1.4 (Both Infeasible)

A problem is **infeasible** when its constraints contradict each other, so that no choice of variables satisfies all of them at once. This typically happens when constraints come from different sources that nobody checked against each other. For example, suppose a hospital staffs a cardiac ward and a general ward, and let a_1, a_2 be the numbers of nurses assigned to each, with the goal of staffing as many nurses as possible ($\max a_1 + a_2$). The nurses' union contract says that the cardiac ward may have at most one more nurse than the general ward,

$$a_1 - a_2 \leq 1, \tag{127}$$

while a new state safety regulation requires the cardiac ward to have at least two more nurses than the general ward,

$$a_1 - a_2 \geq 2. \tag{128}$$

No staffing plan satisfies both since adding the contract inequality $a_1 - a_2 \leq 1$ to the regulation written as $-a_1 + a_2 \leq -2$ gives $0 \leq -1$, which is absurd. Every staffing plan breaks one of the two rules, so the problem is infeasible. Let

$$W = \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}, \qquad r = \begin{pmatrix} 1 \\ -2 \end{pmatrix}, \qquad p = \begin{pmatrix} 1 & 1 \end{pmatrix}. \tag{129}$$

1. *Primal.* The constraints $Wa \leq r$ read $a_1 - a_2 \leq 1$ and $-a_1 + a_2 \leq -2$, i.e. $a_1 - a_2 \leq 1$ and $a_1 - a_2 \geq 2$. These contradict each other, so the primal is infeasible. A certificate (from the inequality form of Farkas' lemma) is $\lambda = (1, 1)$, which satisfies $\lambda \geq 0$ and $\lambda W = (0, 0) \geq 0$, but $\lambda r = 1 - 2 = -1 < 0$.
2. *Dual.* The constraints $\lambda W \geq p$ with $\lambda \geq 0$ read

$$\lambda_1 - \lambda_2 \geq 1, \qquad -\lambda_1 + \lambda_2 \geq 1. \tag{130}$$

   Adding them gives $0 \geq 2$, so the dual is infeasible. A certificate is $d = (1, 1)^\top$, which satisfies $d \geq 0$, $Wd = (0, 0)^\top \leq 0$, and $pd = 2 > 0$.

This is case 4. With the conventions $\sup \emptyset = -\infty$ and $\inf \emptyset = +\infty$, we get $S = -\infty$ and $s = +\infty$. The two values differ, so strong duality gives no information in this case. Note that the vector d would make the primal unbounded *if* the primal had a feasible point, but it does not.

1.2 Complementary Slackness and Sensitivity

Strong duality has two useful consequences. The first tells us exactly how an optimal primal and dual pair must interact.

Theorem 1.10 (Complementary Slackness)

Let a be primal feasible and λ be dual feasible. Then a and λ are both optimal if and only if

$$\lambda(r - Wa) = 0 \qquad \text{and} \qquad (\lambda W - p)a = 0. \tag{131}$$

Proof. We have the identity

$$\lambda r - pa = \underbrace{\lambda(r - Wa)}_{\geq 0} + \underbrace{(\lambda W - p)a}_{\geq 0}. \tag{132}$$

Both terms are nonnegative by feasibility. By the optimality criterion, a and λ are both optimal if and only if $\lambda r - pa = 0$, which holds if and only if both nonnegative terms vanish.

Since every term in these two inner products is nonnegative, this is equivalent to the entrywise statements

$$\lambda_i > 0 \implies (Wa)_i = r_i, \qquad \text{(a priced resource is fully used)} \tag{133}$$
$$a_j > 0 \implies (\lambda W)_j = p_j. \qquad \text{(a produced good has zero profit after pricing)} \tag{134}$$

Example 1.5 (Complementary Slackness for Furniture, No Slack)

For the furniture example above, $a^* = (40, 20)^\top$ uses all of the wood and labor, $Wa^* = (100, 80)^\top = r$, and $\lambda^* W = (50, 40) = p$. So complementary slackness holds with $\lambda^* > 0$ and $a^* > 0$, with no slack in either place. We can also run this in reverse, to *compute* the dual solution from the primal one. Suppose we have only solved the primal and found $a^* = (40, 20)^\top$, and we do not yet know λ^*. Both a_1^* and a_2^* are positive, so complementary slackness forces both dual constraints to be tight, $(\lambda^* W)_j = p_j$ for $j = 1, 2$:

$$2\lambda_1 + \lambda_2 = 50, \tag{135}$$
$$\lambda_1 + 2\lambda_2 = 40. \tag{136}$$

This is a 2×2 linear system, and its solution is $\lambda^* = (20, 10)$. It remains to check that this λ^* is actually dual feasible and optimal: indeed $\lambda^* \geq 0$, $\lambda^* W = (50, 40) \geq p$, and $\lambda^* r = 100 \cdot 20 + 80 \cdot 10 = 2800 = pa^*$, so by the optimality criterion λ^* is optimal. Had a constraint been slack at a^* (say $(Wa^*)_i < r_i$), complementary slackness would instead give $\lambda_i = 0$, which removes an unknown.

In general, if we guess the set B of positive primal variables (the optimal basis in the language of the simplex method below), the equations $(\lambda W)_j = p_j$ for $j \in B$ and $\lambda_i = 0$ for the slack constraints form a linear system for λ^*, which is then verified for feasibility as above.

The second consequence is that the dual variables are marginal prices.

Theorem 1.11 (Dual Variables are Maringal/Shadow Prices)

Suppose the dual is feasible, and let $S(r) = \sup_{a \in \mathbb{R}^m} \{pa \colon Wa \leq r, a \geq 0\}$ denote the optimal value of the primal as a function of the resource vector r, with W and p fixed. Then,

1. On the set of r for which the primal is feasible, $S(r)$ is finite, concave, and piecewise linear in r.
2. If λ^* is an optimal dual solution for r, then

$$S(r') \leq S(r) + \lambda^*(r' - r) \qquad \forall r'. \tag{137}$$

That is, λ^* is a supergradient of S at r. In particular, wherever S is differentiable, $\partial S / \partial r_i = \lambda_i^*$.

Proof. The dual feasible set $D = \{\lambda \geq 0 \colon \lambda W \geq p\}$ does not depend on r. By strong duality, for every r with the primal feasible,

$$S(r) = \inf_{\lambda \in D} \lambda r, \tag{138}$$

which is a pointwise infimum of linear functions of r, and hence concave. It is finite since D and the primal are nonempty, and it is piecewise linear because D is a polyhedron whose infimum is attained at one of its finitely many vertices (it contains no lines since $\lambda \geq 0$). For the supergradient inequality, $\lambda^* \in D$ is dual feasible for every r', so $S(r') \leq \lambda^* r'$ by weak duality (or $S(r') = -\infty$), while $S(r) = \lambda^* r$. Subtracting gives the claim.

In the furniture example, $\lambda^* = (20, 10)$, so one extra unit of wood is worth \$20 and one extra hour of labor is worth \$10. Indeed with 101 units of wood, solving the constraints gives $a = (122/3, 59/3)$ and profit $2820 = 2800 + 20$.

1.3 Geometry of Linear Programs

We now turn to algorithms. For the geometry it is convenient to use the **standard form**, which is related to the canonical form by adding slack variables. In this subsection (and the next) the notation resets: $A \in \mathbb{R}^{m\times n}$ with $\operatorname{rank} A = m$, $b \in \mathbb{R}^m$, $c \in \mathbb{R}^{1\times n}$, and the LP is

$$\min\{cx \colon Ax = b,\ x \geq 0\}. \tag{139}$$

To convert the canonical primal $\max\{pa \colon Wa \leq r, a \geq 0\}$, we introduce slacks $s \geq 0$ with $Wa + s = r$, and set

$$x = \begin{pmatrix} a \\ s \end{pmatrix}, \quad A = \begin{pmatrix} W & I_n \end{pmatrix}, \quad b = r, \quad c = \begin{pmatrix} -p & 0 \end{pmatrix}, \tag{140}$$

so that maximizing pa becomes minimizing cx. Writing the dual of the standard form is easy. For any feasible x and any row vector λ (of either sign) with $\lambda A \leq c$, we have $\lambda b = \lambda A x \leq cx$ since $x \geq 0$. So the dual of the standard form is $\max\{\lambda b \colon \lambda A \leq c\}$, with λ free, matching the remark above that equalities give free dual variables.

The feasible set $P = \{x \colon Ax = b, x \geq 0\}$ is a *polyhedron*, an intersection of finitely many half-spaces.

Definition 1.2 (Extreme Point, Basic Feasible Solution)

A point $x \in P$ is an **extreme point** if it cannot be written as $x = \theta y + (1-\theta)z$ with $y \neq z$ in P and $\theta \in (0,1)$. For a set $B \subset \{1,\dots,n\}$ of m indices with the columns A_j, $j \in B$, linearly independent, let A_B be the invertible matrix of these columns. The point x with $x_B = A_B^{-1} b$ and $x_j = 0$ for $j \notin B$ is the **basic solution** for the basis B. If $x \geq 0$ it is a **basic feasible solution** (BFS). The indices in B are **basic** and the rest are **nonbasic**.

A BFS is determined by its basis, and there are at most $\binom{n}{m}$ bases. So there are only finitely many BFSs.

Lemma 1.12 (Extreme Points are Basic Feasible Solutions)

A point $x \in P$ is an extreme point if and only if the columns $\{A_j \colon x_j > 0\}$ are linearly independent. Equivalently (since $\operatorname{rank} A = m$, a linearly independent set of columns can be extended to a basis), extreme points of P are precisely the BFSs.

Proof. Let $\operatorname{supp}(x) = \{j \colon x_j > 0\}$.
($\leftarrow$) Suppose $x = \theta y + (1-\theta)z$ with $y, z \in P$, $y \neq z$, and $\theta \in (0,1)$. Since $y, z \geq 0$ and $x_j = 0$ forces $\theta y_j + (1-\theta) z_j = 0$, we get $y_j = z_j = 0$ for $j \notin \operatorname{supp}(x)$. So $d = y - z \neq 0$ is supported in $\operatorname{supp}(x)$ and $Ad = 0$, which is a nontrivial linear dependence among the columns A_j, $j \in \operatorname{supp}(x)$. So if those columns are independent, x is extreme.
($\rightarrow$) Suppose there is $d \neq 0$ supported in $\operatorname{supp}(x)$ with $Ad = 0$. Since $x_j > 0$ on $\operatorname{supp}(x)$, for small $\varepsilon > 0$ both $x \pm \varepsilon d \geq 0$, and $A(x \pm \varepsilon d) = b$. Then x is the midpoint of $x + \varepsilon d$ and $x - \varepsilon d$, so it is not extreme.

Theorem 1.13 (Fundamental Theorem of Linear Programming)

Suppose $P \neq \emptyset$ and cx is bounded below on P. Then the minimum of cx over P is attained at a basic feasible solution. In particular, P has a BFS.

Proof. Let $x_0 \in P$. Among all $x \in P$ with $cx \leq cx_0$, choose one with the fewest nonzero entries. We claim that the columns A_j, $j \in \operatorname{supp}(x)$, are linearly independent, so that x is a BFS by the lemma. Suppose not. Then there is $d \neq 0$ supported in $\operatorname{supp}(x)$ with $Ad = 0$. Replacing d by $-d$ if necessary, we may assume $cd \leq 0$. Note that $x + td$ satisfies $A(x + td) = b$ for all t.

- If $d \geq 0$ and $cd < 0$, then $x+td \in P$ for all $t \geq 0$ and $c(x+td) \to -\infty$, contradicting boundedness.
- If $d \geq 0$ and $cd = 0$, replace d by $-d$. Since $d \neq 0$ it now has a negative entry, and $c(-d) = 0$. So we are in the next case.
- If d has a negative entry, let $t^* = \min\{x_j/(-d_j) \colon d_j < 0\} > 0$. Then $x' = x + t^*d \in P$, $cx' = cx + t^*cd \leq cx \leq cx_0$, and x' has strictly fewer nonzero entries than x (its support is contained in $\operatorname{supp}(x)$ and one more entry vanishes), contradicting the choice of x.

So x is a BFS with $cx \leq cx_0$. Since x_0 was arbitrary, $\inf_P cx = \min_{\mathrm{BFS}} cx$, and this minimum exists since there are finitely many BFSs.

This reduces an optimization over an infinite set to a search over finitely many vertices, but the number of vertices can still be exponentially large. The simplex method moves from vertex to adjacent vertex, never increasing the cost.

1.4 The Simplex Method

Fix a basis B with BFS x ($x_B = A_B^{-1}b \geq 0$, $x_N = 0$), where N denotes the nonbasic indices. Every feasible y satisfies $y_B = A_B^{-1}(b - A_N y_N)$, so its cost is

$$cy = c_B y_B + c_N y_N = c_B A_B^{-1} b + \underbrace{(c_N - c_B A_B^{-1} A_N)}_{\bar{c}_N} y_N = cx + \bar{c}_N y_N. \tag{141}$$

Define the **reduced cost** of index j as $\bar{c}_j = c_j - c_B A_B^{-1} A_j$, which is 0 for $j \in B$. Intuitively, $\bar{c}_j$ is the rate at which the cost changes if we increase the nonbasic variable x_j from zero and let the basic variables adjust to maintain $Ax = b$.

Theorem 1.14 (Optimality Test)

If $\bar{c}_j \geq 0$ for all j, then the BFS x is optimal. Moreover, $\lambda = c_B A_B^{-1}$ is then an optimal solution of the dual $\max\{\lambda b \colon \lambda A \leq c\}$.

Proof. From the display above, $cy = cx + \bar{c}_N y_N \geq cx$ for every feasible y, since $\bar{c}_N \geq 0$ and $y_N \geq 0$. For the dual, $\bar{c}_j = c_j - \lambda A_j \geq 0$ says exactly that $\lambda A \leq c$, so λ is dual feasible, and $\lambda b = c_B A_B^{-1} b = cx$. By weak duality (the optimality criterion), both are optimal.

So the simplex method terminates with a primal solution together with its own dual certificate. If instead some $\bar{c}_j < 0$, increasing x_j decreases the cost. Let $d = A_B^{-1} A_j$. Setting $x_j = t$ and keeping the other nonbasic variables zero gives $x_B(t) = x_B - td$. Either $d \leq 0$, so $x(t)$ is feasible for all $t \geq 0$ with cost $cx + t\bar{c}_j \to -\infty$ (the LP is unbounded), or we can increase t until the first basic variable hits 0. This gives the pivot.

Algorithm 1.1 (Simplex Method)

Given a basic feasible solution x with basis B.

1. *Pricing.* Compute $\lambda = c_B A_B^{-1}$ and $\bar{c}_j = c_j - \lambda A_j$ for $j \in N$. If $\bar{c} \geq 0$, stop: x is optimal. Otherwise choose an **entering** index j with $\bar{c}_j < 0$.
2. *Direction.* Compute $d = A_B^{-1} A_j$. If $d \leq 0$, stop: the LP is unbounded.
3. *Ratio test.* Compute $t^* = \min\{x_{B_i}/d_i \colon d_i > 0\}$ and let B_i be an index attaining the minimum; it is the **leaving** index.
4. *Update.* Replace x_B by $x_B - t^*d$, set $x_j = t^*$, and replace B_i by j in the basis. The new basis is again a basis, and x is again a BFS, with cost changed by $t^*\bar{c}_j \leq 0$. Return to step 1.

We make four remarks on the method.

1. *Termination.* If $t^* > 0$ at every step, the cost strictly decreases, so no basis is repeated and since

there are finitely many bases the method terminates. When some basic variable is already zero (a **degenerate** BFS), we can have $t^* = 0$ and the method may *cycle* through the same bases. Bland's rule (among the eligible choices, always pick the entering and leaving indices of smallest index) provably prevents cycling.

2. *Initialization.* A starting BFS comes from **Phase I**. Multiply rows by -1 so that $b \geq 0$ and solve $\min\{\mathbf{1}^\top s \colon Ax + s = b, x, s \geq 0\}$ starting from the obvious BFS $(x, s) = (0, b)$. The original LP is feasible if and only if the optimum is 0, and an optimal BFS with $s = 0$ gives a BFS of P (after pivoting out any zero-valued artificial variables that remain basic).
3. *Complexity.* Each pivot costs a few linear solves, and in practice the method needs a number of pivots that is roughly linear in m. However, the worst case is exponential: for Dantzig's rule (choose the most negative $\bar{c}_j$), the Klee–Minty cube is a deformed n-cube on which the method visits all 2^n vertices. Smoothed analysis (Spielman–Teng) explains why this does not happen on perturbed instances.
4. *Polynomial-time methods.* Khachiyan's ellipsoid method (1979) was the first polynomial-time algorithm for LP, although it is slow in practice. Karmarkar's interior-point method (1984) is both polynomial and practical, and is discussed below.

Example 1.6 (Simplex on the Furniture Problem)

Return to the production problem. In standard form with slacks s_1, s_2 and $x = (a_1, a_2, s_1, s_2)$,

$$\min\ -50a_1 - 40a_2 \quad \text{s.t.} \quad \begin{pmatrix} 2 & 1 & 1 & 0 \\ 1 & 2 & 0 & 1 \end{pmatrix} x = \begin{pmatrix} 100 \\ 80 \end{pmatrix}, \quad x \geq 0. \tag{142}$$

Start from the basis $\{s_1, s_2\}$, i.e. $x = (0, 0, 100, 80)$ with cost 0.

1. Here $\lambda = 0$, so $\bar{c} = (-50, -40, 0, 0)$. We choose a_1 to enter. Then $d = (2, 1)$ and the ratios are $100/2 = 50$ and $80/1 = 80$, so s_1 leaves with $t^* = 50$. The new BFS is $x = (50, 0, 0, 30)$ with cost -2500.
2. With basis $\{a_1, s_2\}$, $A_B = \begin{pmatrix} 2 & 0 \\ 1 & 1 \end{pmatrix}$ and $\lambda = c_B A_B^{-1} = (-25, 0)$. The reduced costs are $\bar{c}_{a_2} = -40 - (-25)(1) = -15$ and $\bar{c}_{s_1} = 0 - (-25) = 25$. We choose a_2 to enter. Then $d = A_B^{-1}(1, 2)^\top = (1/2, 3/2)$ and the ratios are $50/(1/2) = 100$ and $30/(3/2) = 20$, so s_2 leaves with $t^* = 20$. The new BFS is $x = (40, 20, 0, 0)$ with cost -2800.
3. With basis $\{a_1, a_2\}$, $A_B = W$ and $\lambda = c_B W^{-1} = (-20, -10)$. The reduced costs are $\bar{c}_{s_1} = 20$ and $\bar{c}_{s_2} = 10$, both nonnegative, so $x = (40, 20, 0, 0)$ is optimal.

The dual certificate is $\lambda = (-20, -10)$. Since we minimized $-pa$ rather than maximized pa, the sign flips, recovering the shadow prices $(20, 10)$ from before.

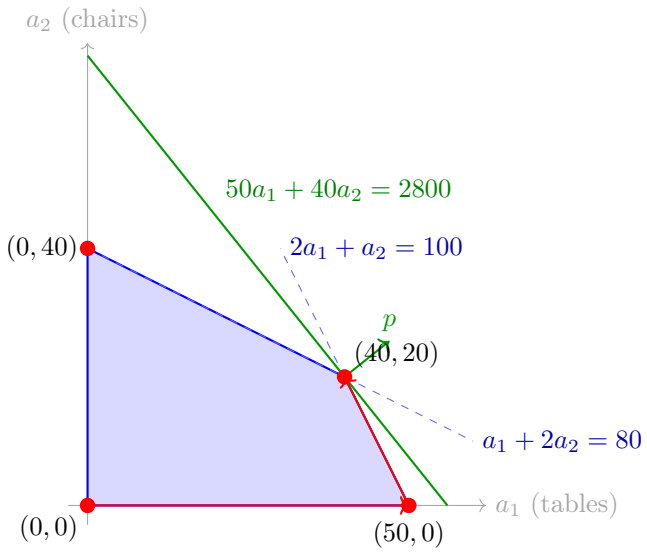


Figure 2: The feasible region of the furniture problem. Simplex starts at the origin and moves along edges (red) to the vertex $(40, 20)$, where the objective level line (green) touches the region last.

1.5 Interior-Point Methods and the Lagrangian View

Simplex walks along the boundary of P. Interior-point methods instead travel through its interior. Replace the standard-form LP by the barrier problem

$$\min_x \; cx - \mu \sum_{j=1}^{n} \log x_j \quad \text{s.t.} \quad Ax = b, \tag{143}$$

for $\mu > 0$. The logarithm forces $x > 0$, so the solution $x(\mu)$ is interior. The optimality conditions of this strictly convex problem are

$$Ax = b, \qquad A^\top \lambda^\top + s = c^\top, \qquad x_j s_j = \mu \;\; \forall j, \qquad x, s > 0, \tag{144}$$

with the sign convention that $s = c^\top - A^\top \lambda^\top \geq 0$ is the dual slack. The curve $\mu \mapsto (x(\mu), \lambda(\mu), s(\mu))$ is the **central path**. For $\mu = 0$ these conditions are exactly primal feasibility, dual feasibility and *complementary slackness* $x_j s_j = 0$, so as $\mu \to 0$ the central path converges to an optimal primal-dual pair. Interior-point methods apply Newton's method to this system while decreasing μ, and they take polynomially many Newton steps. The duality gap along the path is

$$cx - \lambda b = x^\top s = n\mu, \tag{145}$$

which gives a certificate of how far we are from optimal at every iteration.

Finally, LP duality is a special case of Lagrangian duality, which is developed for general constrained problems in later sections. For the canonical primal, define the Lagrangian $L(a, \lambda) = pa + \lambda(r - Wa)$ for $a \geq 0$ and $\lambda \geq 0$. For fixed $a \geq 0$,

$$\inf_{\lambda \geq 0} L(a, \lambda) = \begin{cases} pa & Wa \leq r \\ -\infty & \text{otherwise,} \end{cases} \tag{146}$$

so $S = \sup_{a \geq 0} \inf_{\lambda \geq 0} L(a, \lambda)$. On the other hand, rearranging $L(a, \lambda) = \lambda r + (p - \lambda W)a$, for fixed $\lambda \geq 0$

$$\sup_{a \geq 0} L(a, \lambda) = \begin{cases} \lambda r & \lambda W \geq p \\ +\infty & \text{otherwise,} \end{cases} \tag{147}$$

so $s = \inf_{\lambda \geq 0} \sup_{a \geq 0} L(a, \lambda)$. Weak duality is the general inequality $\sup \inf \leq \inf \sup$, and strong duality says that this minimax inequality is an equality for LPs, i.e. that (a^*, λ^*) is a saddle point of L. Complementary slackness is then the statement that $L(a^*, \lambda^*) = pa^* = \lambda^* r$.

2 Smooth Optimization

Unfortunately, f being merely differentiable does not give too many regularity conditions. A moderately stronger regularity condition is that the gradient itself is Lipschitz continuous.

Definition 2.1 (L-Smooth)

A function $f : \mathbb{R}^d \to \mathbb{R}$ is **L-smooth** if the gradient is L-Lipschtiz (and hence C^1).

$$\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\| \tag{148}$$

Lemma 2.1 (Properties of L-Smoothness)

Let $f, g : \mathbb{R}^d \to \mathbb{R}$ with f L-smooth and g M-smooth.

1. *Monotonicity.* f is L'-smooth for every $L' \geq L$.
2. *Scaling.* For $a \in \mathbb{R}$, af is $|a|L$-smooth.
3. *Addition.* $f + g$ is $(L + M)$-smooth.
4. *Affine Precomposition.* For $A \in \mathbb{R}^{m\times d}$ and $b \in \mathbb{R}^m$, if $f : \mathbb{R}^m \to \mathbb{R}$ is L-smooth, then $h(x) = f(Ax + b)$ is $L\|A\|_{op}^2$-smooth.
5. *Hessian Characterization.* If f is C^2, then f is L-smooth if and only if $\|\nabla^2 f(x)\|_{op} \leq L$ for all x.
6. *Product.* If f, g are bounded and have bounded gradients, say $|f| \leq B_f$, $|g| \leq B_g$, $\|\nabla f\| \leq G_f$, $\|\nabla g\| \leq G_g$, then fg is $(B_f M + B_g L + 2G_f G_g)$-smooth.[a]

[a] The boundedness assumptions in the product rule cannot be dropped. For $f = g = \frac{1}{2}x^2$ on $\mathbb{R}$, both are 1-smooth, but $fg = \frac{1}{4}x^4$ has $(fg)'' = 3x^2$, which is unbounded, so it is not L-smooth for any L by (5).

Proof. Listed.

1. *Monotonicity.* Immediate from definition.
2. *Scaling.* Immediate from definition.
3. *Addition.* by the triangle inequality,

$$\|\nabla(f + g)(x) - \nabla(f + g)(y)\| \leq \|\nabla f(x) - \nabla f(y)\| + \|\nabla g(x) - \nabla g(y)\| \leq (L + M)\|x - y\| \tag{149}$$

4. *Affine Precomposition.* By the chain rule, $\nabla h(x) = A^T \nabla f(Ax + b)$, so

$$\|\nabla h(x) - \nabla h(y)\| \leq \|A^T\|_{op}\|\nabla f(Ax + b) - \nabla f(Ay + b)\| \leq \|A\|_{op} L\|A(x - y)\| \leq L\|A\|_{op}^2\|x - y\| \tag{150}$$

5. *Hessian.* ($\Leftarrow$) By the fundamental theorem of calculus, $\nabla f(x) - \nabla f(y) = \int_0^1 \nabla^2 f(y + t(x - y))(x - y)\, dt$, so $\|\nabla f(x) - \nabla f(y)\| \leq \sup_z \|\nabla^2 f(z)\|_{op}\|x - y\|$. ($\Rightarrow$) For any unit vector u and $t > 0$, $\|\nabla^2 f(x)u\| = \lim_{t\to 0} \frac{1}{t}\|\nabla f(x + tu) - \nabla f(x)\| \leq L$.
6. *Product.* Since $\nabla(fg) = f\nabla g + g\nabla f$, we can write

$$\nabla(fg)(x) - \nabla(fg)(y) = f(x)\big(\nabla g(x) - \nabla g(y)\big) + \big(f(x) - f(y)\big)\nabla g(y) \tag{151}$$

$$+ g(x)\big(\nabla f(x) - \nabla f(y)\big) + \big(g(x) - g(y)\big)\nabla f(y) \tag{152}$$

Since $\|\nabla f\| \leq G_f$ and $\|\nabla g\| \leq G_g$, f and g are G_f- and G_g-Lipschitz respectively (mean value theorem), so taking norms gives

$$\|\nabla(fg)(x) - \nabla(fg)(y)\| \leq \big(B_f M + G_f G_g + B_g L + G_g G_f\big)\|x - y\| \tag{153}$$

When we have L-smoothness, we can get some nice bounds.

Theorem 2.2 (Taylor's 1st Order Bound)

If $f : \mathbb{R}^d \to \mathbb{R}$ is L-smooth, then

$$|f(x) - f(w) - \langle \nabla f(w), x - w \rangle| \leq \frac{L}{2}\|x - w\|^2 \tag{154}$$

and therefore

$$f(x) \leq f(w) + \langle \nabla f(w), x - w \rangle + \frac{L}{2}\|x - w\|^2 \tag{155}$$

Proof. Let $x = w + s$ and define $\theta : [0, 1] \to \mathbb{R}, \theta(t) = f(w + ts)$. Since $f \in C^1$, f' is continuous and hence Riemann integrable. By the 2nd fundamental theorem of calculus, we can write

$$\begin{aligned}
|f(w + s) - f(w) - \nabla f(w)^T s| &= \left| \theta(0) + \int_0^1 \theta'(t)\, dt - \theta(0) - \nabla f(w)^T s \right| && (156)\\
&= \left| \int_0^1 \langle \nabla f(w + ts) - \nabla f(w), s \rangle\, dt \right| && (157)\\
&\leq \int_0^1 |\langle \nabla f(w + ts) - \nabla f(w), s \rangle|\, dt && (158)\\
&\leq \int_0^1 \|\nabla f(w + ts) - \nabla f(w)\| \|s\|\, dt && (159)\\
&\leq \int_0^1 L\|ts\| \|s\|\, dt && (160)\\
&= \frac{1}{2} L \|s\|^2 && (161)
\end{aligned}$$

This is useful since it upper-bounds a potentially complicated nonlinear function by a simple quadratic function. An algorithmic design principle that we will follow is that instead of minimizing f directly, we iteratively minimize an approximation of f centered at the previous iterate.

Lemma 2.3 (Gradient Descent on L-Smooth Functions)

Gradient descent on the function class $\mathcal{F}$ of L-smooth functions is equivalent to the following algorithm consisting of the oracle

$$\mathcal{O}_f : S \to \mathbb{R} \times \mathbb{R}^d, \quad \mathcal{O}_f(x) = \big(f(x), \nabla f(x)\big) \tag{162}$$

and update maps $\Phi_k : (S \times (\mathbb{R} \times \mathbb{R}^d))^{k+1} \to S$

$$\begin{aligned}
w_{k+1} &= \Phi_k\big(w_0, (f(w_0), \nabla f(w_0)), \ldots, w_k, (f(w_k), \nabla f(w_k))\big) && (163)\\
&\coloneqq w_k - \eta_k \nabla f(w_k) && (164)\\
&= \operatorname*{argmin}_w \left\{ \underbrace{\langle \nabla f(w_k), w - w_k \rangle}_{\text{linear approximation}} + \underbrace{\frac{1}{2\eta_k}\|w - w_k\|^2}_{\text{anchor}} \right\} && (165)
\end{aligned}$$

Note that the quadratic term is what makes this function convex, and therefore has a well-defined minimum. It is also a regularizer that prevents us from moving too far from the current point w_k. After k iterations, the algorithm outputs $w_{\mathcal{A}_k}(f) = w_k$.

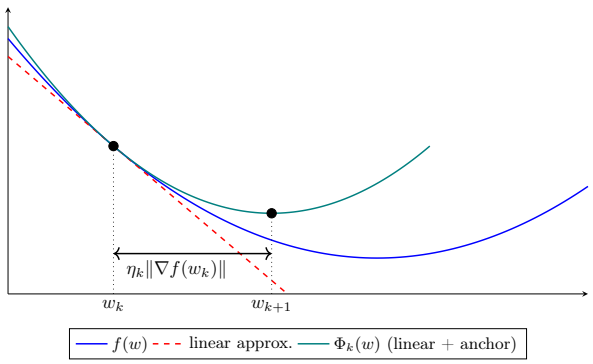


Figure 3: The linear approximation at w_k (dashed) is unbounded below, so the anchor term (teal) is added to keep it bounded. Its minimizer is exactly the gradient descent step $w_{k+1} = w_k - \eta_k \nabla f(w_k)$.

Proof. Recall that at a local minimum of a differentiable function, the gradient must vanish. So,

$$0 = \nabla \Phi_k(w_{k+1}) = \nabla f(w_k) + \frac{1}{\eta_k}(w_{k+1} - w_k) \implies w_{k+1} = w_k - \eta_k \nabla f(w_k) \tag{166}$$

So η is sort of like the step size.

Often times, when we just care about one step, we will just write something like

$$w - \eta \nabla f(w) \tag{167}$$

Okay, so we have an equivalent formulation of GD. This formulation will allow us to establish our desired bounds.

Lemma 2.4 (Descent Lemma)

Let $f : \mathbb{R}^d \to \mathbb{R}$ be L-smooth. Then,

$$f(w - \eta \nabla f(w)) \leq f(w) - \eta\big(1 - \frac{L\eta}{2}\big)\|\nabla f(w)\|^2 \tag{168}$$

Furthermore, if $\eta \in (0, \frac{2}{L})$, then the iterates are guaranteed to decrease, i.e.

$$f(w - \eta \nabla f(w)) \leq f(w) \tag{169}$$

Proof. This follows from Taylor's with $x = w - \eta \nabla f(w)$, since

$$f(w - \eta \nabla f(w)) \leq f(w) + \langle \nabla f(w), -\eta \nabla f(w) \rangle + \frac{L}{2}\|\eta \nabla f(w)\|^2 \tag{170}$$

$$= f(w) - \eta \|\nabla f(w)\|^2 + \frac{L\eta^2}{2}\|\nabla f(w)\|^2 \tag{171}$$

$$= f(w) - \eta\big(1 - \frac{L\eta}{2}\big)\|\nabla f(w)\|^2 \tag{172}$$

Now if $0 < \eta < 2/L$, then $\frac{L\eta}{2} < 1$, so $1 - \frac{L\eta}{2} > 0$, and with the fact that $0 < \eta$,

$$\eta\big(1 - \frac{L\eta}{2}\big) > 0 \implies f(w) - \eta\big(1 - \frac{L\eta}{2}\big)\|\nabla f(w)\|^2 \leq f(w) \tag{173}$$

with strict inequality whenever $\nabla f(w) \neq 0$.

Note that when $\eta = \frac{1}{L}$, then $\eta\left(1 - \frac{L\eta}{2}\right) = \frac{1}{2L}$. For this reason, we will often set the stepsize to be $\eta = \frac{1}{L}$.

Example 2.1 (Descent Lemma on a Nonconvex L-Smooth Function)

Consider the nonconvex function

$$f : \mathbb{R}^2 \to \mathbb{R}, \qquad f(x, y) = \frac{1}{2}x^2 + 5y^2 + 2\sin x \sin y. \tag{174}$$

It is C^2, and its gradient and hessian are

$$\nabla f(x, y) = \begin{pmatrix} x + 2\cos x \sin y \\ 10y + 2\sin x \cos y \end{pmatrix}, \quad \nabla^2 f(x, y) = \begin{pmatrix} 1 & 0 \\ 0 & 10 \end{pmatrix} + 2\begin{pmatrix} -\sin x \sin y & \cos x \cos y \\ \cos x \cos y & -\sin x \sin y \end{pmatrix} \tag{175}$$

we can compute that $\sup_{x,y} \|\nabla^2 f(x, y)\|_{\text{op}} = 12$ (attained at $(\frac{\pi}{2}, -\frac{\pi}{2})$), so f is L-smooth with $L = 12$. We run GD with two values of η from $w_0 = (4, 2.5)$ for 30 steps. By the descent lemma, every iterate must satisfy $f(w_{k+1}) \le f(w_k) - \eta\left(1 - \frac{L\eta}{2}\right)\|\nabla f(w_k)\|^2$ (which equals $f(w_k) - \frac{1}{2L}\|\nabla f(w_k)\|^2$ when $\eta = 1/L$), which we verify numerically.

1. $\eta = 1/L = 1/12$. We are guaranteed that the objective value will decrease, which we can see here.

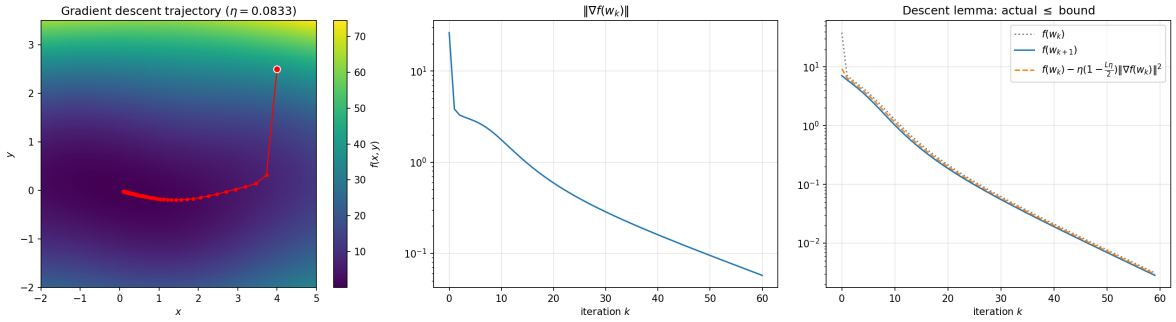


Figure 4: Left: trajectory (red) over the heatmap of f. Middle: gradient norm $\|\nabla f(w_k)\|$. Right: $f(w_{k+1})$ stays below the descent lemma bound $f(w_k) - \eta(1 - \frac{L\eta}{2})\|\nabla f(w_k)\|^2$ at every iteration. Code in `code/descent_lemma.py`.

2. $\eta = 1/5$. This exceeds $2/L = 1/6$, so even though the descent lemma is still valid, now we are not guaranteed that gradient descent will decrease since the bound is now larger than $f(w_k)$.[a]

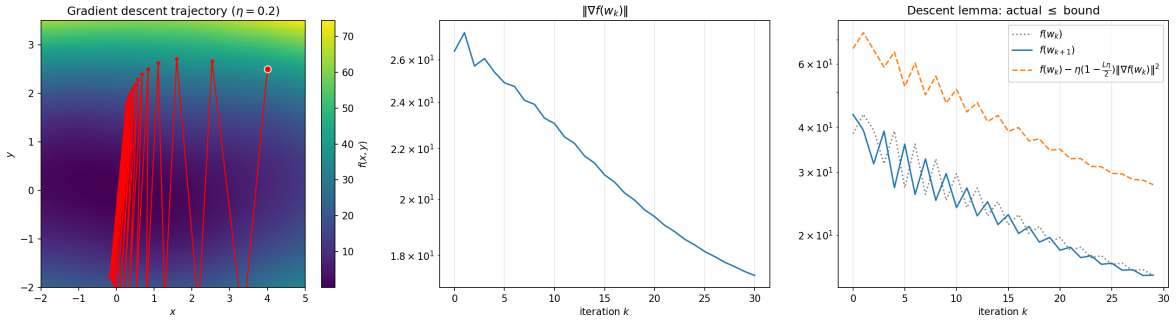


Figure 5: Same setup with $\eta = 0.2 > 2/L$ (log scale on the right). The iterates oscillate across the valley, and $f(w_{k+1}) > f(w_k)$ on every other iteration, although $f(w_{k+1})$ always lies below the (now larger) bound. Code in `code/descent_lemma.py`.

[a] Indeed, the quadratic part of f dominates, and in the y-coordinate the update is approximately $y_{k+1} \approx (1 - 10\eta)y_k = -y_k$, so the iterates oscillate back and forth across the valley without contracting. The function value $f(w_k)$ increases on

about half of the 30 steps (e.g. $f(w_0) \approx 38.3 \to f(w_1) \approx 43.4$), and only makes slow net progress, ending at $f(w_{30}) \approx 15.5$ with $\|\nabla f(w_{30})\| \approx 17$.

Note that the descent lemma is really just a one-step upper bound and not a convergence guarantee (we will build those later).

Theorem 2.5 (Convergence to Criticality of GD)

Suppose that f is L-smooth and $f^* := \inf f > -\infty$. Consider the iterates $w_{k+1} \leftarrow w_k - \frac{1}{L}\nabla f(w_k)$. Then,

$$\min_{k\in[K-1]} \|\nabla f(w_k)\| \leq \sqrt{\frac{2L\big(f(w_0) - f^*\big)}{K}} \tag{176}$$

Therefore, for any $\epsilon > 0$, we have

$$K \geq \frac{2L}{\epsilon^2}\big(f(w_0) - f^*\big) \implies \min_{k\in[K-1]} \|\nabla f(w_k)\| \leq \epsilon \tag{177}$$

Proof. We apply the descent lemma to each iterate and get

$$f(w_{k+1}) \leq f(w_k) - \frac{1}{2L}\|\nabla f(w_k)\|^2 \implies \|\nabla f(w_k)\|^2 \leq 2L\big(f(w_k) - f(w_{k+1})\big) \tag{178}$$

Now the minimum has to be less than the average, so

$$\begin{aligned}
\min_{k\in[K-1]} \|\nabla f(w_k)\|^2 &\leq \frac{1}{K}\sum_{k=0}^{K-1} \|\nabla f(w_k)\|^2 && (179)\\
&\leq \frac{2L}{K}\sum_{k=0}^{K-1} \big(f(w_k) - f(w_{k+1})\big) && (180)\\
&= \frac{2L}{K}\big(f(w_0) - f(w_K)\big) && (181)\\
&\leq \frac{2L}{K}\big(f(w_0) - f^*\big) && (182)
\end{aligned}$$

Therefore,

$$\min_{k\in[K-1]} \|\nabla f(w_k)\| = \sqrt{\min_{k\in[K-1]} \|\nabla f(w_k)\|^2} \leq \sqrt{\frac{2L\big(f(w_0) - f^*\big)}{K}} \tag{183}$$

This means that up to constants, $K \sim \epsilon^{-2}$ for a gradient bound. So if $\epsilon = 10^{-6}$, then we would need $\epsilon^{-2} = 10^{12}$ iterations! This is not good enough, so we would like some extra regularity assumptions, such as convexity.[2]

Example 2.2 (Reaching an ϵ-Critical Point)

Consider the nonconvex function

$$f : \mathbb{R}^2 \to \mathbb{R}, \qquad f(x, y) = \frac{3}{20}x^2 + 5y^2 + 2\sin x \sin y. \tag{184}$$

[2]In fact, most of these regularity/geometric assumptions that people used to prove guarantees amounted to strong convexity.

Its gradient and Hessian are

$$\nabla f(x,y) = \begin{pmatrix} \frac{3}{10}x + 2\cos x \sin y \\ 10y + 2\sin x \cos y \end{pmatrix}, \quad \nabla^2 f(x,y) = \begin{pmatrix} \frac{3}{10} & 0 \\ 0 & 10 \end{pmatrix} + 2\begin{pmatrix} -\sin x \sin y & \cos x \cos y \\ \cos x \cos y & -\sin x \sin y \end{pmatrix} \tag{185}$$

The second matrix has eigenvalues $\cos(x+y)$ and $-\cos(x-y)$, which lie in $[-1,1]$, so $\|\nabla^2 f(x,y)\|_{\text{op}} \le 10 + 2 = 12$ and f is L-smooth with $L = 12$. The x-direction is very flat (curvature $\frac{3}{10}$ against 10 in the y-direction), so gradient descent with $\eta = 1/L$ is slow, and the sine terms make f nonconvex. Numerically, $f^* \approx -0.0096$, attained near $(0.63, -0.12)$.
We run gradient descent with $\eta = 1/12$ from $w_0 = (8,1)$, where $f(w_0) \approx 16.3$, and set $\epsilon = 0.1$. The gradient norm first drops below ϵ at iteration $k = 96$. On the other hand, the theorem only guarantees this after

$$K \ge \frac{2L}{\epsilon^2}\big(f(w_0) - f^*\big) \approx \frac{2 \cdot 12}{0.01} \cdot 16.3 \approx 3.9 \times 10^4 \tag{186}$$

iterations. After $K = 100$ iterations it gives $\min_{k<K} \|\nabla f(w_k)\| \le \sqrt{2L(f(w_0) - f^*)/K} \approx 1.98$, while the actual value is about 0.088. So the bound holds at every K (right panel), but it is quite conservative.

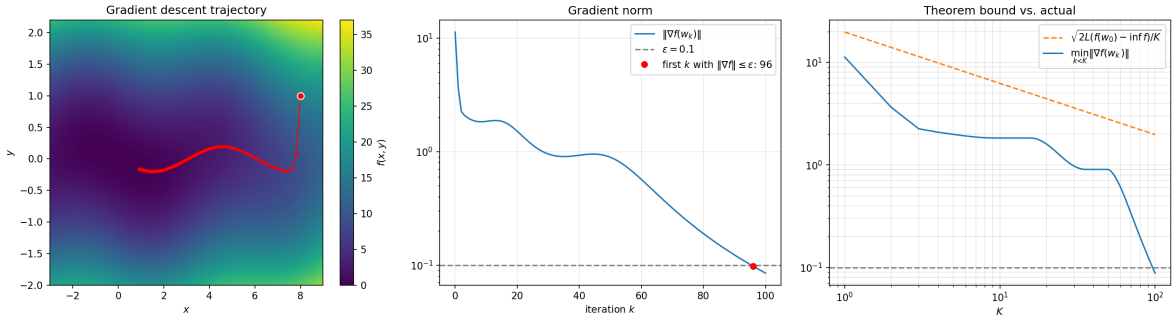


Figure 6: Left: trajectory (red) over the heatmap of f. Middle: $\|\nabla f(w_k)\|$ on a log scale, first dropping below $\epsilon = 0.1$ at $k = 96$. Right: $\min_{k<K} \|\nabla f(w_k)\|$ stays below the guarantee $\sqrt{2L(f(w_0) - f^*)/K}$ for every $K \le 100$. Code in `code/convergence_to_criticality.py`.

The previous example reached ϵ after roughly 0.25% of the guaranteed number of iterations. The theorem is not vacuous, however: it is tight up to a constant factor for gradient descent. To see this, we want a function where each step makes as little progress as the descent lemma allows, i.e. where the gradient stays just above ϵ for as long as possible, which is a long ramp of slope slightly larger than ϵ.

Example 2.3 (A Near-Extremal Function for the Criticality Bound)

Let $L = 12$, $s = 0.101 > \epsilon = 0.1$, $\delta = s/L$, and consider

$$f : \mathbb{R}^2 \to \mathbb{R}, \qquad f(x,y) = s\delta\Big(\sqrt{1 + (x/\delta)^2} - 1\Big) + 6y^2. \tag{187}$$

Its gradient and Hessian are

$$\nabla f(x,y) = \begin{pmatrix} \dfrac{sx}{\sqrt{\delta^2 + x^2}} \\ 12y \end{pmatrix}, \quad \nabla^2 f(x,y) = \begin{pmatrix} \dfrac{s\delta^2}{(\delta^2 + x^2)^{3/2}} & 0 \\ 0 & 12 \end{pmatrix} \tag{188}$$

Since $\frac{s\delta^2}{(\delta^2+x^2)^{3/2}} \le \frac{s}{\delta} = L$, we have $\|\nabla^2 f\|_{\text{op}} \le 12$, so f is L-smooth with $L = 12$, and $f^* = 0$ is attained at the origin. Away from the origin, $\partial_x f \approx s$ is almost constant, so the x-coordinate moves by only $\eta s = s/L$ per step, and f decreases by about s^2/L per step. This f happens to be convex, but the theorem does not use convexity, so it still applies.

We run gradient descent with $\eta = 1/12$ from $w_0 = (1000, 1)$, where $f(w_0) \approx 107$. The theorem guarantees $\|\nabla f(w_k)\| \le \epsilon$ for some $k < K$ once

$$K \ge \frac{2L}{\epsilon^2}\big(f(w_0) - f^*\big) \approx \frac{2 \cdot 12}{0.01} \cdot 107 \approx 2.6 \times 10^5. \tag{189}$$

The gradient norm first drops below ϵ at iteration $k = 118{,}805$, which is 46% of the guaranteed number. Heuristically, this is about $L\Delta/s^2 \approx \frac{1}{2} \cdot \frac{2L\Delta}{\epsilon^2}$, so the remaining gap of a factor of 2 is exactly the slack in the descent lemma: on a ramp, f decreases by $\eta\|\nabla f\|^2 = \frac{1}{L}\|\nabla f\|^2$ per step, rather than the guaranteed $\frac{1}{2L}\|\nabla f\|^2$.

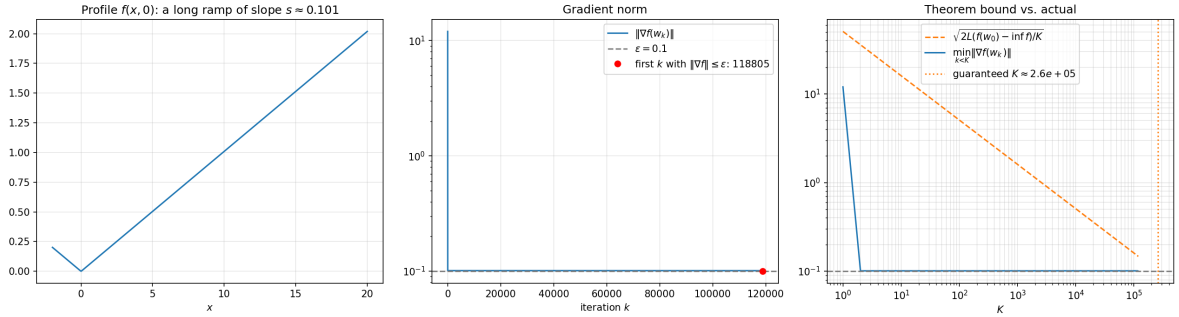


Figure 7: Left: the profile $f(x, 0)$. Middle: $\|\nabla f(w_k)\|$, which stays just above ϵ for $k \approx 1.2 \times 10^5$ iterations. Right: the guarantee $\sqrt{2L(f(w_0) - f^*)/K}$ (orange) and the actual $\min_{k<K} \|\nabla f(w_k)\|$ (blue) on a log-log scale; the vertical dotted line is the guaranteed K for ϵ. Code in `code/convergence_to_criticality.py`.

2.1 Convexity

Recall what a convex function is. We can immediately use this to bound the suboptimality with the distance to the minimizer.

Theorem 2.6 (Convergence of Gradient Descent for Smooth Convex Functions)

Suppose $f : \mathbb{R}^d \to \mathbb{R}$ is convex and L-smooth, and admits a minimizer w^*. Then, gradient descent with step size $\eta = \frac{1}{L}$ satisfies, for every $K \ge 1$,

$$f(w_K) - f^* \le \frac{L\|w_0 - w^*\|^2}{2K}. \tag{190}$$

Consequently, for every $\epsilon > 0$,

$$K \ge \frac{L\|w_0 - w^*\|^2}{2\epsilon} \implies f(w_K) - f^* \le \epsilon \tag{191}$$

giving an iteration complexity of $O(\epsilon^{-1})$.

Proof. By convexity,

$$f(w_k) - f^* \le \langle \nabla f(w_k), w_k - w^* \rangle. \tag{192}$$

On the other hand, by the descent lemma and the update rule,

$$f(w_{k+1}) \le f(w_k) + \langle \nabla f(w_k), w_{k+1} - w_k \rangle + \frac{L}{2}\|w_{k+1} - w_k\|^2 \tag{193}$$

$$= f(w_k) - \frac{1}{2L}\|\nabla f(w_k)\|^2. \tag{194}$$

Combining these inequalities gives

$$f(w_{k+1}) - f^* \leq \langle \nabla f(w_k), w_k - w^* \rangle - \frac{1}{2L}\|\nabla f(w_k)\|^2. \tag{195}$$

Now, expanding the squared distance to the minimizer,

$$\|w_{k+1} - w^*\|^2 = \left\| w_k - w^* - \frac{1}{L}\nabla f(w_k) \right\|^2 \tag{196}$$

$$= \|w_k - w^*\|^2 - \frac{2}{L}\langle \nabla f(w_k), w_k - w^* \rangle + \frac{1}{L^2}\|\nabla f(w_k)\|^2. \tag{197}$$

Rearranging yields the key inequality

$$f(w_{k+1}) - f^* \leq \frac{L}{2}\left(\|w_k - w^*\|^2 - \|w_{k+1} - w^*\|^2\right). \tag{198}$$

Summing over $k = 0, \dots, K-1$, the right-hand side telescopes:

$$\sum_{k=0}^{K-1} \left(f(w_{k+1}) - f^*\right) \leq \frac{L}{2} \sum_{k=0}^{K-1} \left(\|w_k - w^*\|^2 - \|w_{k+1} - w^*\|^2\right) \tag{199}$$

$$= \frac{L}{2}\left(\|w_0 - w^*\|^2 - \|w_K - w^*\|^2\right) \tag{200}$$

$$\leq \frac{L}{2}\|w_0 - w^*\|^2. \tag{201}$$

Since the descent lemma ensures that $f(w_{k+1}) \leq f(w_k)$, we have

$$K\big(f(w_K) - f^*\big) \leq \sum_{k=0}^{K-1} \big(f(w_{k+1}) - f^*\big). \tag{202}$$

Therefore,

$$f(w_K) - f^* \leq \frac{L\|w_0 - w^*\|^2}{2K}. \tag{203}$$

Now let's upgrade convexity to strongly convexity. It has two very important properties.

Lemma 2.7 (Unique Minimizer and Quadratic Growth of Strongly Convex Functions)

Suppose that f is μ-strongly convex. Then, f has a unique minimizer w^*, and for all $w \in \mathbb{R}^d$,

$$\frac{\mu}{2}\|w - w^*\|^2 \leq f(w) - f(w^*) \qquad \text{(quadratic growth)} \tag{204}$$

$$\mu\|w - w^*\| \leq \|\nabla f(w)\| \qquad \text{(error bound)} \tag{205}$$

Proof. To prove existence, we first claim that f is coercive. Indeed,

$$f(x) \geq f(0) + \langle \nabla f(0), x \rangle + \frac{\mu}{2}\|x\|^2 \tag{206}$$

and the quadratic term dominates as $\|x\| \to +\infty$. Therefore, there exists a large enough $c > 0$ s.t. the set

$$A = \{x \in \mathbb{R}^d \colon f(x) \leq c\} \tag{207}$$

is nonempty. It is also closed since f is continuous. By coercivity, A is bounded, so by Heine-Borel, it is compact, and by the extreme value theorem, there exists a minimizer w^*. To prove uniqueness,

assume that w_1, w_2 are minimizers with $w_1 \neq w_2$. Then, by strong convexity,

$$f(w_1) \geq f(w_2) + \langle \nabla f(w_2), w_1 - w_2 \rangle + \frac{\mu}{2}\|w_1 - w_2\|^2 \tag{208}$$

Since f is differentiable and w_2 is a minimizer, $\nabla f(w_2) = 0$, and so $f(w_1) \geq f(w_2) + \frac{\mu}{2}\|w_1 - w_2\|^2 > f(w_2)$, contradicting the fact that w_1 is a minimizer. Finally, the claimed inequality follows from strong convexity since

$$f(w) - f(w^*) \geq \langle \nabla f(w^*), w - w^* \rangle + \frac{\mu}{2}\|w - w^*\|^2 = \frac{\mu}{2}\|w - w^*\|^2 \tag{209}$$

For the error bound, we apply strongly monotone gradient to w and w^* and use $\nabla f(w^*) = 0$ to get

$$\|\nabla f(w)\|\|w - w^*\| \geq \langle \nabla f(w) - \nabla f(w^*), w - w^* \rangle \geq \mu\|w - w^*\|^2 \tag{210}$$

where the first inequality is Cauchy-Schwarz. Dividing by $\|w - w^*\|$ gives the claim (trivial if $w = w^*$).

The motivation of strongly convex functions comes from regularization.

Example 2.4 (Ridge Loss is Strongly Convex)

If g is convex and differentiable, then $f(w) = g(w) + \frac{\lambda}{2}\|w\|^2$ is λ-strongly convex for any $\lambda > 0$, since $f - \frac{\lambda}{2}\|\cdot\|^2 = g$ is convex.

Lemma 2.8 (L-Smooth and μ-Strongly Convex)

If $f : \mathbb{R}^d \to \mathbb{R}$ is both L-smooth and μ-strongly convex, then

$$0 \leq \mu \leq L \tag{211}$$

Proof. f is L-smooth, so $\|\nabla f(x) - \nabla f(w)\| \leq L\|x - w\|$. By multiplying $\|x - w\|$ on both sides and applying Cauchy-Schwarz, we get

$$\langle \nabla f(x) - \nabla f(w), x - w \rangle \leq \|\nabla f(x) - \nabla f(w)\|\|x - w\| \leq L\|x - w\|^2 \tag{212}$$

Since f is μ-strongly convex,

$$\langle \nabla f(x) - \nabla f(w), x - w \rangle \geq \mu\|x - w\|^2 \tag{213}$$

Therefore, combining the two for any $x \neq w$ gives

$$\mu\|x - w\|^2 \leq L\|x - w\|^2 \implies \mu \leq L \tag{214}$$

So we are able to put some bounds on the norm of the gradient, and a natural next question is whether we can guarantee something about the distance to the function. For that, we need additional regularity assumptions, such as strong convexity.

Theorem 2.9 (Linear Convergence of Objective Value on GD Under Strong Convexity)

Suppose that f is L-smooth and μ-strongly convex. Then, the iterates of GD with step size $\eta = 1/L$

satisfy

$$f(w_k) - f(w^*) \leq \left(1 - \frac{\mu}{L}\right)^k \left(f(w_0) - f(w^*)\right) \tag{215}$$

Therefore, for any $\epsilon > 0$, we have

$$K \geq \frac{L}{\mu} \log\left(\frac{f(w_0) - f(w^*)}{\epsilon}\right) \implies f(w_K) - f(w^*) \leq \epsilon \tag{216}$$

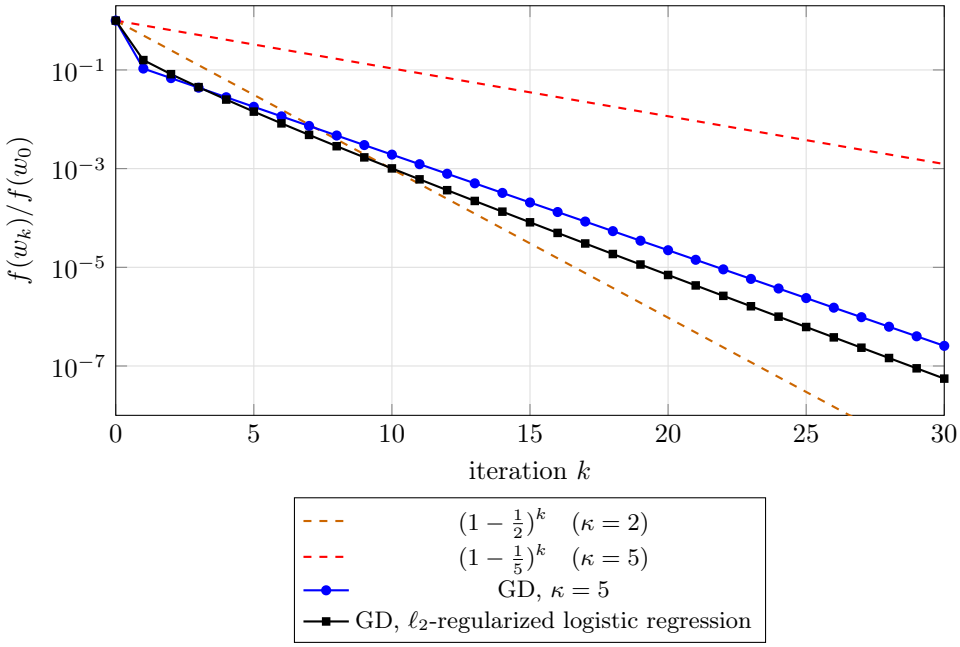


Figure 8: Relative loss $f(w_k)/f(w_0)$ (assume $f^* = 0$) of gradient descent with step size $1/L$ on a strongly convex quadratic on a log scale. This looks linear on a log-plot, hence the name *linear convergence*. In other words, to further reduce the error by a factor of K, we only need to iterate through at a constant number of steps. The black curve is a messier run, GD on ℓ_2-regularized logistic regression ($n = 200$, $d = 20$, badly scaled features, $\lambda = L/5$ so that $\kappa \leq L/\lambda = 5$), with f^* computed numerically and the y-axis $\frac{f(w_k)-f^*}{f(w_0)-f^*}$. It is also λ-strongly convex with $L/\lambda = 5$, so the same bound $(1 - \frac{1}{5})^k$ applies, and the curve stays below it, though its slope is slightly uneven rather than perfectly constant.

Proof. f is μ-strongly convex, and so it is μ-PL (we will cover this in the next subsection), which we know converges linearly.

Corollary 2.10 (Linear Convergence of Distance to Minimizer on GD Under Strong Convexity)

Suppose that f is L-smooth and μ-strongly convex. For any $\epsilon > 0$, we have

$$K \geq \frac{L}{\mu} \log\left(\frac{f(w_0) - f(w^*)}{\epsilon}\right) \implies \|w_K - w^*\| \leq \sqrt{\frac{2\epsilon}{\mu}} \tag{217}$$

Proof. Strong convexity implies quadratic growth $f(w) - f(w^*) \geq \frac{\mu}{2}\|w - w^*\|^2$.

Note that for L-smooth functions in general, GD converges to criticality at a rather slow rate of $O(\epsilon^{-2})$, compared to $K \gtrsim \log(\epsilon^{-1})$. If $\epsilon = 10^{-6}$, then $\epsilon^{-2} = 10^{12}$—which is a very high number of iterations to get a gradient under 10^{-6}—while $\log(\frac{1}{\epsilon}) \approx 14$—much cheaper to get the same bound on function suboptimality *and* distance to minimizer.

Definition 2.2 (Condition Number)

Thus, how fast GD converges is dictated by the ratio

$$\kappa = \frac{L}{\mu} \in [1, \infty] \tag{218}$$

known as the **condition number**.

Example 2.5 (Optimization on Quadratics with Varying Condition Numbers)

Consider the quadratic function

$$f : \mathbb{R}^d \to \mathbb{R}, \quad f(x) = \frac{1}{2} x^T A x - b^T x + c \tag{219}$$

where $A \in \mathbb{R}^{d \times d}$ are (strictly) positive definite. The gradient and Hessian are

$$\nabla f(x) = Ax - b, \quad \nabla^2 f(x) = A \tag{220}$$

If the eigenvalues of A satisfy

$$0 < \mu = \lambda_{\min}(A) \leq \lambda_{\max}(A) = L \tag{221}$$

then f is L-smooth and μ-strongly convex. Therefore, if $L \gg \mu$, then gradient descent will not converge as fast from the theorem above. Visually, we can see that this means the quadratic will become very "stretched," causing the updates to bounce around, the global minimum.

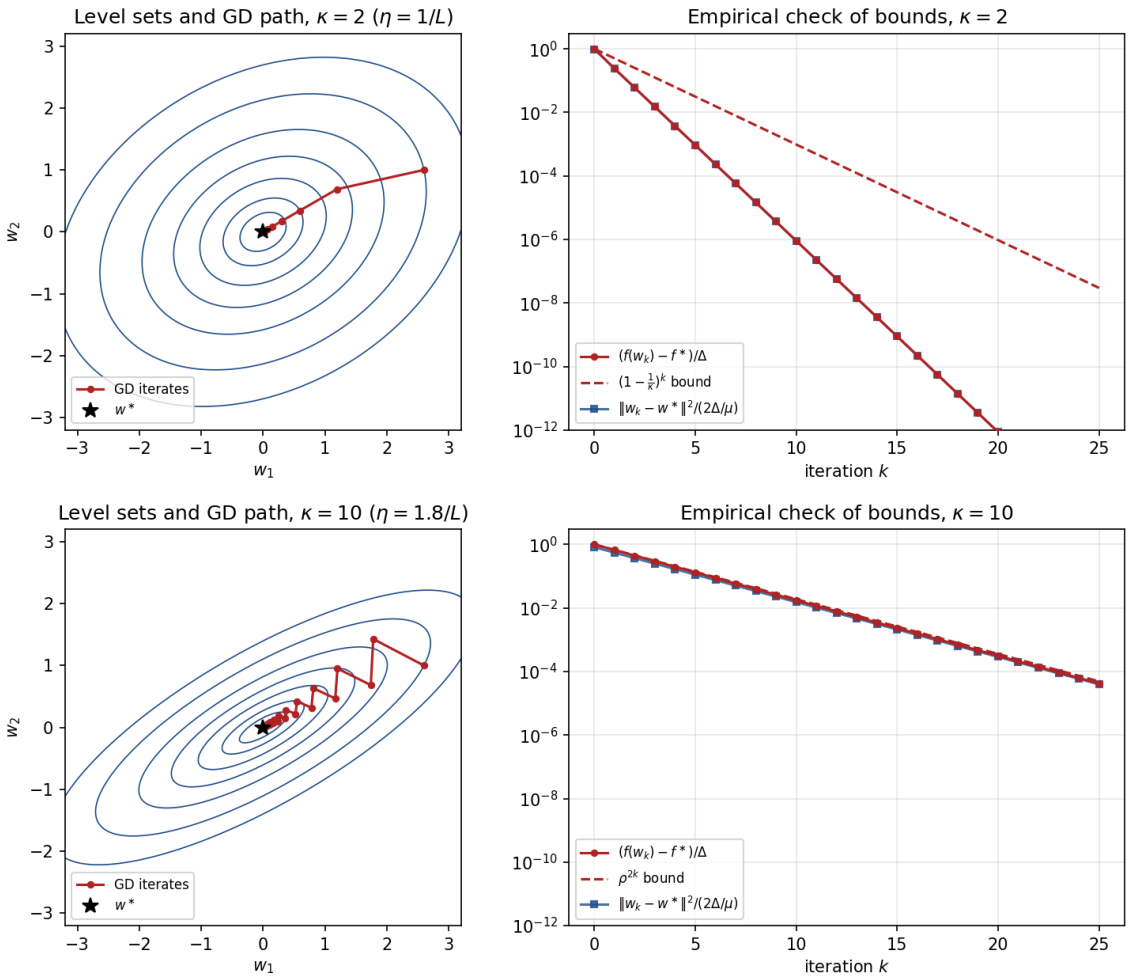


Figure 9: Gradient descent on two quadratics $f(w) = \frac{1}{2}w^T Aw$ with $\mu = 1$ and $L = 2$ (top, $\kappa = 2$) or $L = 10$ (bottom, $\kappa = 10$). Left: level sets of f and the GD iterates (red). Right: empirical check of the bounds on $f(w_k) - f^*$ and $\|w_k - w^*\|^2$, with $\Delta = f(w_0) - f^*$. Top: $\eta = 1/L$, where the bounds are $(1 - \frac{1}{\kappa})^k \Delta$ and $\frac{2\Delta}{\mu}(1 - \frac{1}{\kappa})^k$. Bottom: a larger step $\eta = 1.8/L$ (still below $2/L$), so the iterates overshoot and zig-zag across the narrow valley while making slow progress along its long axis; the theorem no longer applies, but for a quadratic each eigen-coordinate contracts by $|1 - \eta\lambda_i| \le \rho = 0.8$, giving the bounds with ρ^{2k} in place of $(1 - \frac{1}{\kappa})^k$. Code: `code/quadratics.py`.

This is pretty fast, but is there a faster algorithm than GD? For a particular function, we may be able to exploit its mathematical properties such find such a minima. For example, we can find the closed form of the minimum of a quadratic function by finding the roots of its gradient, basically making our algorithm $O(1)$. Therefore, we wish to evaluate GD over a *class of functions*. For such a class, an algorithm may do really well on some $f \in \mathcal{F}$ but may do terribly on other functions. Therefore, we wish to find the minimax oracle error of GD on some defined $\mathcal{F}$. The following result gives a lower bound on the convergence of GD on strongly convex, L-smooth functions.

Theorem 2.11 (Convergence Lower Bound of GD on Strongly Convex Functions)

Let $L, \mu, \Delta > 0$, $\kappa = \frac{L}{\mu} \ge 1$, and $K \ge 1$. Then, there exists a μ-strongly convex, L-smooth function

$f : \mathbb{R}^{2K} \to \mathbb{R}$ with $f(w^*) = 0$ and $f(0) = \Delta$ such that every method started at $w_0 = 0$ and satisfying

$$w_{k+1} \in \text{span}\{\nabla f(w_0), \dots, \nabla f(w_k)\}^a \tag{222}$$

obeys

$$\frac{f(w_K) - f^*}{\Delta} \geq \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1}\right)^{2K} \tag{223}$$

The same holds for the "best" iterate. Whenever $\sqrt{\kappa} \geq 4$ and $\epsilon \in (0, 1)$, then

$$K \leq \frac{\sqrt{\kappa}}{8} \log\left(\frac{1}{\epsilon}\right) \implies \frac{f(w_K) - f^*}{\Delta} > \epsilon \tag{224}$$

[a] GD with $w_0 = 0$ automatically satisfies this, so we can think of this as a generalization of GD.

Proof. The general idea is to first construct the minimizer w^* with exponentially decaying entries and a "chain" function f for it. We verify that the chain function is indeed in our class of L-smooth μ-strongly convex functions, and then show that the chain function only reveals one entry per query. We convert this fact into a lower bound. For simplicity, write

$$p = \frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \in (0, 1), \quad a = \sqrt{\frac{2\Delta(1+p)^2}{L(1-p^2)}} \tag{225}$$

1. *Construct Chain Function.* Let us define our chain function[a] as

$$f : \mathbb{R}^{2K} \to \mathbb{R}, \quad f(w) = \frac{L}{2(1+p)^2}\left[(1-p^2)(a-w_1)^2 + \sum_{i=1}^{2K-1}(w_{i+1} - pw_i)^2\right] \tag{226}$$

   with minimizer $w^* = (a, ap, ap^2, \dots, ap^{2K-1})$.[b] We denote

$$f(0) = \frac{L}{2(1+p)^2}(1-p^2)a^2 = \Delta \tag{227}$$

2. *Verify Smoothness and Strong Convexity.* We can verify that f is μ-strongly convex (and hence μ-PL) by showing that f is C^2 and $\mu I \preceq \nabla^2 f(w)$, and that f is L-smooth by showing that f is C^2 and $\nabla^2 f(w) \preceq LI$. Since f is a quadratic, we can write it as

$$f(w) = \frac{L}{2(1+p)^2}\big(w^T B w - 2(1-p^2)a\, e_1^T w + (1-p^2)a^2\big), \tag{228}$$

   where e_1 is the 1st standard basis vector of $\mathbb{R}^{2K}$ and B is the tridiagonal matrix

$$B = \begin{pmatrix} 1 & -p & & & \\ -p & 1+p^2 & -p & & \\ & -p & 1+p^2 & & \\ & & & \ddots & -p \\ & & & -p & 1 \end{pmatrix} \tag{229}$$

   Expanding, we get

$$w^T B w = w_1^2 + w_{2K}^2 + (1+p^2)\sum_{i=2}^{2K-1} w_i^2 - 2p\sum_{i=1}^{2K-1} w_i w_{i+1} \tag{230}$$

Using the fact that $2|uv| \leq u^2 + v^2$, we get that

$$w^T B w \geq w_1^2 + w_{2K}^2 + (1+p^2)\sum_{i=2}^{2K-1} w_i^2 - p\left(w_1^2 + w_{2K}^2 + 2\sum_{i=2}^{2K-1} w_i^2\right) \tag{231}$$

$$= (1-p)(w_1^2 + w_{2K}^2) + (1-p)^2 \sum_{i=2}^{2K-1} w_i^2 \tag{232}$$

$$\geq (1-p)^2 \|w\|^2 \tag{233}$$

where for the last inequality, we used the fact that $(1-p) \geq (1-p)^2$. A similar argument yields $w^T B w \leq (1+p)^2\|w\|^2$. Therefore,

$$\mu I = \frac{L}{\kappa} I = L\left(\frac{1-p}{1+p}\right)^2 I \preceq \underbrace{\frac{L}{(1+p)^2} B}_{\nabla^2 f(w)} \preceq LI \tag{234}$$

3. *Each query reveals one coordinate.* We wish to show that the iterates of GD on our function f satisfies $w_{k+1} \in \text{span}\{\nabla f(w_0), \dots, \nabla f(w_k)\}$. We do this with induction. Let

$$E_0 = \{0\}, \quad E_j = \text{span}\{e_1, \dots, e_j\} \tag{235}$$

The gradient of f is

$$\nabla f(w) = \frac{L}{(1+p)^2}\big(Bw - (1-p^2)ae_1\big) \tag{236}$$

So, for $w_0 = 0 \in E_0$, we have $w_1 \in \text{span}\{\nabla f(0)\} = E_1$. This proves the base case. Now suppose that $w_0, \dots, w_k \in E_k$. Since B is tridiagonal, paying attention to the entries of the matrix will show us that $Bw_j \in E_{k+1}$ for every $j \leq k$. Hence, $\nabla f(w_j) = \frac{L}{(1+p)^2} Bw_j - \frac{L(1-p^2)}{(1+p)^2} ae_1 \in E_{k+1}$, and so $w_{k+1} \in \text{span}\{\nabla f(w_0), \dots, \nabla f(w_k)\} \subset E_{k+1}$.

4. *Construct Lower Bound.* For index K, we know that for any vector $w \in E_K$ (such as the iterate w_K, or any earlier iterate), we must have that

$$|\underbrace{(w - w^*)_{K+1}}_{z_{K+1}}| = ap^K \tag{237}$$

since $w_{K+1} = 0$. Our goal is to use this as a lower bound for $f(w) - f(w^*)$. To do so, notice that since $f(w^*) = 0$ and $w_{j+1}^* = pw_j^*$, we have

$$f(w) - f(w^*) = \frac{L}{2(1+p)^2}\left[(1-p^2)z_1^2 + \sum_{j=1}^{2K-1}(z_{j+1} - pz_j)^2\right] \tag{238}$$

By first expanding $z_{j+1} = pz_j + (z_{j+1} - pz_j)$ and subsequently Cauchy-Schwarz, we have

$$ap^K = |z_{K+1}| = \left|p^K z_1 + \sum_{j=1}^{K} p^{K-j}(z_{j+1} - pz_j)\right| \tag{239}$$

$$= \left|\left\langle \begin{pmatrix} \frac{p^K}{\sqrt{1-p^2}} \\ p^{K-1} \\ \vdots \\ 1 \end{pmatrix}, \begin{pmatrix} \sqrt{1-p^2} z_1 \\ z_2 - pz_1 \\ \vdots \\ z_{K+1} - pz_K \end{pmatrix} \right\rangle\right| \tag{240}$$

$$\leq \left(\frac{p^{2K}}{(1-p^2)} + \sum_{j=1}^{K} p^{2(K-j)}\right)^{1/2} \cdot \left((1-p^2)z_1^2 + \sum_{j=1}^{K}(z_{j+1} - pz_j)^2\right)^{1/2} \tag{241}$$

The first term simplifies to

$$\frac{p^{2K}}{1-p^2} + \frac{1-p^{2K}}{1-p^2} = \frac{1}{1-p^2} \tag{242}$$

The second term can be bounded as

$$(1-p^2)z_1^2 + \sum_{j=1}^{K}(z_{j+1}-pz_j)^2 \leq (1-p^2)z_1^2 + \sum_{j=1}^{2K-1}(z_{j+1}-pz_j)^2 = \frac{2(1+p)^2}{L}\big(f(w)-f(w^*)\big) \tag{243}$$

Squaring and combining all of this yields

$$a^2p^{2K} \leq \frac{2(1+p)^2}{L(1-p^2)}\big(f(w)-f(w^*)\big) \tag{244}$$

Rearranging and simplifying gives

$$f(w)-f(w^*) \geq \frac{L(1-p^2)}{2(1+p)^2}a^2p^{2K} = \Delta\left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^{2K} \tag{245}$$

Since every earlier iterate w_j lies in $E_j \subset E_K$, the same bound holds for the best iterate. For the last claim, note that $\log(1/p) \leq \frac{2}{\sqrt{\kappa}-1}$, so $p^{2K} \geq \exp\big(-\frac{4K}{\sqrt{\kappa}-1}\big)$. If $K \leq \frac{\sqrt{\kappa}}{8}\log(1/\epsilon)$ and $\sqrt{\kappa} \geq 4$, then $\frac{4K}{\sqrt{\kappa}-1} \leq \frac{\sqrt{\kappa}}{2(\sqrt{\kappa}-1)}\log(1/\epsilon) \leq \frac{2}{3}\log(1/\epsilon)$, and so $p^{2K} > \epsilon$.

[a] We can interpret this as the worst smooth and strongly convex function in the world.
[b] It is easy to show that this is indeed the minimizer since $f(w^*) = 0$ and $f(w) \geq 0$.

So what happens when $\sqrt{\kappa}\log(1/\epsilon) \lesssim K \lesssim \kappa\log(1/\epsilon)$. The answer to this question depends on the family of functions.

2.2 Polyak-Lojasiewicz Inequality

In recent decades, strong convexity fell out of favor for the PL-inequality, which generalizes the notion of strong convexity.

Definition 2.3 (Polyak-Lojasiewicz Inequality)

A differentiable function $f : \mathbb{R}^d \to \mathbb{R}$ satisfies the **PL-inequality**[a] if there exists a $\mu > 0$ s.t.

$$\mu\big(f(x)-f^*\big) \leq \frac{1}{2}\|\nabla f(x)\|^2 \text{ for all } x \in \mathbb{R}^d \tag{246}$$

[a] This was published in 1963 and was ignored until 2016.

The first thing is that we should think of this as a weaker notion of strong convexity. But this is still a relatively strong assumption, and in particular it yields that $\nabla f(x) = 0 \implies f(x) = f^*$

Lemma 2.12 (Strong Convexity Implies PL-Inequality)

Suppose $f : \mathbb{R}^d \to \mathbb{R}$ is differentiable and μ-strongly convex. Then, it satisfies μ-PL.

Proof. Since f is strongly convex, it attains the global minimizer w^*. Therefore,

$$f(w^*) \geq f(w) + \langle \nabla f(w), w^* - w \rangle + \frac{\mu}{2}\|w - w^*\|^2 \tag{247}$$

$$= f(w) + \frac{\mu}{2}\left\|(w^* - w) + \frac{\nabla f(w)}{\mu}\right\|^2 - \frac{1}{2\mu}\|\nabla f(w)\|^2 \tag{248}$$

$$\geq f(w) - \frac{1}{2\mu}\|\nabla f(w)\|^2 \tag{249}$$

and we rearrange the terms to get PL.

PL does not require convexity nor even a unique minimizer. Intuitively, we should think of a strongly convex function as a "bowl." So how should we interpret a function that satisfies PL? We should think of it as "almost a bowl," which can be curvy.

Example 2.6 (PL and non-PL Functions)

Here are some PL and non-PL functions.

1. *PL (strongly convex).* $f(x) = \frac{1}{2}x^T Ax$ with $A \succ 0$. Here $\frac{1}{2}\|\nabla f(x)\|^2 = \frac{1}{2}x^T A^2 x \geq \lambda_{\min}(A) \cdot \frac{1}{2}x^T Ax$, so f is $\lambda_{\min}(A)$-PL. It is also convex since $\nabla^2 f = A \succeq 0$ (in fact $\lambda_{\min}(A)$-strongly convex).
2. *PL, but not strongly convex (rank-deficient least squares).* $f(w) = \frac{1}{2}\|Xw - y\|^2$ with $X \in \mathbb{R}^{n\times d}$ of rank $r < d$. It is convex, but constant along $\ker X$, so it has no unique minimizer and is not strongly convex. Still, with $\sigma^+_{\min}(X)$ the smallest nonzero singular value of X and P the orthogonal projection onto range(X), we have $\nabla f(w) = X^T(Xw - y) = X^T P(Xw - y)$ and $f(w) - f^* = \frac{1}{2}\|P(Xw - y)\|^2$, so

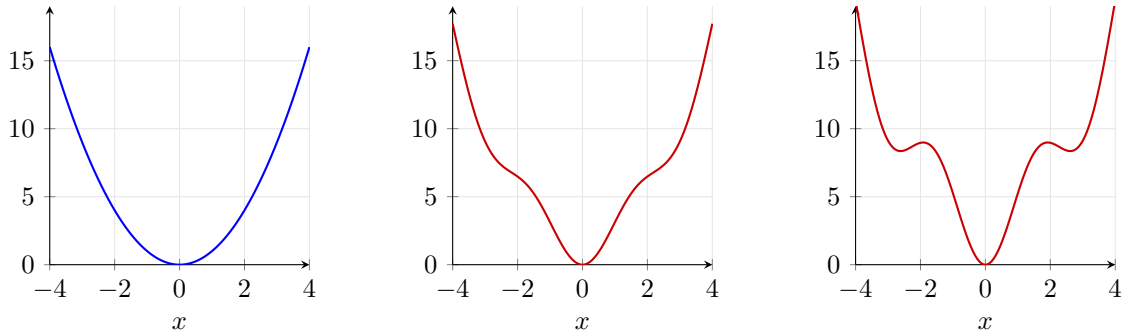
$$\frac{1}{2}\|\nabla f(w)\|^2 \geq \frac{\sigma^+_{\min}(X)^2}{2}\|P(Xw - y)\|^2 = \sigma^+_{\min}(X)^2\big(f(w) - f^*\big) \tag{250}$$

   that is, f is $\sigma^+_{\min}(X)^2$-PL, and this constant is tight (the ratio $\frac{\|\nabla f\|^2}{2(f-f^*)}$ equals $\sigma^+_{\min}(X)^2$ when $P(Xw - y)$ is a left singular vector for $\sigma^+_{\min}(X)$). It is convex since $\nabla^2 f = X^T X \succeq 0$. More generally, $w \mapsto g(Xw)$ with g strongly convex is PL, for the same reason.
3. *PL, but not convex.* $f(x) = x^2 + 3\sin^2 x$ (see the next example). Its second derivative $f''(x) = 2 + 6\cos 2x$ reaches -4, so it is not convex, yet numerically $\inf_{x\neq 0} \frac{|f'(x)|^2}{2f(x)} \approx 0.1755 > 0$.
4. *Not PL (flat minimum).* $f(x) = x^4$ is convex with a unique minimizer, but $\frac{1}{2}|f'(x)|^2 = 8x^6$ while $f(x) - f^* = x^4$, so $8x^6 \geq \mu x^4$ fails for $|x| < \sqrt{\mu/8}$ (the ratio $\frac{|f'|^2}{2(f-f^*)} = 8x^2 \to 0$). It is convex since $f''(x) = 12x^2 \geq 0$. Convexity is not enough: PL forces curvature at least μ in the sense of growth of the gradient near the minimum.
5. *Not PL (minimum at infinity).* $f(x) = e^{-x}$ is convex and bounded below with $f^* = 0$, but $\frac{1}{2}|f'(x)|^2 = \frac{1}{2}e^{-2x}$ and $f(x) - f^* = e^{-x}$, so $\frac{1}{2}e^{-2x} \geq \mu e^{-x}$ fails as $x \to \infty$ (the ratio is $\frac{1}{2}e^{-x} \to 0$, e.g. $\approx 10^{-9}$ at $x = 20$). It is convex since $f''(x) = e^{-x} > 0$. Equivalently, the infimum is not attained, which PL rules out.
6. *Not PL (non-global stationary point).* $f(x) = (x^2 - 1)^2$ has a stationary point at $x = 0$ with $f(0) = 1 > 0 = f^*$, so the inequality fails there ($0 \geq \mu$ is false). It is not convex since $f''(x) = 12x^2 - 4 < 0$ for $|x| < 1/\sqrt{3}$. The same happens for $x^2 + 5\sin^2 x$ below. In general, any function with a local minimum, local maximum, or saddle that is not a global minimizer is not PL.
7. *Not PL (unbounded below).* $f(x, y) = x^2 - y^2$ has no finite f^*. PL (as defined here) requires f to be bounded below. It is also not convex, since its Hessian $\mathrm{diag}(2, -2)$ is indefinite.

Example 2.7 (μ-PL vs μ-Strongly Convex)

Compare the functions.

1. *Strongly Convex.* $f(x) = x^2$. Convex with $f'' = 2$, so 2-strongly convex, and $\frac{1}{2}|f'(x)|^2 = 2x^2 = 2f(x)$, so 2-PL.
2. *Non-Strongly Convex and PL.* $f(x) = x^2 + 3\sin^2(x)$. [KNS16] Here $f''(x) = 2 + 6\cos 2x \in [-4, 8]$, so it is not convex (hence not strongly convex), while numerically $\inf_{x \neq 0} \frac{|f'(x)|^2}{2f(x)} \approx 0.1755$ (near $x \approx 2.2$), so it is PL with $\mu \approx 0.1755$.
3. *Neither Strongly Convex nor PL.* $f(x) = x^2 + 5\sin^2(x)$. Here $f''(x) = 2 + 10\cos 2x \in [-8, 12]$, so it is not convex. It has a stationary point near $x \approx 2.45$ (where $2x + 5\sin 2x = 0$) with $f > 0 = f^*$, so the PL ratio there is 0 and PL fails.



(a) x^2: strongly convex ($\mu = 2$). (b) $x^2 + 3\sin^2 x$: PL, nonconvex. (c) $x^2 + 5\sin^2 x$: not PL.

Figure 10: Graphs of x^2 and two sine perturbations of it. In (c) the perturbation is large enough to create spurious stationary points.

Here $f(x) = x^2$ is strongly convex with $\mu = 2$ since $\frac{1}{2}|f'(x)|^2 = 2f(x)$. Adding $3\sin^2 x$ destroys convexity ($f''(x) = 2 + 6\cos 2x$ is negative somewhere) but keeps the unique minimizer and PL, with a much smaller constant: numerically $\mu = \inf_{x \neq 0} \frac{|f'(x)|^2}{2(f(x) - f^*)} \approx 0.176$, attained near $x \approx 2.2$. Increasing the coefficient to 5 creates extra stationary points (e.g. near $x \approx 2.45$, where $2x + 5\sin 2x = 0$) that are not global minimizers, so PL fails. In several dimensions, PL functions need not have a unique minimizer either: $(x, y) \mapsto h(x + y)$ with $h(t) = t^2 + 3\sin^2 t$ is PL by the affine precomposition property (with constant ≈ 0.351) but is constant along the lines $x + y = c$, so it is not strongly convex.

Lemma 2.13 (Properties of the PL-Inequality)

Let $f : \mathbb{R}^d \to \mathbb{R}$ be differentiable and bounded below, and satisfy μ-PL.

1. *Monotonicity.* f satisfies μ'-PL for every $0 < \mu' \leq \mu$.
2. *Scaling.* For $a > 0$, af satisfies $(a\mu)$-PL. For any $c \in \mathbb{R}$, $f + c$ satisfies μ-PL.
3. *Compatibility with Smoothness.* If f is also L-smooth, then $\mu \leq L$ (unless f is constant).
4. *Gradient Lower Bound.* $\|\nabla f(x)\| \geq \sqrt{2\mu\big(f(x) - f^*\big)}$ for all x.
5. *Affine Precomposition.* If $A \in \mathbb{R}^{m \times d}$ is onto and $b \in \mathbb{R}^m$, then $\varphi(x) = f(Ax + b)$ for $f : \mathbb{R}^m \to \mathbb{R}$ satisfies $(\mu\lambda_{\min}(AA^T))$-PL, where $\lambda_{\min}(AA^T) = \inf_{\|v\|=1} v^T AA^T v$. If f is also L-smooth, then φ is $L\|A\|_{\text{op}}^2$-smooth (this part does not need A onto).

Proof. Listed.

1. *Monotonicity.* Trivial.

2. *Scaling.* Since $\inf(af+c) = af^* + c$ for $a > 0$,
$$\tfrac{1}{2}\|\nabla(af)(x)\|^2 = a^2 \cdot \tfrac{1}{2}\|\nabla f(x)\|^2 \geq a^2\mu\big(f(x) - f^*\big) = a\mu\big(af(x) - \inf(af)\big) \tag{251}$$
and adding the constant c changes neither side.
3. *Compatibility with Smoothness.* By the descent lemma, for any x,
$$f^* \leq f\big(x - \tfrac{1}{L}\nabla f(x)\big) \leq f(x) - \frac{1}{2L}\|\nabla f(x)\|^2 \tag{252}$$
so $\|\nabla f(x)\|^2 \leq 2L\big(f(x) - f^*\big)$. Combining with PL, $2\mu\big(f(x) - f^*\big) \leq 2L\big(f(x) - f^*\big)$ for all x. If f is not constant, then it is not constantly equal to its infimum, so we may pick x with $f(x) > f^*$ and divide to get $\mu \leq L$.
4. *Gradient Lower Bound.* Trivial.
5. *Affine Precomposition.* Since A is onto, $x \mapsto Ax + b$ is onto $\mathbb{R}^m$, and so $\inf \varphi = f^*$. Also AA^T is positive definite so $\lambda_{\min}(AA^T) > 0$, and for any v, $\|A^T v\|^2 = v^T AA^T v \geq \lambda_{\min}(AA^T)\|v\|^2$. Therefore, with $z = Ax + b$ and $\nabla\varphi(x) = A^T\nabla f(z)$,
$$\tfrac{1}{2}\|\nabla\varphi(x)\|^2 \geq \lambda_{\min}(AA^T) \cdot \tfrac{1}{2}\|\nabla f(z)\|^2 \geq \mu\lambda_{\min}(AA^T)\big(f(z) - f^*\big) = \mu\lambda_{\min}(AA^T)\big(\varphi(x) - \inf\varphi\big) \tag{253}$$
For smoothness, $\|\nabla\varphi(x) - \nabla\varphi(x')\| = \|A^T(\nabla f(Ax+b) - \nabla f(Ax'+b))\| \leq \|A\|_{\text{op}} L\|A(x-x')\| \leq L\|A\|_{\text{op}}^2\|x - x'\|$.

Theorem 2.14 (Linear Convergence of Objective Value on GD Under PL Inequality)

Suppose that f is L-smooth and satisfies μ-PL. Then, the iterates of GD $w_{k+1} = w_k - \frac{1}{L}\nabla f(w_k)$ satisfy
$$f(w_k) - f^* \leq \left(1 - \frac{\mu}{L}\right)^k \big(f(w_0) - f^*\big) \tag{254}$$
Therefore, for any $\epsilon > 0$, we have
$$k \geq \frac{L}{\mu}\log\left(\frac{f(w_0) - f^*}{\epsilon}\right) \implies f(w_k) - f^* \leq \epsilon \tag{255}$$
i.e. $k \gtrsim \kappa\log(\epsilon^{-1}) \implies f(w_k) - f^* \leq \epsilon$.

Proof. Using the descent lemma first and then μ-PL second, we get
$$\begin{aligned} f(w_k) - f^* &\leq f(w_{k-1}) - f^* - \frac{1}{2L}\|\nabla f(w_{k-1})\|^2 && (256)\\ &\leq f(w_{k-1}) - f^* - \frac{\mu}{L}\big(f(w_{k-1}) - f^*\big) && (257)\\ &= \big(1 - \frac{\mu}{L}\big)\big(f(w_{k-1}) - f^*\big) && (258)\\ &\leq \left(1 - \frac{\mu}{L}\right)^k \big(f(w_0) - f^*\big) && (259)\end{aligned}$$
where on the last step, we recursed.

Corollary 2.15 (One-Shot Minimum for GD)

If f is L-smooth and L-PL, then for any initial $w_0 \in \mathbb{R}^d$, GD will find the global minimum in 1 step.

Proof. By substituting into the kth iteration function suboptimality bound, we get

$$f(w_1) - f^* \leq 0 \implies f(w_1) = f^* \tag{260}$$

Theorem 2.16 (Linear Convergence of GD in Overparameterized Linear Regression)

Consider the linear regression model where we want to optimize

$$f_n(w) = \frac{1}{2n}\sum_{i=1}^{n}(w^T x_i - y_i)^2 = \frac{1}{2n}\|Xw - y\|^2 \tag{261}$$

where $d > n$ (overparamterized) and $\text{rank}(X) = n$ (independent inputs). Define

$$S^* = \{w \in \mathbb{R}^d \colon Xw = y\}, \quad f^* = \min f_n(w) = 0 \tag{262}$$

Suppose we ran GD with stepsize $\eta_k = \frac{n}{\|X\|_{\text{op}}^2} = \frac{1}{L}$ on the loss f_n. Then, its iterates satisfy

$$f_n(w_k) - f^* = f_n(w_k) \leq \left(1 - \frac{\lambda_{\min}(XX^T)}{\lambda_{\max}(XX^T)}\right)^k f_n(w_0) \tag{263}$$

Proof. Let $f(z) = \frac{1}{2n}\|z\|^2$. Then, $\nabla f(z) = \frac{1}{n}z$, and so f is both $\frac{1}{n}$-smooth and $\frac{1}{n}$-PL. Therefore, we can apply our properties of affine precomposition (applied with $A = X$ and $b = -y$) to show that $\frac{1}{2n}\|Xw - y\|^2$ is both $\frac{\|X\|^2}{n}$-smooth—i.e. $\frac{\lambda_{\max}(XX^T)}{n}$-smooth—and $\frac{\lambda_{\min}(XX^T)}{n}$-PL. Therefore, fix some $w^* \in S^*$, and applying our linear convergence result gives us

$$f_n(w_k) - f_n(w^*) \leq \left(1 - \frac{\lambda_{\min}(XX^T)}{\lambda_{\max}(XX^T)}\right)^k \big(f_n(w_0) - f_n(w^*)\big) \tag{264}$$

Since $f_n(w^*) = 0$ for every $w^* \in S^*$, this is the claim.

Now we try to establish lower bounds on function classes of PL functions. Note that the lower bound result for strongly convex functions hold since strong convexity implies PL.

Theorem 2.17 (Convergence Lower Bound of GD on Strongly Convex Functions)

Let $L, \mu, \Delta > 0$, $\kappa = \frac{L}{\mu} \geq 1$, and $K \geq 1$. Then, there exists a μ-PL, L-smooth function $f : \mathbb{R}^{2K} \to \mathbb{R}$ with $f(w^*) = 0$ and $f(0) = \Delta$ such that every method started at $w_0 = 0$ and satisfying

$$w_{k+1} \in \text{span}\{\nabla f(w_0), \ldots, \nabla f(w_k)\}^a \tag{265}$$

obeys

$$\frac{f(w_K) - f^*}{\Delta} \geq \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1}\right)^{2K} \tag{266}$$

The same holds for the "best" iterate. Whenever $\sqrt{\kappa} \geq 4$ and $\epsilon \in (0, 1)$, then

$$K \leq \frac{\sqrt{\kappa}}{8}\log\left(\frac{1}{\epsilon}\right) \implies \frac{f(w_K) - f^*}{\Delta} > \epsilon \tag{267}$$

[a]GD with $w_0 = 0$ automatically satisfies this, so we can think of this as a generalization of GD.

Proof. We can always find such a μ-strongly convex function, which is μ-PL.

For functions satisfying PL, we know that this is a larger family, so perhaps the lower bound might be higher. In fact, this is the case.

Theorem 2.18 (Convergence Lower Bound of GD on μ-PL Functions[YFL23])

There is a universal constant $C > 0$ such that the following holds. Let $L \geq \mu > 0$ with $\kappa = \frac{L}{\mu} > C$, and let $\Delta, \epsilon > 0$ (with ϵ small enough relative to Δ). Then, in sufficiently high dimension, there exists an L-smooth, μ-PL (in general nonconvex) function $f : \mathbb{R}^d \to \mathbb{R}$ with $f(0) - f^* = \Delta$ such that every method started at $w_0 = 0$ and satisfying

$$w_{k+1} \in \text{span}\{\nabla f(w_0), \dots, \nabla f(w_k)\}^a \tag{268}$$

needs

$$K \geq \Omega\left(\kappa \log \frac{1}{\epsilon}\right) \quad \text{iterations to guarantee} \quad \frac{f(w_K) - f^*}{\Delta} \leq \epsilon. \tag{269}$$

The result is stated in the paper for zero-respecting algorithms and extended to general first-order algorithms there; the numerical constants are not explicit.

[a]GD with $w_0 = 0$ automatically satisfies this, so we can think of this as a generalization of GD.

Proof. See cited paper.

Hence, up to constants, we have

$$K \lesssim \kappa \log(1/\epsilon) \implies f(w_K) - f(w^*) > \epsilon \tag{270}$$

$$K \gtrsim \kappa \log(1/\epsilon) \implies f(w_K) - f(w^*) \leq \epsilon \tag{271}$$

Example 2.8 (Implicit Regularization)

Whenever a problem is ill-posed (more than one solution), it is common to add a regularier to promote structure. In overparameterized OLS linear regression, it is natural to use ridge regression.

$$w_\alpha = \underset{w}{\text{argmin}} \frac{1}{2n}\|Xw - y\|^2 + \frac{\alpha}{2}\|w\|^2 \tag{272}$$

Uniqueness of w_α follows by strong convexity. By first-order optimality conditions,

$$\frac{1}{n}X^T(Xw_\alpha - y) + \alpha w_\alpha = 0 \iff w_\alpha = (X^TX + n\alpha I)^{-1}X^Ty \tag{273}$$

This gives us a set of solutions $\{w_\alpha : \alpha > 0\}$ that approaches S^* as $\alpha \to 0^+$.

Lemma 2.19 (Limit of the Ridge Path)

Suppose $\text{rank}(X) = n$ and $d > n$. We claim

$$w^+ := \lim_{\alpha \to 0^+} w_\alpha = \underset{w \in S^*}{\text{argmin}} \|w\| \tag{274}$$

Proof. Let $X = U\Sigma V^T$ be the SVD of X. Then,

$$\lim_{\alpha \to 0^+} w_\alpha = \lim_{\alpha \to 0^+} V \text{diag}\big(\frac{\sigma_i}{\sigma_i^2 + \alpha n}\big) U^T y \tag{275}$$

$$= V\Sigma^{-1}U^T y \tag{276}$$

$$= X^T(XX^T)^{-1}y = w^+ \tag{277}$$

We claim that $w^+ \perp \ker(X)$. Let $z \in \ker(X)$.

$$(w^+)^T z = y^T(XX^T)^{-1}Xz = 0 \tag{278}$$

Also $w^+ \in S^*$ since $Xw^+ = XX^T(XX^T)^{-1}y = y$. Now let $w \in S^* \setminus \{w^+\}$. Then, $z := w - w^+ \in \ker(X) \setminus \{0\}$ (since $Xw = Xw^+ = y$) and $w = w^+ + z$, so

$$\|w\|^2 = \|w^+\|^2 + \|z\|^2 > \|w^+\|^2 \tag{279}$$

which completes the proof.

In turn, GD also favors a similar solution.

Theorem 2.20 (Implicit Bias)

Consider the linear regression model where we want to optimize

$$f_n(w) = \frac{1}{2n}\sum_{i=1}^n (w^T x_i - y_i)^2 = \frac{1}{2n}\|Xw - y\|^2 \tag{280}$$

where $\text{rank}(X) = n$ and $d > n$. Suppose we ran GD with stepsize $\eta \in (0, \frac{2n}{\|X\|_{\text{op}}^2})$ and initial iterate w_0. Then, its iterates converge to

$$w_\infty = X^T(XX^T)^{-1}y + \pi_{\ker(X)} w_0 \tag{281}$$

where $\pi_{\ker(X)}$ is the orthogonal projection. Further,

$$w_\infty = \underset{w \in S^*}{\text{argmin}} \|w - w_0\|^2 \tag{282}$$

Proof.

2.3 Momentum

Recall that badly conditioned quadratics have slow GD convergence. What we want to do is to making learning progress faster in the low curvature direction. A natural idea due to Polyak is to add momentum, called *Polyak's Heavy-Ball Method*.

Definition 2.4 (Gradient Descent with Momentum)

Let $\mathcal{F}$ be the class of differentiable functions $f : S \subset \mathbb{R}^d \to \mathbb{R}$. Fix an initial point $w_0 \in S$ and a sequence of positive step sizes (η_k). Then, **gradient descent with momentum** $\beta > 0$ is a deterministic algorithm defined by the first-order oracle

$$\mathcal{O}_f : S \to \mathbb{R} \times \mathbb{R}^d, \quad \mathcal{O}_f(x) = \big(f(x), \nabla f(x)\big) \tag{283}$$

and update maps $\Phi_k : (S \times (\mathbb{R} \times \mathbb{R}^d))^{k+1} \to S$ defined

$$w_{k+1} = \Phi_k\big(w_0, (f(w_0), \nabla f(w_0)), \dots, w_k, (f(w_k), \nabla f(w_k))\big) \tag{284}$$
$$\coloneqq w_k - \eta_k \nabla f(w_k) + \beta(w_k - w_{k-1})^a \tag{285}$$

After K iterations, the algorithm outputs $w_{\mathcal{A}_K}(f) = w_K$.

[a] where the term $\beta(w_k - w_{k-1})$ "moves in our previous direction."

[a] where the term $\beta(w_k - w_{k-1})$ "moves in our previous direction."

If the stepsize η is constant, then we can unroll the updates and get

$$w_{k+1} - w_k = -\eta \sum_{j=0}^{k} \beta^{k-j} \nabla f(w_j) \tag{286}$$

Thus, we are using all previous gradient information, albeit reweighted. Another algorithmic design principle is that we should use previous first-order information when possible. It turns out that for strongly convex quadratics, this is actually optimal.

Theorem 2.21 (Momentum GD is Optimal for Quadratics [Pol64])

Let $0 < \mu < L$, $\kappa = L/\mu$, and define

$$\mathcal{F}_{\mu,L} = \left\{ f(w) = \frac{1}{2} w^T A w - b^T w + c : A = A^T,\ \mu I \preceq A \preceq LI \right\}. \tag{287}$$

Consider heavy-ball momentum initialized at $w_{-1} = w_0$,

$$w_{k+1} = w_k - \eta \nabla f(w_k) + \beta(w_k - w_{k-1}). \tag{288}$$

With parameters

$$\eta^* = \frac{4}{(\sqrt{L} + \sqrt{\mu})^2}, \qquad \beta^* = \left(\frac{\sqrt{L} - \sqrt{\mu}}{\sqrt{L} + \sqrt{\mu}} \right)^2, \tag{289}$$

for every $f \in \mathcal{F}_{\mu,L}$ and every $k \geq 0$,

$$\|w_k - w^*\| \leq (1 + 2k) \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^k \|w_0 - w^*\|. \tag{290}$$

Consequently, for every $f \in \mathcal{F}_{\mu,L}$,

$$f(w_k) - f^* \leq \frac{L}{2}(1 + 2k)^2 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^{2k} \|w_0 - w^*\|^2. \tag{291}$$

Theorem 2.22 (Suboptimality of Heavy Ball for General Smooth Functions)

For general smooth functions, and convex functions, it is sub-optimal. (Lessard, Recht, Packard 2016)

However, for general smooth functions, momentum is not optimal, so we would like to improve it. We will do so by incorporating Nesterov's strategy. To do this, we can try to incorporate all previous information

like we did with the gradients. Recall that gradient descent tries to minimize

$$w_{k+1} = \underset{w}{\operatorname{argmin}} \left\{ \underbrace{f(w_k) + \langle \nabla f(w_k), w - w_k \rangle}_{\text{convex lower bound}} + \frac{1}{2\eta_k} \|w - w_k\|^2 \right\} \tag{292}$$

Recall that convexity gives a linear lower bound on f.

$$l_y(z) \coloneqq f(y) + \langle \nabla f(y), z - y \rangle \leq f(z) \quad \forall z \tag{293}$$

GD uses one gradient to build the model it minimizes at each iteration. Just like how momentum does, let's use all of them by taking a linear combination of such linear approximations, and then adding the quadratic anchor. At the kth iteration, we should have information on the k previous gradients. Define the following.

Definition 2.5 (Nesterov's Auxiliary Estimation Function)

Given $a_j > 0$ and $y_j \in \mathbb{R}^d$, define

$$\Phi_k(z) = \sum_{j=1}^{k} a_j l_{y_{j-1}}(z) + \frac{1}{2}\|z - w_0\|^2, \quad a_j > 0, y_j \in \mathbb{R}^d \tag{294}$$

It is not clear what the coefficients a_j (how much "weight" we should put to this gradient information) and y_j (the center points where the evaluate the gradient at) should be, but we will decide this later.

We can then bound Φ_k to get

$$\Phi_k(z) \leq A_k f(z) + \frac{1}{2}\|z - w_0\|^2, \quad \forall z, \qquad A_k = a_1 + \dots a_k \tag{295}$$

Just like how we did for GD, we would like to find $\operatorname{argmin}_z \Phi_K(z)$. Since $\Phi_K(z)$ is differentiable and strongly convex, the critical point is the unique minimizer. Evaluating gives

$$0 = \nabla \Phi_k(z) = \sum_{j=1}^{k} a_j \nabla f(y_{j-1}) + (z - w_0) \implies z = w_0 - \sum_{j=1}^{k} a_j \nabla f(y_{j-1}) \tag{296}$$

Therefore, at the kth iteration, our z_k should be the following, which can be stated recursively

$$z_k = w_0 - \sum_{j=1}^{k} a_j \nabla f(y_{j-1}) \iff z_{k+1} = z_k - a_{k+1} \nabla f(y_k) \tag{297}$$

Therefore, at the kth iteration, we wish to minimize the isotropic[3] quadratic bowl with minimizer at z_k.

$$\Phi_k(z) = \Phi_k(z_k) + \langle \nabla \Phi_k(z_k), z - z_k \rangle + \frac{1}{2}\|z - z_k\|^2 = \Phi_k(z_k) + \frac{1}{2}\|z - z_k\|^2 \tag{298}$$

To construct a convergence guarantee, we want to show something like

$$f(w_k) - f^* \leq \frac{\frac{1}{2}\|w_0 - w^*\|^2}{A_k} \iff A_k f(w_k) \leq A_k f^* + \frac{1}{2}\|w_0 - w^*\|^2 \tag{299}$$

where A_k will increase. But we already have

$$\Phi_k(w^*) \leq A_k f^* + \frac{1}{2}\|w^* - w_0\|^2 \tag{300}$$

[3]Since Φ_k is quadratic consisting of linear terms and an isotropic quadratic, so $\nabla^2 \Phi_k(z) = I$.

Therefore, if we could prove that $A_k f(w_k) \leq \Phi_k(w^*)$, then we can indeed show that

$$A_k f(w_k) \leq \Phi_k(w^*) \leq A_k f^* + \frac{1}{2}\|w^* - w_0\|^2 \tag{301}$$

So this is what we will show. However, the RHS of $A_k f(w_k) \leq \Phi_k(w^*)$ depends on the unknown minimizer w^*. Instead, we impose the stronger condition that

$$A_k f(w_k) \leq \min_z \Phi_k(z) = \Phi_k(z_k) \tag{302}$$

If this holds, then we get

$$A_k f(w_k) \leq \Phi_k(z_k) \leq \Phi_k(w^*) \leq A_k f^* + \frac{1}{2}\|w_0 - w^*\|^2 \implies f(w_k) - f^* \leq \frac{\|w_0 - w^*\|^2}{2A_k} \tag{303}$$

So our job is to now choose the evaluation points y_k and weights a_{k+1} such that the inequality holds. Therefore, the general idea is to construct 3 sequences of points in $\mathbb{R}^d$.

1. The w_k which are the weight updates.
2. The z_k which are the optima of our auxiliary estimate function.
3. The y_k which are the evaluation points of our gradients.

So, let's assume we always update

$$z_{k+1} = z_k - a\nabla f(y), \quad w_{k+1} = y - \frac{1}{L}\nabla f(y) \tag{304}$$

A question is that suppose that $A_k f(w_k) \leq \min_z \Phi_k(z) = \Phi(z_k)$ holds. Then, how do we set a and y to ensure $A_{k+1} f(w_{k+1}) \leq \min_z \Phi_{k+1}(z) = \Phi(z_{k+1})$?

Lemma 2.23 (One-Step Estimate)

Suppose $f : \mathbb{R}^d \to \mathbb{R}$ is convex and L-smooth, and assume that $A_k f(w_k) \leq \Phi_k(z_k)$. Consider the updates

$$z_{k+1} = z_k - a_{k+1}\nabla f(y_k), \qquad w_{k+1} = y_k - \frac{1}{L}\nabla f(y_k), \tag{305}$$

where $A_{k+1} = A_k + a_{k+1}$. Then,

$$\Phi_{k+1}(z_{k+1}) \geq A_{k+1} f(w_{k+1}) + \langle \nabla f(y_k), A_k(w_k - y_k) + a_{k+1}(z_k - y_k)\rangle + \frac{1}{2}\Big(\frac{A_{k+1}}{L} - a_{k+1}^2\Big)\|\nabla f(y_k)\|^2. \tag{306}$$

In particular,

$$y_k = \frac{A_k w_k + a_{k+1} z_k}{A_{k+1}}, a_{k+1}^2 \leq \frac{A_{k+1}}{L} \implies A_{k+1} f(w_{k+1}) \leq \Phi_{k+1}(z_{k+1}). \tag{307}$$

Proof. By definition of Φ_{k+1} and the inductive hypothesis,

$$\begin{aligned}
\Phi_{k+1}(z_{k+1}) &= \Phi_k(z_{k+1}) + a_{k+1} l_{y_k}(z_{k+1}) && (308)\\
&= \Phi_k(z_k) + \frac{1}{2}\|z_{k+1} - z_k\|^2 + a_{k+1}\left(f(y_k) + \langle \nabla f(y_k), z_{k+1} - y_k\rangle\right) && (309)\\
&\geq A_k f(w_k) + \frac{1}{2}\|z_{k+1} - z_k\|^2 + a_{k+1}\left(f(y_k) + \langle \nabla f(y_k), z_{k+1} - y_k\rangle\right). && (310)
\end{aligned}$$

Substituting $z_{k+1} = z_k - a_{k+1}\nabla f(y_k)$ gives

$$\Phi_{k+1}(z_{k+1}) \geq A_k f(w_k) + a_{k+1} f(y_k) + a_{k+1}\langle \nabla f(y_k), z_k - y_k\rangle \tag{311}$$

$$- \frac{a_{k+1}^2}{2}\|\nabla f(y_k)\|^2. \tag{312}$$

By convexity,

$$f(w_k) \geq f(y_k) + \langle \nabla f(y_k), w_k - y_k\rangle. \tag{313}$$

Therefore,

$$\Phi_{k+1}(z_{k+1}) \geq A_{k+1} f(y_k) + \langle \nabla f(y_k), A_k(w_k - y_k) + a_{k+1}(z_k - y_k)\rangle \tag{314}$$

$$- \frac{a_{k+1}^2}{2}\|\nabla f(y_k)\|^2. \tag{315}$$

Finally, by the descent lemma,

$$f(w_{k+1}) \leq f(y_k) - \frac{1}{2L}\|\nabla f(y_k)\|^2. \tag{316}$$

Hence,

$$\Phi_{k+1}(z_{k+1}) \geq A_{k+1} f(w_{k+1}) + \langle \nabla f(y_k), A_k(w_k - y_k) + a_{k+1}(z_k - y_k)\rangle \tag{317}$$

$$+ \frac{1}{2}\left(\frac{A_{k+1}}{L} - a_{k+1}^2\right)\|\nabla f(y_k)\|^2. \tag{318}$$

Choosing

$$y_k = \frac{A_k w_k + a_{k+1} z_k}{A_{k+1}} \tag{319}$$

makes the inner-product term vanish. Moreover, if

$$a_{k+1}^2 \leq \frac{A_{k+1}}{L}, \tag{320}$$

the remaining term is nonnegative. Consequently,

$$A_{k+1} f(w_{k+1}) \leq \Phi_{k+1}(z_{k+1}), \tag{321}$$

as desired.

A direct consequence of this is the analysis of vanilla GD. Take

$$a_j = \frac{1}{L}, \quad z_0 = w_0 \tag{322}$$

Then, it isn't too hard to see that

$$y_k = z_k = w_k \quad \forall k \tag{323}$$

Note that if we just took $a_k = \frac{1}{L}$, then we would automatically have

$$a_{k+1}^2 = \frac{1}{L^2} \leq \frac{k+1}{L^2} = \frac{A_{k+1}}{L} \tag{324}$$

but this leaves a lot of slack. Instead, we can take $a_{k+1} = \sqrt{\frac{A_{k+1}}{L}}$, i.e. $La_{k+1}^2 = A_{k+1}$, with $A_0 = 0$. Then, since $A_{k+1} - A_k = a_{k+1}$,

$$\sqrt{A_{k+1}} - \sqrt{A_k} = \frac{a_{k+1}}{\sqrt{A_{k+1}} + \sqrt{A_k}} \geq \frac{a_{k+1}}{2\sqrt{A_{k+1}}} = \frac{1}{2\sqrt{L}} \tag{325}$$

which, telescoping from $A_0 = 0$, implies that

$$\sqrt{A_k} - \sqrt{A_0} \geq \frac{k}{2\sqrt{L}} \implies A_k \geq \frac{k^2}{4L} \tag{326}$$

This allows the A_k to increase quadratically, but the price to pay is that then the coefficients of the gradients might become too big, so z_k might diverge if we don't couple it with a much more conservative w_k. To see why this is more conservative, note that

$$\begin{aligned} w_{k+1} &= y_k - \frac{1}{L}\nabla f(y_k) && (327)\\ &= \frac{A_k w_k + a_{k+1} z_k}{A_{k+1}} - \frac{a_{k+1}^2}{A_{k+1}} \nabla f(y_k) && (328)\\ &= \frac{A_k w_k + a_{k+1} z_{k+1}}{A_{k+1}} && (329) \end{aligned}$$

This is precisely the method that Nesterov proposed in 83.

Definition 2.6 (Nesterov Accelerated Gradient Descent)

Initialize $A_0 = 0$ and $z_0 = w_0$. We set a_{k+1} recursively as the positive solution of

$$La_{k+1}^2 = A_k + a_{k+1} = A_{k+1} \tag{330}$$

and iteratively update

$$\begin{aligned} y_k &= \frac{A_k w_k + a_{k+1} z_k}{A_{k+1}} && (331)\\ w_{k+1} &= y_k - \frac{1}{L}\nabla f(y_k) && (332)\\ z_{k+1} &= z_k - a_{k+1}\nabla f(y_k) && (333) \end{aligned}$$

Theorem 2.24 (Convergence)

Suppose that $f : \mathbb{R}^d \to \mathbb{R}$ is convex, L-smooth function. Then, the iterates of AGD satisfy

$$f(w_k) - f^* \leq \frac{2L\|w_0 - w^*\|^2}{k^2} \quad \forall k \geq 1 \tag{334}$$

Proof. We first prove that $A_k f(w_k) \leq \min_z \Phi_k(z) = \Phi_k(z_k)$ holds for all k. For $k = 0$, this is trivial since $A_0 = 0 = \min \Phi_0$. Then, the one step lemma proves it for all k (note that $a_{k+1}^2 = A_{k+1}/L$ satisfies its hypothesis). Then, the fact that

$$\sqrt{A_k} - \sqrt{A_0} \geq \frac{k}{2\sqrt{L}} \implies A_k \geq \frac{k^2}{4L} \tag{335}$$

and

$$f(w_k) - f^* \leq \frac{\|w_0 - w^*\|^2}{2A_k} \tag{336}$$

gives the result.

Remarkably, we can write this method as a modified version of Polyak's heavy ball. Define

$$\lambda_0 = 0, \quad \lambda_{k+1} = \frac{1 + \sqrt{1 + 4\lambda_k^2}}{2} \tag{337}$$

Further,

$$w_{k+1} = y_k - \frac{1}{L}\nabla f(y_k), \quad y_{k+1} = w_{k+1} + \beta_k(w_{k+1} - w_k), \quad y_0 = w_0 \tag{338}$$

with $\beta_k = \frac{\lambda_{k+1}-1}{\lambda_{k+2}}$.

Exercise 2.1 (Equivalence of the Update with AGD)

Show that the iterates generated by this update are the same as those generated by AGD.

2.4 Preconditioning

Another way to speed up algorithms is by simply changing the model we minimize at each iteration. The idea is pretty simple. Recall that in GD, we simply iterate as

$$w_{k+1} \leftarrow \underset{z}{\operatorname{argmin}} \left\{ \langle \nabla f(w_k), z - w_k \rangle + \frac{1}{2\eta_k}\|z - w_k\|_2^2 \right\} \tag{339}$$

Perhaps using the Euclidean norm may not be best choice, especially since for badly conditioned quadratics, this leads to very slow convergence.

Definition 2.7 (Mahalanobis Norm)

Given a symmetric positive definite matrix $\Sigma \succ 0$, define the norm

$$\|w\|_\Sigma = \sqrt{w^T \Sigma w} \tag{340}$$

Thus, giving a sequence of matrices $P_1, P_2, \ldots$, setting the "right" geometry at iteration k, we can define the following.

Definition 2.8 (Preconditioned Gradient Descent)

$$w_{k+1} \leftarrow \underset{z}{\operatorname{argmin}} \left\{ \langle \nabla f(w_k), z - w_k \rangle + \frac{1}{2\eta_k}\|z - w_k\|_{P_k}^2 \right\} = w_k - \eta_k P_k^{-1} \nabla f(w_k) \tag{341}$$

Recall that by the spectral theorem, we can diagonalize PSD symmetric matrices as $P = UDU^T$. The preconditioning rescales certain directions via D^{-1}, hopefully making conditioning better. There are several natrual questions. First, does preconditioning actually help? How do we design good preconditioners? Are they efficient? Let's establish some definitions first.

Definition 2.9 (L_P-Smooth)

We say that differentiable $f : \mathbb{R}^d \to \mathbb{R}$ is L_P-smooth with respect to $P \succ 0$ if

$$\|\nabla f(w_1) - \nabla f(w_2)\|_{P^{-1}} \leq L_P \|w_1 - w_2\|_P \quad \forall w_1, w_2 \in \mathbb{R}^d \tag{342}$$

Definition 2.10 (μ_P-Strongly Convex)

$f : \mathbb{R}^d \to \mathbb{R}$ is μ_P-strongly convex w.r.t. $P \succ 0$ if

$$\frac{\mu_P}{2}\|w_1 - w_2\|_P^2 + \langle \nabla f(w_1), w_2 - w_1 \rangle + f(w_1) \leq f(w_2) \quad \forall w_1, w_2 \tag{343}$$

Definition 2.11 (μ_P-PL)

$f : \mathbb{R}^d \to \mathbb{R}$ satisfies μ_P-PL w.r.t. $P \succ 0$ if

$$\mu_P(f(w) - f^*) \leq \frac{1}{2}\|\nabla f(w)\|^2_{P^{-1}} \tag{344}$$

If we fix $P_k = P$, then there is a very slick interpretation.

Lemma 2.25 (Scale Change)

Let P be a symmetric and positive-definite. $f : \mathbb{R}^d \to \mathbb{R}$ be differentiable, and $\varphi(z) \coloneqq f(P^{-1/2}z)$. Initialize w_0 and $z_0 = P^{1/2}w_0$. Then, consider the following.

1. *Iterates of PGD.* Take the iterates of PGD of f from w_0.

$$w_{k+1} = w_k - \eta_k P^{-1}\nabla f(w_k) \tag{345}$$

2. *Iterates of GD.* Take the iterates of GD of φ from z_0.

$$z_{k+1} = z_k - \eta_k \nabla \varphi(z_k) \tag{346}$$

Then,

$$w_k = P^{-1/2}z_k, \quad f(w_k) = \varphi(z_k) \tag{347}$$

This can be seen as the commuting diagram.

$$\begin{array}{ccc} z_0 & \xrightarrow{P^{-1/2}} & w_0 \\ {\scriptstyle \text{(GD) on } \varphi}\downarrow & & \downarrow{\scriptstyle \text{(PGD) on } f} \\ z_K & \xleftarrow[P^{1/2}]{} & w_K \end{array} \tag{348}$$

Proof. This is trivial using induction.

From this result, we can easily derive the convergence bounds.

Theorem 2.26 (Convergence to Criticality of PGD)

Let $f : \mathbb{R}^d \to \mathbb{R}$ be L_P-smooth w.r.t. $P \succ 0$, $f^* = f^* > -\infty$, and run PGD with $\eta_k = 1/L_P$. Then,

$$\min_{k \in [K-1]} \|\nabla f(w_k)\|^2_{P^{-1}} \leq \frac{2L_P(f(w_0) - f^*)}{K} \tag{349}$$

Proof.

Theorem 2.27 (Linear Convergence of Objective Value on PGD Under PL Inequality)

Let $f : \mathbb{R}^d \to \mathbb{R}$ be L_P-smooth w.r.t. $P \succ 0$, $f^* = f^* > -\infty$, and run PGD with $\eta_k = 1/L_P$. If f is μ_P-PL w.r.t. P, then

$$f(w_k) - f^* \leq \left(1 - \frac{\mu_P}{L_P}\right)^k \big(f(w_0) - f^*\big) \quad \forall k \tag{350}$$

Proof.

Example 2.9 (Quadratic)

Performing PGD with preconditioning matrix P on the quadratic

$$f(z) = \frac{1}{2} z^T A z + b^T z + c, \quad A \succ 0, b \in \mathbb{R}^d, c \in \mathbb{R} \tag{351}$$

is equivalent to performing GD on the function

$$\varphi(w) = f(P^{-1/2} w) = \frac{1}{2} w^T (P^{-1/2} A P^{-1/2}) w + (b^T P^{-1/2}) w + c \tag{352}$$

which we know is μ-PL and L-smooth. So,

$$\mu_P = \lambda_{\min}(P^{-1/2} A P^{-1/2}), \quad L_P = \lambda_{\max}(P^{-1/2} A P^{-1/2}) \tag{353}$$

Thus, PGD is faster than GD when

$$\kappa(A) > \kappa(P^{-1/2} A P^{-1/2}) \tag{354}$$

The best preconditioner would be $P = A$, but this corresponds to inverting a potentially arbitrary matrix, which has complexity $\Theta(d^3)$, which is very expensive.

2.5 Saddle Points

We started with GD, and we said that if we only have smoothness, we can make sure that the norm of the gradient goes to 0. If we have more structure like SC or PL, then we will have fast convergence. We then talked about when is it that we have local convergence. But we have only covered nonconvex problems where the critical points are local minima. This is what PL tells us. So when do first order methods converge to local/global minimizers only? In neural networks, we often just initialize at random and it seems to work. So we want to figure out why.

Today, we will "show" that GD avoids certain critical points, which are called *strict saddles*.

Definition 2.12 (Strict Saddle)

A point $w_\#$ is a **strict saddle point** if $\nabla f(w_\#) = 0$ and $\lambda_{\min}\big(\nabla^2 f(w_\#)\big) < 0$.[a]

[a] Note that a maximizer could be a saddle. We just require some direction to point down.

Example 2.10

Consider the function

$$f(w, z) = -(\alpha/2) w^2 + (\beta/2) z^2, \quad \alpha, \beta > 0 \tag{355}$$

From the simulation, we can see that when we initialize the optimizer to the form $(0, z)$, then it gets stuck at this saddle point. Fortunately, this set is measure 0.

Fubini's theorem from measure theory implies the Fubini's theorem in the notes. Now let's talk about escaping strict saddles.

Theorem 2.28 (Escaping Strict Saddles)

Let $f : \mathbb{R}^d \to \mathbb{R}$ be a C^2 function with an L-Lipschitz gradient. Suppose we ran GD with $\eta \in (0, \frac{1}{L})$ and w_0 is picked at random w.r.t. a continuous density. Then, for any strict saddle $w_\#$, we have

$$\mathbb{P}(\lim_{k\to+\infty} w_k = w_\#) = 0 \tag{356}$$

Note that this limit may not converge at all.

Proof. The idea is to take each critical point, and think of the set of points that will converge onto that point. WLOG, assume $w_\# = 0$ (since we can translate) and the Hessian is diagonal.

$$\nabla^2 f(0) = \begin{pmatrix} D_- & 0 \\ 0 & D_+ \end{pmatrix}, \quad D_- \prec 0, D_+ \succeq 0 \tag{357}$$

Define the GD map as $T(w) = w - \eta\nabla f(w)$. Define

$$\gamma = \lambda_{\min}(-D_-) > 0 \tag{358}$$

By continuity of the Hessian, there is a radius $r > 0$ s.t.

$$\|\nabla^2 f(w) - \nabla^2 f(0)\|_{\text{op}} \le \frac{\gamma}{4}, \quad w \in \underbrace{\overline{B_r}(0)}_{K} \tag{359}$$

which is equivalent to

$$\|\nabla T(w) - \nabla T(0)\|_{\text{op}} \le \frac{\eta\gamma}{4} \tag{360}$$

The idea is to divide up the measure 0 "line" into a closer set and a further set. Set

$$S = \{w \in K : T^k(w) \in K \forall k\} = \bigcap_{k=0}^{\infty} (T^k)^{-1}(K) \tag{361}$$

Finally, let

$$\mathcal{C}(w_\#) = \{w_0 \mid T^k(w_0) \to w_\#\} \tag{362}$$

Our goal is to show that this set has measure 0. Any trajectory that converges to $w_\#$ eventually stays in S. Thus,

$$\mathcal{C}(w_\#) \subset \bigcup_{j=0}^{\infty} (T^j)^{-1}(S) \tag{363}$$

and we wish to show that the RHS has measure 0. We do this in multiple steps.

1. *S has measure* 0. The idea is that given any two points in the ball K, the points must get further and further away, and so at most 1 point can stay in the ball. Since the Hessian is of the diagonal form above, we must have

$$\nabla T(0) = \begin{pmatrix} I - \eta D_- & \\ & I - \eta D_+ \end{pmatrix} \tag{364}$$

where the first and second diagonal blocks are $m \times m$ and $(d-m)\times(d-m)$ matrices. In particular, for $u \in \mathbb{R}^m, v \in \mathbb{R}^{d-m}$,

$$\|(I - \eta D_-)u\| \ge (1 + \eta\gamma)\|u\| \tag{365}$$

where $\gamma = \lambda_{\min}(-D_-)$, so that $1 + \eta\gamma$ is the minimum eigenvalue of $I - \eta D_-$. We also know that (since $0 \preceq \eta D_+ \preceq \eta L I \preceq I$)

$$\|(I - \eta D_+)v\| \le \|v\| \tag{366}$$

Consider four points

$$w = (u, v), \tilde{w} = (\tilde{u}, \tilde{v}), T(w) = (u^+, v^+), T(\tilde{w}) = (\tilde{u}^+, \tilde{v}^+) \tag{367}$$

plus the four distances

$$U = \|u - \tilde{u}\|, V = \|v - \tilde{v}\|, U^+ = \|u^+ - \tilde{u}^+\|, V^+ = \|v^+ - \tilde{v}^+\| \tag{368}$$

Include an image here from the notes. So what I want to show that if $d(w, \tilde{w}) > 0$, then $d(w^+, \tilde{w}^+)$ gets bigger. We claim that if $U \geq V$, then we have $U^+ \geq V^+$, and $U^+ \geq \big(1 + \frac{\eta\gamma}{2}\big)U$. So we should study how the iterates move, and a good way to study this is to linearize everything, seeing how that moves, and analyze the errors which should be small. By the FTC, we can linearize

$$T(w) - T(\tilde{w}) = \nabla T(0)(w - \tilde{w}) + \underbrace{\int_0^1 \big[\nabla T(\tilde{w} + t(w - \tilde{w})) - \nabla T(0)\big](w - \tilde{w})\, dt}_{e = (e_u, e_v)} \tag{369}$$

By definition of K,

$$\begin{aligned}
\|e\| &\leq \int_0^1 \|\nabla T(\tilde{w} + t(w - \tilde{w})) - \nabla T(0)\|_{\text{op}} \|w - \tilde{w}\|\, dt && (370)\\
&\leq \frac{\eta\gamma}{4}\|w - \tilde{w}\| && (371)\\
&\leq \frac{\eta\gamma}{4}(U + V) && (372)\\
&\leq \frac{\eta\gamma}{2}U && (373)
\end{aligned}$$

Then, using reverse triangle inequality,

$$\begin{aligned}
U^+ &= \|u^+ - \tilde{u}^+\| && (374)\\
&= \|(I - \eta D_-)(u - \tilde{u}) + e_u\| && (375)\\
&\geq \|(I - \eta D_-)(u - \tilde{u})\| - \|e_u\| && (376)\\
&\geq (1 + \eta\gamma)\underbrace{\|u - \tilde{u}\|}_{=U} - \underbrace{\|e_u\|}_{\leq \frac{\eta\gamma}{2}U} && (377)\\
&\geq \big(1 + \frac{\eta\gamma}{2}\big)U && (378)\\
&\geq V + \|e\| && (379)\\
&\geq \|(I - \eta D_+)(v - \tilde{v}) + e_v\| && (380)\\
&= V^+ && (381)
\end{aligned}$$

So by taking any pair of points $w, \tilde{w} \in S$ such that

$$w - \tilde{w} = \begin{pmatrix} q \\ 0 \end{pmatrix} \neq 0 \tag{382}$$

Inductively applying the lemma, we get

$$U_k \geq \big(1 + \frac{\eta\gamma}{2}\big)^k U_0 \implies U_k \to +\infty \tag{383}$$

This contradicts the fact that S is bounded. Therefore, we can only intersect this set at most once, and by Fubini's S has measure zero.

Given the next 2 lemmas, we can see that $(T^j)^{-1}(S)$ is measure 0. Therefore, by countable subadditivity, their countable union is measure 0. Thus, $\mathcal{C}(w_\#)$ has measure 0. Further, for any $\epsilon > 0$, there is a constant $M > 0$ s.t.

$$\int_{\{p \geq M\}} p(w_0)\, dw_0 \leq \frac{\epsilon}{2} \tag{384}$$

Then,

$$\mathbb{P}(\lim w_k = w_\#) = \int_{\mathcal{C}(w_\#)} p(w)\,dw \tag{385}$$

$$= \int_{\mathcal{C}(w_\#)} \mathbb{1}_{\{p\le M\}} p(w)\,dw + \int_{\mathcal{C}(w_\#)} \mathbb{1}_{\{p>M\}} p(w)\,dw \tag{386}$$

$$\le M \int_{\mathcal{C}(w_\#)} dw + \int_{\{p>M\}} p(w)\,dw \tag{387}$$

$$\le M \cdot \frac{\epsilon}{2M} + \frac{\epsilon}{2} = \epsilon \tag{388}$$

Note that there may be an uncountable number of saddle points.

Lemma 2.29 (A Three-Point Identity for Smooth Functions)

Another lemma is that for any pair $w, y \in \mathbb{R}^d$,

$$\|T(w) - T(y)\| \ge (1 - \eta L)\|w - y\| \tag{389}$$

This is pretty counterintuitive, but remember that $\eta \in (0, \frac{1}{L})$. Hence, T is injective and its inverse T^{-1} is $(1-\eta L)^{-1}$-Lipschitz.

Proof. Expanding,

$$\|T(w) - T(y)\| = \|w - y - \eta(\nabla f(w) - \nabla f(y))\| \tag{390}$$

$$\ge \|w - y\| - \eta\|\nabla f(w) - \nabla f(y)\| \tag{391}$$

$$\ge (1 - \eta L)\|w - y\| \tag{392}$$

where the first inequality comes from reverse triangle inequality, and the second comes from the fact that f is L-smooth.

Lemma 2.30 (Lipschitz Maps Preserve Null Sets)

Suppose $F : \mathbb{R}^d \to \mathbb{R}^d$ is L-lipschitz, and Q has measure 0. Then, $F(Q)$ has measure 0.

Some remarks. When Mateo was in grad school, this theorem was shown was in a recent paper by Jason Lee, Michael I Jordan. It should have been a result that was already published, but they couldn't find it. The gradients take exponential time to escape, where it is respect to the dimension d. There is a version of gradient descent that adds noise and escapes in polynomial time.

3 Nonsmooth Optimization

3.1 Convex Conjugacy

Definition 3.1 (Proper Functions)

Given a function $f : \mathbb{R}^d \to (-\infty, +\infty]$, it is called **proper** if it is finite somewhere. The **effective domain** is

$$\operatorname{dom}(f) \coloneqq \{x \in \mathbb{R}^d \colon f(x) < +\infty\} \tag{393}$$

Definition 3.2 (Convex, Fenchel Conjugate)

Given a proper function $f : \mathbb{R}^d \to (-\infty, +\infty]$, its **convex conjugate**, or **Fenchel conjugate**, is defined to be

$$f^* : \mathbb{R}^d \to (-\infty, +\infty], \quad f^*(y) = \sup_{x \in \mathbb{R}^d} \{\langle y, x\rangle - f(x)\} \tag{394}$$

satisfying

1. *Convexity.* f^* is convex: for all $y_1, y_2 \in \mathbb{R}^d$ and $t \in [0,1]$, $f^*(ty_1 + (1-t)y_2) \leq tf^*(y_1) + (1-t)f^*(y_2)$.
2. *Lower Semicontinuous.* f^* is lower semicontinuous.

Proof. It is a pointwise supremum of continuous affine functions of y.

This generalizes the classical Legendre transform.

Theorem 3.1 (Fenchel-Young Inequality)

Let f be a proper function and f^* its convex conjugate. For every $x \in \operatorname{dom}(f)$ and $y \in \mathbb{R}^d$[a],

$$f(x) + f^*(y) \geq \langle y, x\rangle \tag{395}$$

[a] We don't need to restrict to $y \in \operatorname{dom}(f^*)$.

Proof.

Definition 3.3 (Subgradients, Subdifferentials)

Given proper function f, a vector $y \in \mathbb{R}^d$ is a **subgradient** of f at $x \in \operatorname{dom}(f)$ if

$$f(z) \geq f(x) + \langle y, z - x\rangle \quad \forall z \in \mathbb{R}^d \tag{396}$$

The set of all subgradients at x is the **subdifferential**

$$\partial f(x) \coloneqq \{y \in \mathbb{R}^d \colon f(z) \geq f(x) + \langle y, z - x\rangle \ \forall z\} \tag{397}$$

It is well known that the mean square error of a linear map is convex. However, when we impose—say the L1 penalty—the loss function is now not differentiable at 0. The subgradient also acts as a linear approximation of f, but now at nondifferentiable points of convex functions, we have a set of linear approximations. It is clear that the subgradient of a convex function at a point where it is differentiable is uniquely the gradient ($\partial f(\mathbf{x}) = \{\nabla f(\mathbf{x})\}$), but for places like the absolute value, we can have infinite linear approximations.

Example 3.1 (Subgradients of Absolute Value Function)

The subdifferential of the absolute value function $f(x) = |x|$ at any given x is

$$\partial f(x) = \begin{cases} 1 & \text{if } x > 0 \\ [-1, 1] & \text{if } x = 0 \\ -1 & \text{if } x < 0 \end{cases} \tag{398}$$

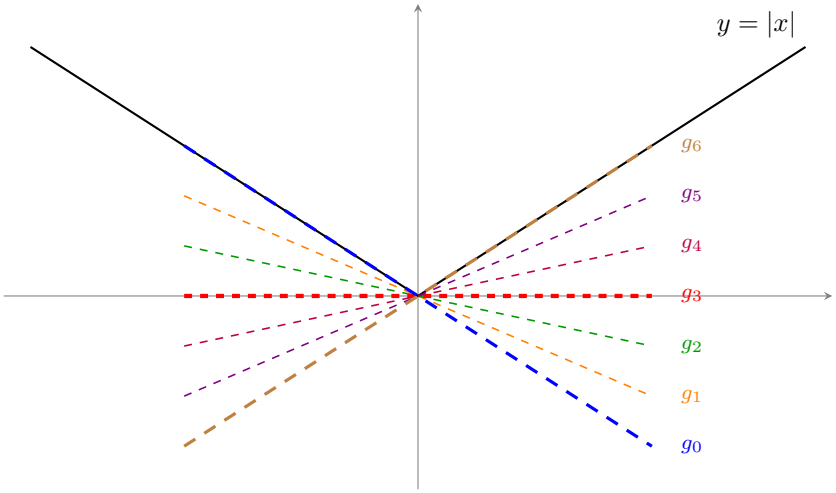


Figure 11

Given the subdifferential, thus the optimality condition for any convex $\mathbf{f}$ (differentiable or not) is

$$f(\mathbf{x}^*) = \min_{\mathbf{x}} f(\mathbf{x}) \iff \mathbf{0} \in \partial f(\mathbf{x}^*) \tag{399}$$

known as the subgradient optimality condition.

Theorem 3.2 (Equality in Fenchel-Young)

For any proper f and $x \in \text{dom}(f)$,

$$f(x) + f^*(y) = \langle y, x \rangle \iff y \in \partial f(x) \tag{400}$$

Proof.

Definition 3.4 (Biconjugate)

Given proper f and its convex conjugate f^*, the **biconjugate** of f is the conjugate of f^*.

$$f^{**}(x) = \sup_{y \in \mathbb{R}^d} \{\langle y, x \rangle - f^*(y)\} \tag{401}$$

Corollary 3.3 (Affine Lower Bound Reconstruction)

Given proper f, we have

$$f^{**} \leq f \tag{402}$$

That is, f^{**} reconstructs f from its affine lower bounds.

Theorem 3.4 (Fenchel-Moreau)

If f is proper, then the following are equivalent.
1. $f = f^{**}$.
2. f is convex and lower-semicontinuous.

Proof.

Therefore, a proper lower semicontinuous convex function can be recovered entirely from its conjugate.

Theorem 3.5 (Reciprocity of Subgradients)

If f is proper, lower-semicontinuous, and convex, then the following are equivalent.
1. $y \in \partial f(x)$.
2. $x \in \partial f^*(y)$.
3. $f(x) + f^*(y) = \langle y, x \rangle$.

Proof.

3.2 Subgradient Descent

Since $\partial f(x)$ is a *set*, the first-order oracle of the smooth case no longer makes sense: there is nothing for $\mathcal{O}_f(x)$ to return at a point where f is kinked. We must fix a choice.

Definition 3.5 (Weak Subgradient Oracle)

Let $\mathcal{F}$ be the class of proper convex functions $f : S \subset \mathbb{R}^d \to \mathbb{R}$. A **subgradient selection** for f is any map

$$g_f : S \to \mathbb{R}^d, \quad g_f(x) \in \partial f(x) \quad \forall x \in S \tag{403}$$

and the **weak subgradient oracle** associated with it is

$$\mathcal{O}_f : S \to \mathbb{R} \times \mathbb{R}^d, \quad \mathcal{O}_f(x) = \big(f(x), g_f(x)\big) \tag{404}$$

Unlike the first-order oracle, $\mathcal{O}_f$ is *not* determined by f alone, since a proper convex f admits many selections—in Figure 11, any of the g_i is a legal answer at $x = 0$. The convention is that an algorithm must work for *every* admissible selection, i.e. the oracle is free to return the worst element of $\partial f(w_k)$ at each query. This is what makes the model a fair one for proving lower bounds. At any x where f is differentiable, $\partial f(x) = \{\nabla f(x)\}$, the selection is forced, and we recover the first-order oracle exactly.

Definition 3.6 (Subgradient Descent)

Let $\mathcal{F}$ be the class of proper convex functions $f : S \subset \mathbb{R}^d \to \mathbb{R}$ that are G-Lipschitz, i.e. $\|g\| \leq G$ for every $x \in S$ and every $g \in \partial f(x)$. Fix an initial point $w_0 \in S$, a sequence of positive step sizes (η_k), and a weak subgradient oracle

$$\mathcal{O}_f : S \to \mathbb{R} \times \mathbb{R}^d, \quad \mathcal{O}_f(x) = \big(f(x), g_f(x)\big), \quad g_f(x) \in \partial f(x) \tag{405}$$

Then **subgradient descent** is a deterministic algorithm defined by $\mathcal{O}_f$ and update maps $\Phi_k : (S \times (\mathbb{R} \times \mathbb{R}^d))^{k+1} \to S$

$$w_{k+1} = \Phi_k\big(w_0, (f(w_0), g_0), \ldots, w_k, (f(w_k), g_k)\big) \coloneqq w_k - \eta_k g_k \tag{406}$$

$$= \operatorname*{argmin}_w \left\{ \underbrace{\langle g_k, w - w_k \rangle}_{\text{affine minorant}} + \underbrace{\frac{1}{2\eta_k} \|w - w_k\|^2}_{\text{anchor}} \right\} \tag{407}$$

where $g_k \coloneqq g_f(w_k) \in \partial f(w_k)$. After K iterations, the output rule Ψ_K returns either the **best iterate**

$$w_{\mathcal{A}_K}(f) = w_{k^*}, \quad k^* = \operatorname*{argmin}_{0 \leq k \leq K} f(w_k) \tag{408}$$

or the step-size weighted **ergodic average**

$$w_{\mathcal{A}_K}(f) = \bar{w}_K \coloneqq \frac{\sum_{k=0}^K \eta_k w_k}{\sum_{k=0}^K \eta_k} \tag{409}$$

The update is the verbatim analogue of gradient descent: minimize an affine model of f plus a quadratic anchor. The only change is that the model is now a genuine *global lower bound* on f, by the very definition of the subgradient, rather than a local Taylor approximation. Two other parts of the definition, however, really do differ from the smooth case, and both are forced on us rather than chosen.

First, the output rule cannot be $w_{\mathcal{A}_K}(f) = w_K$. Despite its name, subgradient descent is **not a descent method**. Where f is nondifferentiable, $-g_k$ need not be a descent direction at all, so $f(w_{k+1}) > f(w_k)$ can happen for *every* positive step size.

Example 3.2 (Subgradients of $|w_1| + 2|w_2|$)

Let $f(w_1, w_2) = |w_1| + 2|w_2|$ and take $w_k = (1, 0)$, so that $f(w_k) = 1$. Here $\partial f(w_k) = \{1\} \times [-2, 2]$, so $g_k = (1, 2)$ is a legal oracle response. Then $w_{k+1} = (1 - \eta, -2\eta)$ and

$$f(w_{k+1}) = |1 - \eta| + 4\eta = 1 + 3\eta > 1 = f(w_k) \tag{410}$$

for every $0 < \eta \leq 1$. The objective increases no matter how small the step.

So $(f(w_k))_k$ is not monotone, and the final iterate carries no guarantee on its own. Taking the running minimum or averaging is what restores one, and both achieve the rate below.[4]

Second, the step sizes cannot be a fixed constant chosen independently of the horizon. With $\eta_k = \eta$ fixed, the iterates only reach a neighborhood of the optimum: the error bound floors out at $\eta G^2/2$, and no amount of further iteration removes it. The standard requirement is that (η_k) be diminishing but nonsummable,

$$\eta_k \to 0, \quad \sum_{k=0}^{\infty} \eta_k = \infty \tag{411}$$

[4]The last iterate *does* converge under carefully chosen schedules, but this is a considerably harder result than the averaged case (Shamir and Zhang, 2013).

which drives the floor to 0 while still letting the iterates travel an unbounded distance from w_0.

Theorem 3.6 (Convergence of Subgradient Descent)

Let f be convex and G-Lipschitz with minimizer w^*, and set $R = \|w_0 - w^*\|$. Running subgradient descent for a fixed horizon K with the step size $\eta = \frac{R}{G\sqrt{K}}$ gives

$$\min_{0 \le k \le K} f(w_k) - f(w^*) \le f(\bar{w}_K) - f(w^*) \le \frac{GR}{\sqrt{K}} \tag{412}$$

Proof.

This $O(1/\sqrt{K})$ rate is far worse than the $O(1/K)$ that gradient descent achieves on L-smooth convex functions, and it cannot be improved: it is the minimax rate over the class of G-Lipschitz convex functions in the black-box first-order model, so no cleverer choice of Φ_k and Ψ_K does better. Note also that momentum buys us nothing here. The gap is the price of nonsmoothness alone, which motivates the proximal methods of the next section: rather than probing the nonsmooth part of the objective with a subgradient, we handle it exactly.

3.3 Proximal Methods

Definition 3.7 (Proximal Operator)

Given a lower semicontinuous convex function f mapping from Hilbert space X to $(-\infty, +\infty]$ (and proper), its **proximal operator** associated with a point u is defined

$$\text{prox}_{f,\tau}(u) = \underset{x}{\text{argmin}} \left(f(x) + \frac{1}{2\tau}\|x - u\|^2 \right) \tag{413}$$

where $\tau > 0$ is a parameter that scales the quadratic term. This is basically the point that minimizes the sum of $f(x)$ and the square of the Euclidean distance between x and u, scaled by $1/2\tau$.

Now given the loss function $L(\boldsymbol{\theta}) = L_{\text{obj}}(\boldsymbol{\theta}) + L_{\text{reg}}(\boldsymbol{\theta})$, we want to compute the proximal operator on the regularization loss and update that with the gradient of the smooth objective loss.

$$\boldsymbol{\theta}^{(k+1)} = \text{prox}_{L_{\text{reg}},\tau} \left[\boldsymbol{\theta}^{(k)} - \tau \nabla L_{\text{obj}}(\boldsymbol{\theta}^{(k)}) \right] \tag{414}$$

Let's compute the proximal operator of the L1 loss $h(\boldsymbol{\theta}) = \lambda\|\boldsymbol{\theta}\|_1$. We can parameterize this loss by the λ, so we will use the notation $\text{prox}_{\lambda,\tau}$ rather than $\text{prox}_{h,\tau}$.

$$\begin{aligned} \text{prox}_{\lambda,\tau}(\mathbf{u}) &= \underset{\boldsymbol{\theta}}{\text{argmin}} \left(\lambda\|\boldsymbol{\theta}\|_1 + \frac{1}{2\tau}\|\boldsymbol{\theta} - \mathbf{u}\|_2^2 \right) \\ &= \underset{\boldsymbol{\theta}}{\text{argmin}} \left(\sum_{i=1}^{n} \lambda|\theta_i| + \frac{1}{2\tau}(\theta_i - u_i)^2 \right) \end{aligned}$$

These are separable functions that can be decoupled and optimized component-wise. So, we really just want to find

$$\theta_i^* = \underset{\theta_i}{\text{argmin}} \left(\lambda|\theta_i| + \frac{1}{2\tau}(\theta_i - u_i)^2 \right) \tag{415}$$

The sum of convex functions is convex, and so we should differentiate it and find where the gradient is 0 to optimize it.

1. When $\theta_i > 0$, then we minimize $\lambda\theta_i + \frac{1}{2\tau}(\theta_i - u_i)^2$, so taking the gradient and setting to 0 gives

$$\theta_i = u_i - \lambda\tau \tag{416}$$

subject to the constraint that $\theta_i > 0$, or equivalently, that $u_i > \lambda\tau$.

2. When $\theta_i < 0$, then we minimize $-\lambda\theta_i + \frac{1}{2\tau}(\theta_i - u_i)^2$, so taking the gradient and setting to 0 gives

$$\theta_i = u_i + \lambda\tau \tag{417}$$

subject to the constraint that $\theta_i < 0$, or equivalently, that $u_i < -\lambda\tau$.

3. When $\theta_i = 0$, then we minimize $\lambda|\theta_i| + \frac{1}{2\tau}(\theta_i - u_i)^2$, which doesn't have derivative at $\theta_i = 0$. So, we can compute the subdifferential of it to get

$$0 \in \partial\left(\lambda|\theta_i| + \frac{1}{2\tau}(\theta_i - u_i)^2\right) = \lambda\partial(|\theta_i|) + \frac{1}{\tau}(\theta_i - u_i) \tag{418}$$

Now at $\theta_i = 0$, the subdifferential can be any value in $[-1, 1]$, and the above reduces to

$$0 \in \lambda[-1, 1] - \frac{1}{\tau}u_i \tag{419}$$

this is equivalent to saying that u_i/τ is contained in the interval $[-\lambda, \lambda]$, meaning that $u_i \in [-\lambda\tau, \lambda\tau]$.

Ultimately we get that

$$\text{prox}_{\lambda,\tau}(u) = \begin{cases} u - \lambda\tau & \text{if } u > \lambda\tau \\ 0 & \text{if } |u| \leq \lambda\tau \\ u + \lambda\tau & \text{if } u < -\lambda\tau \end{cases} \tag{420}$$

which can be simplified to

$$\text{prox}_{\lambda,\tau}(u) = \text{sign}(u)\max\{|u| - \lambda\tau, 0\} \tag{421}$$

4 Second-Order Optimizers

Let's take a look at one of the simplest numerical root-finders.[5]

Theorem 4.1 (Newton-Raphson Method)

Given a differentiable function $f : \mathbb{R} \to \mathbb{R}$, the sequence (x_n) defined

$$x_n = x_{n-1} - \frac{f(x_{n-1})}{f'(x_{n-1})} \tag{422}$$

for any $x_0 \in \mathbb{R}$, converges to a root of f.

Proof. Need to verify this.

Algorithm 4.1 (Secant Method)

The Secant Method approximates Newton's method by estimating the derivative using two most recent iterates instead of requiring $f'(x)$.

Require: Function $f(x)$, initial guesses x_0, x_1, tolerance ϵ, maximum iterations N_{max}

procedure SECANT($f, x_0, x_1, \epsilon, N_{max}$)
  $k \leftarrow 0$
  while $k < N_{max}$ **do**
    if $|f(x_1)| < \epsilon$ **then**
      return x_1
    end if
    $x_{new} \leftarrow x_1 - f(x_1)\dfrac{x_1 - x_0}{f(x_1) - f(x_0)}$
    $x_0 \leftarrow x_1$
    $x_1 \leftarrow x_{new}$
    $k \leftarrow k + 1$
  end while
  return Failure
end procedure

4.1 Newton's Method

Note that in GD, we have to tune the step size to ensure that our objective function decreases. If we set it too small, then we learn too slowly, and if we set it too high, then we may overshoot the minimum. We can develop an even better approximation by accounting for the curvature, aka the Hessian, and Newton's method automatically tunes this α using the second derivative.

Theorem 4.2 (Taylor's 2nd Order Bound)

If $f : \mathbb{R}^d \to \mathbb{R}$ is C^2 and has a Q-Lipschitz Hessian $\nabla^2 f$, then

$$\left| f(x+s) - f(x) - \nabla f(x)^T s - \frac{1}{2} s^T \nabla^2 f(x) s \right| \leq \frac{1}{6} Q \|s\|^3 \tag{423}$$

[5] Optimization and root finding are very similar, since to optimize f you should first find a root of f'.

Proof. Let $\theta : [0,1] \to \mathbb{R}$, $\theta(t) = f(x+ts)$, so that $\theta''(t) = s^T \nabla^2 f(x+ts)s$. We can use FOTC twice since f is C^2. So,

$$\theta(1) = \theta(0) + \int_0^1 \theta'(t)\,dt \tag{424}$$

$$= \theta(0) + \int_0^1 \left(\theta'(0) + \int_0^t \theta''(\alpha)\,d\alpha\right) dt \tag{425}$$

$$= \theta(0) + \theta'(0) + \int_0^1 \int_0^t \theta''(\alpha)\,d\alpha\,dt \tag{426}$$

Note that

$$\int_0^1 \int_0^t \theta''(0)\,d\alpha\,dt = \theta''(0) \int_0^1 \int_0^t d\alpha\,dt = \frac{1}{2}\theta''(0) \tag{427}$$

Therefore,

$$\left| f(x+s) - f(x) - \nabla f(x)^T s - \frac{1}{2} s^T \nabla^2 f(x) s \right| \tag{428}$$

$$= \left| \theta(0) + \theta'(0) + \int_0^1 \int_0^t \theta''(\alpha)\,d\alpha\,dt - \theta(0) - \theta'(0) - \frac{1}{2}\theta''(0) \right| \tag{429}$$

$$= \left| \int_0^1 \int_0^t \theta''(\alpha)\,d\alpha\,dt - \int_0^1 \int_0^t \theta''(0)\,d\alpha\,dt \right| \tag{430}$$

$$= \left| \int_0^1 \int_0^t s^T \big(\nabla^2 f(x+\alpha s) - \nabla^2 f(x)\big) s\,d\alpha\,dt \right| \tag{431}$$

$$\leq \int_0^1 \int_0^t \left| s^T \big(\nabla^2 f(x+\alpha s) - \nabla^2 f(x)\big) s \right| d\alpha\,dt \tag{432}$$

$$\leq \int_0^1 \int_0^t \left\| \nabla^2 f(x+\alpha s) - \nabla^2 f(x) \right\|_{op} \|s\|^2\,d\alpha\,dt \tag{433}$$

$$\leq \int_0^1 \int_0^t Q\|\alpha s\| \|s\|^2\,d\alpha\,dt \tag{434}$$

$$= Q\|s\|^3 \int_0^1 \int_0^t \alpha\,d\alpha\,dt \tag{435}$$

$$= \frac{1}{6} Q \|s\|^3 \tag{436}$$

Therefore, we might have try to optimize the following 2nd-order Taylor expansion.

$$w_{k+1} \leftarrow \operatorname*{argmin}_w \left\{ f(w_k) + \langle \nabla f(w_k), w - w_k \rangle + \frac{1}{2} \langle \nabla^2 f(w_k)(w - w_k), w - w_k \rangle \right\} \tag{437}$$

Note that this step isn't really well-defined since this approximation might not be convex when $\nabla^2 f(w_k)$ is not positive definite. Yet, writing $\Phi_k(w)$ for the quadratic model inside the argmin and assuming $\nabla^2 f(w_k)$ is invertible, we can use the first-order optimality condition to get

$$0 = \nabla \Phi_k(w) = \nabla f(w_k) + \nabla^2 f(w_k)(w - w_k) \implies w = w_k - \nabla^2 f(w_k)^{-1} \nabla f(w_k). \tag{438}$$

This leads to the following.

Definition 4.1 (Netwon's Method)

Let $f : \mathbb{R}^d \to \mathbb{R}$ be C^2 and initialize $w_0 \in \mathbb{R}^d$, assuming that $\nabla^2 f(w_k)$ is invertible at every iterate.

$$w_{k+1} \leftarrow w_k - \nabla^2 f(w_k)^{-1} \nabla f(w_k) \tag{439}$$

But a few caveats are that this method converges to critical points, not minimizers. [PDGB14] In particular, if we take $g(w) = -f(w)$, the method doesn't change. Second is affine invariance.

Lemma 4.3 (Affine Invariance)

Let $A \in \mathbb{R}^{d \times d}$ be invertible, $f : \mathbb{R}^d \to \mathbb{R}$ be C^2 with invertible Hessian at every iterate, and $\varphi(z) \coloneqq f(Az)$. Initialize w_0 and $z_0 = A^{-1} w_0$. Then, consider the following.

1. *Iterates of w.* Take the Newton iterates of f from w_0.

$$w_{k+1} \leftarrow w_k - \nabla^2 f(w_k)^{-1} \nabla f(w_k) \tag{440}$$

2. *Iterates of z.* Take the Newton iterates of φ from z_0.

$$z_{k+1} \leftarrow z_k - \nabla^2 \varphi(z_k)^{-1} \nabla \varphi(z_k) \tag{441}$$

Then,

$$z_k = A^{-1} w_k, \quad \forall k \tag{442}$$

$$\begin{array}{ccc} z_0 & \xrightarrow{A} & w_0 \\ {\scriptstyle \text{(NM) on } \varphi}\downarrow & & \downarrow{\scriptstyle \text{(NM) on } f} \\ z_k & \xleftarrow[A^{-1}]{} & w_k \end{array} \tag{443}$$

Proof.

Therefore, Newton's method doesn't care about change of variables. This suggests that a good guarantee shouldn't depend on the condition number. (The guarantee below is stated in terms of μ and L_H, which are not themselves affine-invariant; self-concordance is the standard affine-invariant refinement.)

Theorem 4.4 (Quadratic Convergence)

Let $f : \mathbb{R}^d \to \mathbb{R}$ be twice differentiable with critical point $w_\#$. Suppose that there exists a $r > 0$ s.t. for all $w, u \in B_r(w_\#) = B$,

1. $\|\nabla^2 f(w) - \nabla^2 f(u)\|_{\text{op}} \leq L_H \|w - u\|$.
2. $\nabla^2 f(w) \succeq \mu I$,
3. $r < 2\mu / L_H$.

Then, for all $k \geq 0$,

$$\|w_{k+1} - w_\#\| \leq \frac{L_H}{2\mu} \|w_k - w_\#\|^2 \tag{444}$$

Proof. We use induction. Suppose $w_\# \in B = B_r(w_k)$, let $e_k = w_k - w_\#$. We ultimately wish to bound $\|e_k\|$. Define

$$\varphi : [0, 1] \to \mathbb{R}^d, \quad \varphi(t) = \nabla f(w_\# + t(w_k - w_\#)) \tag{445}$$

Then, by the chain rule

$$\frac{d\varphi}{dt} = \nabla^2 f(w_\# - t(w_k - w_\#))\,(w_k - w_\#) \tag{446}$$

By the vector valued FTC and the fact that $\nabla f(w_\#) = 0$,

$$\nabla f(w_k) = \varphi(1) = \int_0^1 \nabla^2 f(w_\# + te_k)\, e_k\, dt \tag{447}$$

Then, we can see that

$$\begin{aligned}
e_{k+1} &= w_{k+1} - w_\# && (448)\\
&= w_k - w_\# - \nabla^2 f(w_k)^{-1} \nabla f(w_k) && (449)\\
&= w_k - w_\# - \nabla^2 f(w_k)^{-1} \int_0^1 \nabla^2 f(w_\# + te_k)\, e_k\, dt && (450)\\
&= \int_0^1 w_k - w_\# - \nabla^2 f(w_k)^{-1} \nabla^2 f(w_\# + te_k) e_k\, dt && (451)\\
&= \nabla^2 f(w_k)^{-1} \int_0^1 \big(\nabla^2 f(w_k) - \nabla^2 f(w_\# + te_k)\big) e_k\, dt && (452)
\end{aligned}$$

Now taking norms and applying the triangle inequality, we have

$$\begin{aligned}
\|e_{k+1}\| &\leq \|\nabla f(w_k)^{-1}\|_{\text{op}} \int_0^1 \|\nabla^2 f(w_k) - \nabla^2 f(w_\# + te_k)\|_{\text{op}} \|e_k\|\, dt && (453)\\
&\leq \frac{1}{\mu} \int_0^1 L_H \|te_k\| \|e_k\|\, dt && (454)\\
&= \frac{L_H}{2\mu} \|e_k\|^2 && (455)
\end{aligned}$$

This implies that $\frac{L_H}{2\mu}\|e_{k+1}\| \leq \frac{L_H}{4\mu^2}\|e_k\|^2$ on rolling this inequality using property 3 finishes the proof.

In general, Newton might diverge or cycle without ever converging and is very expensive $O(d^3)$. It generates beautiful fractals.

4.2 BFGS

Netwon's method is extremely effective for finding the minimum of a convex function, but there are two disadvantages. First, it is sensitive to initial conditions, and second, it is extremely expensive, with a computational complexity of $O(n^3)$ from having to invert the Hessian. An alternative family of optimizers, called *quasi-Newton* methods, try to *approximate* the Hessian (or Jacobian) with $\hat{H}f$, reducing the computational cost without too much loss in convergence rates, and simply use this approximation in the Newton's update:

$$\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - [\hat{H} f_{\mathbf{w}_k}]^{-1} \nabla_{\mathbf{w}} f(\mathbf{w}_k) \tag{456}$$

The method of the Hessian approximation varies by algorithm, but the most popular is BFGS.

So how do we approximate the Hessian with only the gradient information? With secants. Starting off with $f : \mathbb{R} \to \mathbb{R}$, let us assume that we have two points $(w_k, f(w_k))$ and $(w_{k+1}, f(w_{k+1}))$. We can approximate our derivative (gradient in dimension 1) at w_{k+1} using finite differences:

$$f'(w_{k+1})(w_{k+1} - w_k) \approx f(w_{k+1}) - f(w_k) \tag{457}$$

and doing the same for f' gives us the second derivative approximation:

$$f''(w_{k+1})(w_{k+1} - w_k) \approx f'(w_{k+1}) - f'(w_k) \tag{458}$$

which gives us the update:

$$w_{k+1} \leftarrow w_k - \frac{w_k - w_{k-1}}{f'(w_k) - f'(w_{k-1})} f'(w_k) \tag{459}$$

This method of approximating Netwon's method in one dimension by replacing the second derivative with its finite difference approximation is called the *secant method*. In multiple dimensions, given two points $\mathbf{w}_k, \mathbf{w}_{k+1}$ with their respective gradients $\nabla f(\mathbf{w}_k), \nabla f(\mathbf{w}_{k+1})$, we can approximate the Hessian $\hat{H}f_{\mathbf{w}_{k+1}} \approx D(\nabla f)_{\mathbf{w}_{k+1}}$ (which is the total derivative of the gradient) at $\mathbf{w}_{k+1}$ with the equation

$$\hat{H}f_{\mathbf{w}_{k+1}}(\mathbf{w}_{k+1} - \mathbf{w}_k) = \nabla_{\mathbf{w}} f(\mathbf{w}_{k+1}) - \nabla_{\mathbf{w}} f(\mathbf{w}_k) \tag{460}$$

This is solving the equation of form $A\mathbf{x} = \mathbf{y}$ for some linear map A. Since $\hat{H}f_{\mathbf{w}_{k+1}}$ is a symmetric $n \times n$ matrix with $n(n+1)/2$ components, there are $n(n+1)/2$ unknowns with only n equations, making this an underdetermined system. Quasi-Newton methods have to impose additional constraints, with the BFGS choosing the one where we want $\hat{H}f_{\mathbf{w}_{k+1}}$ to be as close as to $\hat{H}f_{\mathbf{w}_k}$ at each update $k+1$. Luckily, we can formalize this notion as minimizing the distance between $f_{\mathbf{w}_{k+1}}$ and $\hat{H}f_{\mathbf{w}_k}$. Therefore, we wish to find

$$\arg\min_{\hat{H}f_{\mathbf{w}_{k+1}}} \|\hat{H}f_{\mathbf{w}_{k+1}} - \hat{H}f_{\mathbf{w}_k}\|_F, \tag{461}$$

where $\|\cdot\|_F$ is the Frobenius matrix norm, subject to the restrictions that $\hat{H}f_{\mathbf{w}_{k+1}}$ be symmetric and that $\hat{H}f_{\mathbf{w}_{k+1}}(\mathbf{w}_{k+1} - \mathbf{w}_k) = \nabla_{\mathbf{w}} f(\mathbf{w}_{k+1}) - \nabla_{\mathbf{w}} f(\mathbf{w}_k)$ is satisfied. Since we have to invert it eventually, we can actually just create an optimization problem that directly computes the inverse (doing the same on the Hessian itself, rather than its inverse, leads to DFP instead). So, we wish to find

$$\arg\min_{(\hat{H}f_{\mathbf{w}_{k+1}})^{-1}} \|(\hat{H}f_{\mathbf{w}_{k+1}})^{-1} - (\hat{H}f_{\mathbf{w}_k})^{-1}\|_F \tag{462}$$

where, more precisely, $\|\cdot\|_F$ is replaced by a weighted Frobenius norm $\|W^{1/2}(\cdot)W^{1/2}\|_F$ for a weight matrix W with $W\Delta\mathbf{w}_k = \mathbf{y}_k$, subject to the restrictions that

1. $(\hat{H}f_{\mathbf{w}_{k+1}})^{-1}$ be symmetric. (Positive definiteness is not imposed as a constraint; instead, it is preserved by the update whenever $\mathbf{y}_k^T \Delta\mathbf{w}_k > 0$.)
2. $\mathbf{w}_{k+1} - \mathbf{w}_k = (\hat{H}f_{\mathbf{w}_{k+1}})^{-1}[\nabla_{\mathbf{w}} f(\mathbf{w}_{k+1}) - \nabla_{\mathbf{w}} f(\mathbf{w}_k)]$

After some complicated mathematical derivation, which we will not go over here, the problem ends up being equivalent to updating our approximate Hessian at each iteration by adding two symmetric, rank-one matrices U and V scaled by some constant, which can each be computed as an outer product of vectors with itself.

$$\hat{H}f_{\mathbf{w}_{k+1}} = \hat{H}f_{\mathbf{w}_k} + aU + bV = \hat{H}f_{\mathbf{w}_k} + a\mathbf{u}\mathbf{u}^T + b\mathbf{v}\mathbf{v}^T \tag{463}$$

where $\mathbf{u}$ and $\mathbf{v}$ are linearly independent. This addition of a rank-2 sum of matrices $aU + bV$, known as a rank-2 update, guarantees the "closeness" of $\hat{H}f_{\mathbf{w}_{k+1}}$ to $\hat{H}f_{\mathbf{w}_k}$ at each iteration. With this form, we now impose the quasi-Newton condition. Substituting $\Delta\mathbf{w}_k = \mathbf{w}_{k+1} - \mathbf{w}_k$ and $\mathbf{y}_k = \nabla_{\mathbf{w}} f(\mathbf{w}_{k+1}) - \nabla_{\mathbf{w}} f(\mathbf{w}_k)$, we have

$$\hat{H}f_{\mathbf{w}_{k+1}}\Delta\mathbf{w}_k = \hat{H}f_{\mathbf{w}_k}\Delta\mathbf{w}_k + a\mathbf{u}\mathbf{u}^T\Delta\mathbf{w}_k + b\mathbf{v}\mathbf{v}^T\Delta\mathbf{w}_k = \mathbf{y}_k \tag{464}$$

A natural choice of vectors turn out to be $\mathbf{u} = \mathbf{y}_k$ and $\mathbf{v} = \hat{H}f_{\mathbf{w}_k}\Delta\mathbf{w}_k$, and substituting this and solving gives us the optimal values

$$a = \frac{1}{\mathbf{y}_k^T \Delta\mathbf{w}_k}, \qquad b = -\frac{1}{\Delta\mathbf{w}_k^T \hat{H}f_{\mathbf{w}_k}\Delta\mathbf{w}_k} \tag{465}$$

and substituting these values back to the Hessian approximation update gives us the BFGS update:

$$\hat{H}f_{\mathbf{w}_{k+1}} = \hat{H}f_{\mathbf{w}_k} + \frac{\mathbf{y}_k\mathbf{y}_k^T}{\mathbf{y}_k^T\Delta\mathbf{w}_k} - \frac{\hat{H}f_{\mathbf{w}_k}\Delta\mathbf{w}_k\Delta\mathbf{w}_k^T\hat{H}f_{\mathbf{w}_k}}{\Delta\mathbf{w}_k^T\hat{H}f_{\mathbf{w}_k}\Delta\mathbf{w}_k} \tag{466}$$

We still need to invert this, and using the *Woodbury formula*

$$(A+UCV)^{-1} = A^{-1} - A^{-1}U(C^{-1}+VA^{-1}U)^{-1}VA^{-1} \tag{467}$$

which tells us how to invert the sum of an intertible matrix A and a rank-k correction, we can derive the iterative update of the inverse Hessian as

$$(\hat{H}f_{\mathbf{w}_{k+1}})^{-1} = \left(I - \frac{\Delta\mathbf{w}_k\mathbf{y}^T}{\mathbf{y}_k^T\Delta\mathbf{w}_k}\right)(\hat{H}f_{\mathbf{w}_k})^{-1}\left(I - \frac{\mathbf{y}_k\Delta\mathbf{w}_k^T}{\mathbf{y}_k^T\Delta\mathbf{w}_k}\right) + \frac{\Delta\mathbf{w}_k\Delta\mathbf{w}_k^T}{\mathbf{y}_k^T\Delta\mathbf{w}_k} \tag{468}$$

Remember that this is the iterative step that we want to actually compute, rather than the ones computing the regular Hessian. The whole point of using the Woodbury formula to derive an inverse update step was to do away with the tedious $O(n^3)$ computations of inverting an $n \times n$ matrix. This rank-2 update also preserves positive-definiteness, provided that $\mathbf{y}_k^T\Delta\mathbf{w}_k > 0$ (which the Wolfe conditions guarantee).

Finally, we can choose the initial inverse Hessian approximation $(\hat{H}f_{\mathbf{w}_0})^{-1}$ to be the identity I or the true inverse Hessian $(Hf_{\mathbf{w}_0})^{-1}$ (computed just once), which would lead to more efficient convergence. The pseudocode for BFGS is a bit too long and confusing to include here, but most of the time, we won't be implementing BFGS by hand; efficient and established BFGS optimizers are already in numerous packages. Like most optimizers, BFGS is not guaranteed to converge to the true global minimum.

Algorithm 4.2 (BFGS)

The BFGS algorithm maintains an approximation B_k to the inverse Hessian, updating it using gradient differences and parameter steps. This avoids explicitly computing or inverting the Hessian at each iteration.

Require: Objective $f(\theta)$, gradient ∇f, initial guess θ_0, tolerance ϵ, maximum iterations N_{max}

procedure BFGS($f, \nabla f, \theta_0$)
  $B \leftarrow I$ ▷ Initialize inverse Hessian approximation
  $\theta \leftarrow \theta_0$
  $k \leftarrow 0$
  while $k < N_{max}$ and $\|\nabla f(\theta)\| > \epsilon$ **do**
    $g \leftarrow \nabla f(\theta)$
    $d \leftarrow -Bg$
    $\alpha \leftarrow \text{LineSearch}(f, \theta, d)$ ▷ Enforce Wolfe conditions
    $\theta_{new} \leftarrow \theta + \alpha d$
    $s \leftarrow \theta_{new} - \theta$
    $y \leftarrow \nabla f(\theta_{new}) - g$
    if $y^T s \leq 0$ **then**
      break ▷ Curvature condition violated
    end if
    $B \leftarrow \left(I - \frac{sy^T}{y^T s}\right)B\left(I - \frac{ys^T}{y^T s}\right) + \frac{ss^T}{y^T s}$
    $\theta \leftarrow \theta_{new}$
    $k \leftarrow k + 1$
  end while
  return θ
end procedure

Algorithm 4.3 (Limited-memory BFGS)

L-BFGS avoids storing the full inverse Hessian by keeping only the last m update pairs (s_i, y_i). At each step, the search direction is computed using a two-loop recursion. This reduces storage from $O(n^2)$ to

$O(mn)$ and is widely used in large-scale optimization.

Require: Objective $f(\theta)$, gradient ∇f, initial θ_0, history size m, tolerance ϵ, maximum iterations N_{max}

procedure L-BFGS($f, \nabla f, \theta_0, m$)
  Initialize $\theta \leftarrow \theta_0$
  Initialize empty history lists S, Y
  $k \leftarrow 0$
  while $k < N_{max}$ and $\|\nabla f(\theta)\| > \epsilon$ **do**
    $g \leftarrow \nabla f(\theta)$
    $q \leftarrow g$
    Initialize empty list α ▷ First loop: backward pass
    for $i = |S|$ down to 1 **do**
      $\rho_i \leftarrow 1/(y_i^T s_i)$
      $\alpha_i \leftarrow \rho_i \, s_i^T q$
      $q \leftarrow q - \alpha_i y_i$
    end for
    Choose scalar H_0 (e.g., $H_0 = \frac{s_{k-1}^T y_{k-1}}{y_{k-1}^T y_{k-1}} I$ if available, else I)
    $r \leftarrow H_0 q$ ▷ Second loop: forward pass
    for $i = 1$ to $|S|$ **do**
      $\beta \leftarrow \rho_i \, y_i^T r$
      $r \leftarrow r + s_i(\alpha_i - \beta)$
    end for
    $d \leftarrow -r$ ▷ Search direction
    $\alpha \leftarrow \text{LineSearch}(f, \theta, d)$
    $\theta_{new} \leftarrow \theta + \alpha d$
    $s \leftarrow \theta_{new} - \theta$
    $y \leftarrow \nabla f(\theta_{new}) - g$
    if $y^T s > 0$ **then** ▷ Curvature condition
      Append s, y to S, Y
      if $|S| > m$ **then**
        Remove oldest pair
      end if
    end if
    $\theta \leftarrow \theta_{new}$
    $k \leftarrow k + 1$
  end while
  return θ
end procedure

4.3 Davidon-Fletcher-Powell (DFP)

Algorithm 4.4 (DFP Quasi-Newton Method)

The DFP method is a quasi-Newton optimization algorithm that maintains an approximation H_k of the inverse Hessian to update parameters efficiently.

Require: Objective $f(\theta)$, gradient ∇f, initial guess θ_0, tolerance ϵ
procedure DFP($f, \nabla f, \theta_0$)
  $H \leftarrow I$ ▷ Initialize inverse Hessian approximation
  while not converged **do**
    $g \leftarrow \nabla f(\theta)$
    $d \leftarrow -Hg$
    $\alpha \leftarrow \text{LineSearch}(f, \theta, d)$
    $\theta_{new} \leftarrow \theta + \alpha d$
    $s \leftarrow \theta_{new} - \theta$
    $y \leftarrow \nabla f(\theta_{new}) - g$
    $H \leftarrow H + \frac{ss^T}{s^T y} - \frac{Hyy^T H}{y^T H y}$
    $\theta \leftarrow \theta_{new}$
  end while
  return θ
end procedure

4.4 Broyden's Method

Algorithm 4.5 (Broyden's Method)

Broyden's method is a quasi-Newton method that updates an approximation B_k to the Jacobian matrix without computing derivatives directly.

Require: Function $F(\theta)$, initial guess θ_0, tolerance ϵ
procedure BROYDEN(F, θ_0)
  $B \leftarrow I$ ▷ Initial Jacobian approximation
  while not converged **do**
    $\Delta\theta \leftarrow -B^{-1}F(\theta)$
    $\theta_{new} \leftarrow \theta + \Delta\theta$
    $\Delta F \leftarrow F(\theta_{new}) - F(\theta)$
    $B \leftarrow B + \frac{(\Delta F - B\Delta\theta)\Delta\theta^T}{\Delta\theta^T \Delta\theta}$
    $\theta \leftarrow \theta_{new}$
  end while
  return θ
end procedure

4.5 Symmetric Rank-1 Update (SR1)

Algorithm 4.6 (Symmetric Rank-1 Update)

The SR1 update is another quasi-Newton method that maintains an approximation H_k of the inverse Hessian, using a symmetric rank-1 correction.

Require: Objective $f(\theta)$, gradient ∇f, initial guess θ_0, tolerance ϵ

procedure SR1($f, \nabla f, \theta_0$)
  $H \leftarrow I$
  while not converged **do**
    $g \leftarrow \nabla f(\theta)$
    $d \leftarrow -Hg$
    $\alpha \leftarrow \text{LineSearch}(f, \theta, d)$
    $\theta_{new} \leftarrow \theta + \alpha d$
    $s \leftarrow \theta_{new} - \theta$
    $y \leftarrow \nabla f(\theta_{new}) - g$
    if $(s - Hy)^T y \neq 0$ **then**
      $$H \leftarrow H + \frac{(s - Hy)(s - Hy)^T}{(s - Hy)^T y}$$
    end if
    $\theta \leftarrow \theta_{new}$
  end while
  return θ
end procedure

5 Constrained Optimization

5.1 Lagrangian Duality and KKT Conditions

Convex Duality.

5.2 Augmented Lagrangian Methods

ADMM

5.3 Projected Gradient Descent

5.4 Interior Point Methods

6 Stochastic Optimization

Originated from "A stochastic Approximation Method."

Remember that in GD, we have an oracle. Now, our oracle will output noisy gradients. Suppose we have some distribution D a set Z. We have access to a noisy oracle afor the gradient.

$$w \mapsto g(w, z), \quad z \sim D \tag{469}$$

We assume that this is an unbiased estimator.

$$\nabla f(w) = \mathbb{E}[g(w, z)] \tag{470}$$

This suggests a natural iterative method, called SGD.

1. Draw a fresh sample $z_k \sim D$ independent from all other z_j.
2. Update $w_{k+1} = w_k - \eta_k g(w_k, z_k)$.

Example 6.1 (Coordinate Descent)

Computing the full gradient is expensive, so we compute one partial derivative at a time. So, our procedure is

1. Draw $i \sim \text{Uniform}([d])$.
2. $g(w, i) = d\partial_i f(w) e_i$. So we take the ith partial derivative, multiply it by e_i, and then multiply the whole thing by d.

Then,

$$\mathbb{E}[g(w, i)] = \frac{1}{d} \sum_{i=1}^{d} d\partial_i f(w) e_i = \sum_{i=1}^{d} \partial_i f(w) e_i = \nabla f(w) \tag{471}$$

A more relevant example for data science.

Example 6.2 (Finite Sums)

Suppose we can write our loss as

$$f(w) = \frac{1}{n} \sum_{i=1}^{n} f_i(w) \tag{472}$$

e.g. least squares when we try to fit one data example.[a] Then, we can write

$$i \sim \text{Uniform}([n]), \quad g(w, i) = \nabla f_i(w) \tag{473}$$

We can see that

$$\mathbb{E}[g(w, i)] = \frac{1}{n} \sum_{i=1}^{n} \nabla f_i(w) = \nabla f(w) \tag{474}$$

[a] This is pretty broad and covers all ML training. This is why ML people like SGD so much.

Example 6.3 (Stochastic Programming)

In practice, we might have a continuous stream of data, so we consider the case of an infinite sum. Suppose we can write our loss as

$$f(w) = \mathbb{E}_{z \sim D}[f(w, z)] \tag{475}$$

Define

$$g(w, z) = \nabla_w f(w, z) \tag{476}$$

Under mild assumptions,[a]

$$\mathbb{E}[g(w, z)] = \nabla f(w) \tag{477}$$

[a]Leibniz rule

If we run SGD for a long time, we want to prove that the norm of the gradient will decrease in expectation. We used the descent lemma for GD, but we want to do this in expectation. We will take expectations conditioned on the past, i.e.

$$\mathbb{E}[X \mid \mathcal{F}_k], \quad \mathcal{F}_k = \sigma(w_0, z_0, \dots, z_{k-1}) \tag{478}$$

so that w_k is $\mathcal{F}_k$-measurable, while the fresh sample z_k is independent of $\mathcal{F}_k$ (and we take η_k to be deterministic, or at least $\mathcal{F}_k$-measurable).

Recall that if r.v. Y is determined by $\mathcal{F}_k$, then the *pull-out* property implies that

$$\mathbb{E}[Y X_1 \mid \mathcal{F}_k] = Y \mathbb{E}[X_1 \mid \mathcal{F}_k] \tag{479}$$

Theorem 6.1 (Expected Descent)

Assume that $f : \mathbb{R}^d \to \mathbb{R}$ is L-smooth, and $f^* = \inf f > -\infty$. Further, $g : \mathbb{R}^d \times Z \to \mathbb{R}^d$ is our noisy gradient satisfies unbiasedness—$\mathbb{E}_z[g(w, z)] = \nabla f(w)$—and

$$\mathbb{E}[\|g(w, z) - \nabla f(w)\|^2] \le \sigma^2 \quad \forall w \in \mathbb{R}^d \tag{480}$$

Then, at each k, we have

$$\mathbb{E}[f(w_{k+1}) \mid \mathcal{F}_k] \le f(w_k) - \eta_k \big(1 - \frac{L\eta_k}{2}\big) \|\nabla f(w_k)\|^2 + \frac{L\eta_k^2 \sigma^2}{2} \tag{481}$$

Note that we are assuming that the w_k is deterministic since we are conditioning on our past history $\mathcal{F}_k$.

Proof. Using Taylor's theorem,

$$f(w_{k+1}) \le f(w_k) - \eta_k \langle \nabla f(w_k), \underbrace{w_{k+1} - w_k}_{= g(w_k, z_k) = g_k} \rangle + \frac{L\eta_k^2}{2} \|g_k\|^2 \tag{482}$$

If we just take expectations, then by linearity, we know that $\mathbb{E}[\langle \nabla f(w_k), g(w_k, z_k) \rangle]$. Just continue next class.

Now let's go back to gradient descent. Usually, in machine learning, we are trying to optimize a parameteric model $\mathcal{P} = \{\mathbb{P}_\theta \mid \theta \in \Theta\}$ over a dataset $\mathcal{D} = \{x_i\}_{i=1}^n$., For example, our estimator in the maximum-likelihood approach will be

$$\delta(\mathcal{D}) = \operatorname*{argmax}_\theta \log p(\mathcal{D} \mid \theta) \tag{483}$$

i.e. we minimize the loss $L(\theta \mid \mathcal{D}) = -\log p(\mathcal{D} \mid \theta)$.

If we assume that the samples are iid, then we can decompose the gradient as

$$\nabla_\theta \log p(\mathcal{D} \mid \theta) = \sum_i \nabla_\theta \log p(d_i \mid \theta) \tag{484}$$

which scales linearly with the size of a dataset. This is not scalable with extremely large datasets, and so we must remove this $O(n)$ term. Intuitively, we can think of approximating this gradient by taking a minibatch b of $\mathcal{D}$ and computing the gradient only across that minibatch.

Theorem 6.2 (Minibatch Gradient is an Unbiased Estimator)

Let us take a minibatch of b samples $\mathcal{B} \subset \mathcal{D}$ without replacement, where $b << D$. Then, our approximation of the gradient of the log likelihood

$$\nabla_\theta \log p(\mathcal{B} \mid \theta) \coloneqq \frac{1}{b} \sum_{x \in \mathcal{B}} \nabla_\theta \log p(x \mid \theta) \tag{485}$$

is an unbiased estimator of the true gradient $\nabla_\theta \log p(\mathcal{D} \mid \theta)$. That is, setting $\mathcal{M}$ as a random variable of samples over $\mathcal{D}$, we have

$$\mathbb{E}_{\mathcal{M}}[\nabla \mathcal{L}_{\mathcal{M}}(\mathbf{w})] = \nabla \mathcal{L}(\mathbf{w}) \tag{486}$$

Proof. We use linearity of expectation for all $\mathcal{M} \subset \mathcal{D}$ of size M.

This also has the additional advantage of saving memory. You don't have to load in the gradients for the whole dataset (which may be a few TB), and can allocate just enough memory for each batch (perhaps a few GB).

Algorithm 6.1 (Stochastic Gradient Descent)

Require: Function $L(\theta)$, initialization θ_0, learning rate, batch size b, dataset $\mathcal{D}$

procedure StochasticGradientDescent(f, θ_0, b)
  $\theta \leftarrow \theta_0$
  while not converged **do**
    Sample minibatch $\mathcal{B} \subset \mathcal{D}$.
    $\theta \leftarrow \theta + \eta \nabla_\theta \log p(\mathcal{B} \mid \theta)$ ▷ ascent on the log-likelihood
  end while
  return θ
end procedure

Example 6.4 (Linear Regression)

We have assumed knowledge of gradient descent in the back propagation step in the previous section, but let's revisit this by looking at linear regression. Given our dataset $\mathcal{D} = \{\mathbf{x}^(n), y^{(n)}\}$, we are fitting a linear model of the form

$$f(\mathbf{x}; \mathbf{w}, b) = \mathbf{w}^T \mathbf{x} + b \tag{487}$$

The squared loss function is

$$\mathcal{L}(\mathbf{w}, b) = \frac{1}{2} \sum_{n=1}^{N} \big(y - f(\mathbf{x}; \mathbf{w}, b)\big)^2 = \frac{1}{2} \sum_{n=1}^{N} \big(y - (\mathbf{w}^T \mathbf{x} + b)\big)^2 \tag{488}$$

If we want to minimize this function, we can visualize it as a d-dimensional surface that we have to traverse. Recall from multivariate calculus that the gradient of an arbitrary function $\mathcal{L}$ points in the steepest direction in which $\mathcal{L}$ increases. Therefore, if we can compute the gradient of $\mathcal{L}$ and step in the *opposite direction*, then we would make the more efficient progress towards minimizing this function

(at least locally). The gradient can be solved using chain rule. Let us solve it with respect to $\mathbf{w}$ and b separately first. Beginners might find it simpler to compute the gradient element-wise.

$$\frac{\partial}{\partial w_j}\mathcal{L}(\mathbf{w}, b) = \frac{\partial}{\partial w_j}\left(\frac{1}{2}\sum_{n=1}^{N}\left(f(\mathbf{x}^{(n)}; \mathbf{w}, b) - y^{(n)}\right)^2\right) \tag{489}$$

$$= \frac{1}{2}\sum_{n=1}^{N}\frac{\partial}{\partial w_j}\left(f(\mathbf{x}^{(n)}; \mathbf{w}, b) - y^{(n)}\right)^2 \tag{490}$$

$$= \frac{1}{2}\sum_{n=1}^{N}2\left(f(\mathbf{x}^{(n)}) - y^{(n)}\right)\cdot\frac{\partial}{\partial w_j}\left(f(\mathbf{x}^{(n)}; \mathbf{w}, b) - y^{(n)}\right) \tag{491}$$

$$= \frac{1}{2}\sum_{n=1}^{N}2\left(f(\mathbf{x}^{(n)}) - y^{(n)}\right)\cdot\frac{\partial}{\partial w_j}\left(\mathbf{w}^T\mathbf{x}^{(n)} + b - y^{(n)}\right) \tag{492}$$

$$= \sum_{n=1}^{N}\left(f(\mathbf{x}^{(n)}; \mathbf{w}, b) - y^{(n)}\right)\cdot x_j^{(n)} \qquad (\text{for } j = 0, 1, \ldots, d) \tag{493}$$

As for getting the derivative w.r.t. b, we can redo the computation and get

$$\frac{\partial}{\partial b}\mathcal{L}(\mathbf{w}, b) = \sum_{n=1}^{N}\left(f(\mathbf{x}^{(n)}; \mathbf{w}, b) - y^{(n)}\right) \tag{494}$$

and in the vector form, setting $\boldsymbol{\theta} = (\mathbf{w}, b)$, we can set

$$\nabla\mathcal{L}(\mathbf{w}) = \mathbf{X}^T(\hat{\mathbf{y}} - \mathbf{y}) \tag{495}$$

$$\nabla\mathcal{L}(b) = (\hat{\mathbf{y}} - \mathbf{y})\cdot\mathbf{1} \tag{496}$$

where $\hat{\mathbf{y}}_n = f(\mathbf{x}^{(n)}; \mathbf{w}, b)$ are the predictions under our current linear model and $\mathbf{X} \in \mathbb{R}^{n\times d}$ is our design matrix. This can easily be done on a computer using a package like `numpy`.

Rather than updating the vector $\boldsymbol{\theta}$ in batches, we can apply **stochastic gradient descent** that works incrementally by updating $\boldsymbol{\theta}$ with each term in the summation. That is, rather than updating as a batch by performing the entire matrix computation by multiplying over N dimensions,

$$\nabla\mathcal{L}(\mathbf{w}) = \underbrace{\mathbf{X}^T}_{D\times N}\underbrace{(\hat{\mathbf{y}} - \mathbf{y})}_{N\times 1} \tag{497}$$

we can reduce this load by choosing a smaller subset $\mathcal{M} \subset \mathcal{D}$ of $M < N$ elements, which gives

$$\nabla\mathcal{L}_{\mathcal{M}}(\mathbf{w}) = \underbrace{\mathbf{X}_{\mathcal{M}}^T}_{D\times M}\underbrace{(\mathbf{y}\hat{}_{\mathcal{M}} - \mathbf{y})}_{\mathcal{M}}{}_{M\times 1} \tag{498}$$

Even though these estimators are noisy, we get to do much more iterations and therefore have a faster net rate of convergence. But now we have an additional choice to make. What should our minibatch size be?

Heuristic 6.1 (Choosing Batch Size)

It really depends on your hardware, but generally,

1. A smaller batch size might mean more noisy estimates, and therefore may not converge. However, it tends to escape local minima better.
2. A high batch size means more exact estimates, but it tends to get stuck in local minima.

You want to make sure that the batches fit into memory.

6.1 Convergence

6.2 Momentum and Adaptive Methods

Algorithm 6.2 (Stochastic Gradient Descent with Momentum)

This modifies vanilla SGD by keeping a running velocity term that accumulates past gradients, which smooths updates and helps escape sharp local minima.

Require: Learning rate η, momentum $0 \leq \mu < 1$, initial parameters θ_0, loss L

procedure SGDMomentum(η, μ, θ)
  $v \leftarrow 0$ ▷ Initialize velocity
  while not converged **do**
    $v \leftarrow \mu v + \nabla L(\theta)$ ▷ Decay prev. velocity and add in gradient (acceleration)
    $\theta \leftarrow \theta - \eta v$
  end while
  return θ
end procedure

Algorithm 6.3 (Stochastic Gradient Descent with Nesterov Momentum)

Nesterov momentum modifies standard momentum by computing the gradient at the *lookahead* position $\theta - \eta\mu v$, leading to faster convergence in practice.

Require: Learning rate η, momentum $0 \leq \mu < 1$, initial parameters θ_0, loss L

procedure SGDNesterov(η, μ, θ)
  $v \leftarrow 0$ ▷ Initialize velocity
  while not converged **do**
    $g \leftarrow \nabla L(\theta - \eta\mu v)$ ▷ Lookahead gradient
    $v \leftarrow \mu v + g$
    $\theta \leftarrow \theta - \eta v$
  end while
  return θ
end procedure

Adagrad. RMSProp, Adadelta, Adam.

6.3 Variance Reduction

6.4 Proximal Methods

Proximal SGD, Stochastic subgradient descent, projected SGD.

6.5 Block Coordinate Descent

Algorithm 6.4 (Block Gradient Descent)

Block Gradient Descent partitions the parameter vector θ into m disjoint blocks. At each iteration, it selects one block (cyclically or randomly) and updates only that block's parameters using the gradient, while keeping the other blocks fixed.

Require: Learning rate η, number of blocks m, partition $\theta = (\theta^{(1)}, \dots, \theta^{(m)})$, loss L

procedure BlockGradientDescent($\eta, \{\theta^{(j)}\}_{j=1}^{m}$)
  $t \leftarrow 0$
  while not converged **do**
    $j \leftarrow \text{SelectBlock}(t, m)$ ▷ e.g., cyclic: $j = (t \bmod m) + 1$
    $\theta^{(j)} \leftarrow \theta^{(j)} - \eta \nabla_{\theta^{(j)}} L(\theta^{(1)}, \dots, \theta^{(m)})$
    $t \leftarrow t + 1$
  end while
  return θ
end procedure

7 Gradient Free Methods

7.1 Simulated Annealing

Unlike the previous optimizers, *simulated annealing* is useful in finding *global* optima in the presence of multimodal functions within a usually very large discrete space $\mathcal{S}$. Given some function f defined on $\mathcal{S}$, we would like to find its global maximum. Rather than picking the "best move" using gradient information (like SGD), we propose a random move. Let us start at a state θ_k and propose a random P_{k+1}. We denote $\Delta f = f(P_{k+1}) - f(\theta_k)$.

1. If the selected move improves the solution (i.e. $\Delta f \geq 0$, then it is always accepted and we set $\theta_{k+1} \leftarrow P_{k+1}$.
2. Otherwise, when $\Delta f < 0$ it makes the move with the following acceptance probability

$$p(\theta_{k+1} \leftarrow P_{k+1} \mid \Delta f < 0) = e^{\Delta f / T(k)} \tag{499}$$

We can see that if Δf is very negative (the move is very bad), then this probability of acceptance decreases as well. Furthermore, $T(k)$ represents some sort of "temperature" that we anneal as a function of time, called the *annealing schedule*. T starts off at a high value, increasing the rate at which bad moves are accepted, which promotes exploration of $\mathcal{S}$ and allows the algorithm to travel to suboptimal areas. As T decreases, the vast majority of steps move uphill, promoting exploitation, which means that once the algorithm is in the right search space, there is no need to search other sections of the search space.

Algorithm 1 Simulated Annealing

Require: Initial θ_0, Transition kernel $\pi(\theta_{k+1} \mid \theta_k)$, Annealing schedule $T(k)$
  for $k = 0$ to K until convergence **do**
    $P_{k+1} \sim \pi(\cdot \mid \theta_k)$
    if $f(P_{k+1}) - f(\theta_k) \geq 0$ **then**
      $\theta_{k+1} \leftarrow P_{k+1}$
    else
      $\delta \sim \text{Uniform}[0, 1]$
      if $\delta < \exp[(f(P_{k+1}) - f(\theta_k))/T(k)]$ **then**
        $\theta_{k+1} \leftarrow P_{k+1}$
      else
        $\theta_{k+1} \leftarrow \theta_k$
      end if
    end if
  end for

This algorithm is very easy to implement and provides optimal solutions to a wide range of problems (e.g. TSP and nonlinear optimization), but it can take a long time to run if the annealing schedule is very long. We can stop either if T reaches a certain threshold or if we have determined convergence.

7.2 Nelder-Mead

Uses simplex.

References

[KNS16] Hamed Karimi, Julie Nutini, and Mark Schmidt. Linear convergence of gradient and proximal-gradient methods under the polyak-łojasiewicz condition. *CoRR*, abs/1608.04636, 2016.

[PDGB14] Razvan Pascanu, Yann N. Dauphin, Surya Ganguli, and Yoshua Bengio. On the saddle point problem for non-convex optimization, 2014.

[Pol64] B.T. Polyak. Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics*, 4(5):1–17, 1964.

[YFL23] Pengyun Yue, Cong Fang, and Zhouchen Lin. On the lower bound of minimizing polyak-lojasiewicz functions. In Gergely Neu and Lorenzo Rosasco, editors, *Proceedings of Thirty Sixth Conference on Learning Theory*, volume 195 of *Proceedings of Machine Learning Research*, pages 2948–2968. PMLR, 12–15 Jul 2023.